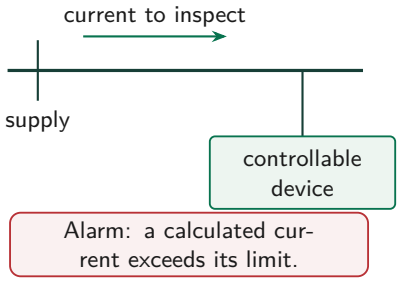
# An alarm is a question: what can the operator change?




**A physical question**

Which permitted command reduces the loading enough, and what evidence supports it?

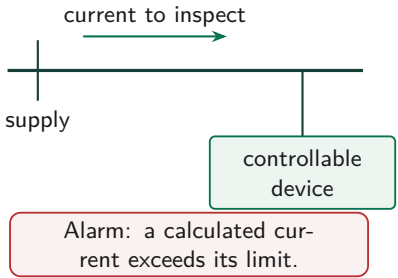
**An operator needs**

The candidate, the calculated consequence, the limit, and the model used.

We start with one small worked decision. Its arithmetic is invented so that every step can be checked by hand; the feeder models come later.

Teaching schematic; Graph lab G0–G5 declares the toy scope.

# An alarm is a question: what can the operator change?




**A physical question**

Which permitted command reduces the loading enough, and what evidence supports it?

**An operator needs**

The candidate, the calculated consequence, the limit, and the model used.

We start with one small worked decision. Its arithmetic is invented so that every step can be checked by hand; the feeder models come later.

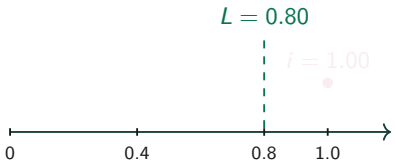
Teaching schematic; Graph lab G0–G5 declares the toy scope.

# Work the alarm: the proposed command gives too much current

$$\underbrace{i}_{\text{toy current}} = 0.4 + 0.8\ \underbrace{u}_{\text{command}}, \qquad \underbrace{L}_{\text{toy limit}} = 0.80.$$

**Proposed input:** set the dimensionless command to $u = 0.75$.

$$i = 0.4 + 0.8(0.75) = 1.00.$$




**Verdict:** $1.00 > 0.80$, so the candidate fails. The excess is $i - L = 0.20$.

Both $i$ and $L$ are normalized illustrative quantities, not amperes. A correct calculation can still return an unacceptable operating point.

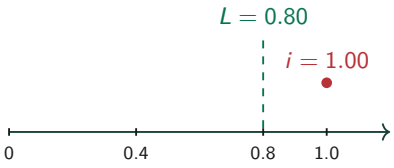
Exact response and comparison used in G2 and G4.

# Work the alarm: the proposed command gives too much current

$$\underbrace{i}_{\text{toy current}} = 0.4 + 0.8\ \underbrace{u}_{\text{command}}, \qquad \underbrace{L}_{\text{toy limit}} = 0.80.$$

**Proposed input:** set the dimensionless command to $u = 0.75$.

$$i = 0.4 + 0.8(0.75) = 1.00.$$




**Verdict:** $1.00 > 0.80$, so the candidate fails. The excess is $i - L = 0.20$.

Both $i$ and $L$ are normalized illustrative quantities, not amperes. A correct calculation can still return an unacceptable operating point.

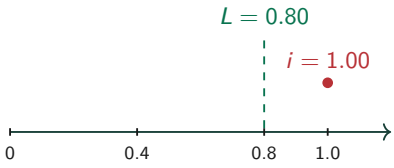
Exact response and comparison used in G2 and G4.

# Work the alarm: the proposed command gives too much current

$$\underbrace{i}_{\text{toy current}} = 0.4 + 0.8\ \underbrace{u}_{\text{command}}, \qquad \underbrace{L}_{\text{toy limit}} = 0.80.$$

**Proposed input:** set the dimensionless command to $u = 0.75$.

$$i = 0.4 + 0.8(0.75) = 1.00.$$



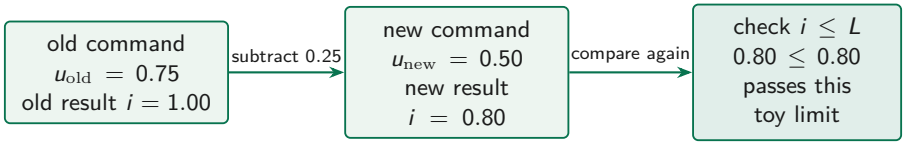

**Verdict:** $1.00 > 0.80$, so the candidate fails. The excess is $i - L = 0.20$.

Both $i$ and $L$ are normalized illustrative quantities, not amperes. A correct calculation can still return an unacceptable operating point.

Exact response and comparison used in G2 and G4.

# Repair the command, then recalculate its consequence

$$u_{\text{new}} = \max(0, u_{\text{old}} - 0.25) = \max(0, 0.75 - 0.25) = 0.50.$$

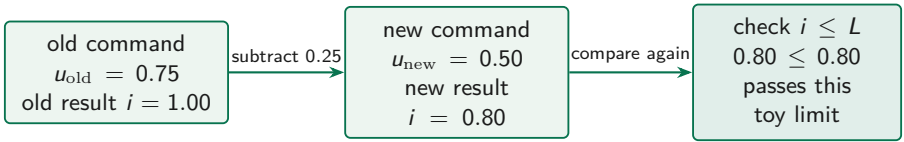



$u_{\text{old}}$ and $u_{\text{new}}$ are the commands before and after repair. The lower bound of zero prevents a negative command. The old current belongs to the old command. Reusing it would give the wrong decision about the repaired command.

Repair and simulation functions: G2.

# Repair the command, then recalculate its consequence

$$u_{\text{new}} = \max(0, u_{\text{old}} - 0.25) = \max(0, 0.75 - 0.25) = 0.50.$$

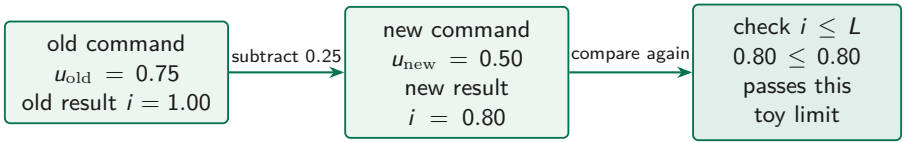



$u_{\text{old}}$ and $u_{\text{new}}$ are the commands before and after repair. The lower bound of zero prevents a negative command. The old current belongs to the old command. Reusing it would give the wrong decision about the repaired command.

Repair and simulation functions: G2.

# Repair the command, then recalculate its consequence

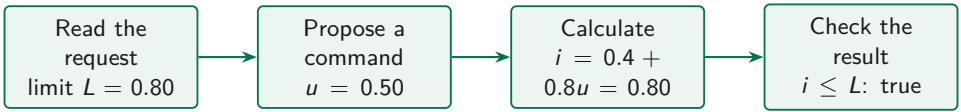
$$u_{\text{new}} = \max(0, u_{\text{old}} - 0.25) = \max(0, 0.75 - 0.25) = 0.50.$$




$u_{\text{old}}$ and $u_{\text{new}}$ are the commands before and after repair. The lower bound of zero prevents a negative command. The old current belongs to the old command. Reusing it would give the wrong decision about the repaired command.

Repair and simulation functions: G2.

# The operator's calculation already has a workflow




## A dependency

We cannot check the consequence of this command until its calculation returns.

## A record

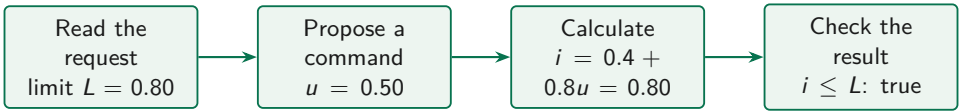
Keep the request, command, result, limit, and verdict together.

## A next decision

A pass permits review; a failure asks for repair or a hold.

Tutorial decomposition; executable functions in G1–G3.

# The operator's calculation already has a workflow




## A dependency

We cannot check the consequence of this command until its calculation returns.

## A record

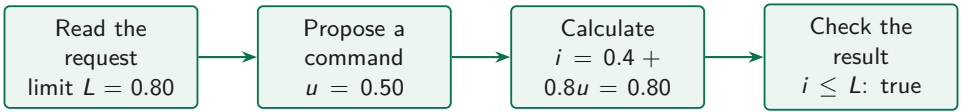
Keep the request, command, result, limit, and verdict together.

## A next decision

A pass permits review; a failure asks for repair or a hold.

Tutorial decomposition; executable functions in G1–G3.

# The operator's calculation already has a workflow




## A dependency

We cannot check the consequence of this command until its calculation returns.

## A record

Keep the request, command, result, limit, and verdict together.

## A next decision

A pass permits review; a failure asks for repair or a hold.

Tutorial decomposition; executable functions in G1–G3.

# A reviewable recommendation is more than a pass label

| Recommendation record | Value |
|---|---|
| Candidate command | $u = 0.50$ |
| Toy current | $i = 0.80$ |
| Declared limit | $L = 0.80$ |
| Model scope | invented toy equation |
| Review identity | bound to this recommendation |

**Predict:** Would the same acknowledgment apply after changing $u$?

**Explanation:** No. A changed recommendation needs its own evidence and matching readback.

The G5 exercise records a simulated acknowledgment. It neither authenticates an operator nor sends a command to equipment.

Review payload and identity test: G5; engineering handoff: L12.

# A reviewable recommendation is more than a pass label

| Recommendation record | Value |
|---|---|
| Candidate command | $u = 0.50$ |
| Toy current | $i = 0.80$ |
| Declared limit | $L = 0.80$ |
| Model scope | invented toy equation |
| Review identity | bound to this recommendation |

**Predict:** Would the same acknowledgment apply after changing $u$?

**Explanation:** No. A changed recommendation needs its own evidence and matching readback.

The G5 exercise records a simulated acknowledgment. It neither authenticates an operator nor sends a command to equipment.

Review payload and identity test: G5; engineering handoff: L12.

# A reviewable recommendation is more than a pass label

| Recommendation record | Value |
|---|---|
| Candidate command | $u = 0.50$ |
| Toy current | $i = 0.80$ |
| Declared limit | $L = 0.80$ |
| Model scope | invented toy equation |
| Review identity | bound to this recommendation |

**Predict:** Would the same acknowledgment apply after changing $u$?

**Explanation:** No. A changed recommendation needs its own evidence and matching readback.

The G5 exercise records a simulated acknowledgment. It neither authenticates an operator nor sends a command to equipment.

Review payload and identity test: G5; engineering handoff: L12.

# Three hours: understand a process, then challenge it in code

| Minutes | Question we will answer | Hands-on checkpoint |
|---|---|---|
| 0–40 | How do state, tools, and decisions connect? | G0–G5; L0–L1 |
| 40–80 | Which inverter command meets the limits? | L2–L5; Llama evidence |
| 80–90 | Break | Keep the kernels available |
| 90–115 | Which permitted restoration choice serves priorities? | L6–L7 |
| 115–142 | Which measurements can support a response? | L8–L9 |
| 142–180 | What does the evidence justify for the operator? | L10–L12; capstone |

Every exercise follows **predict, run, explain**. A fresh numerical result, an archived actual Llama reply, and a new Llama call have different labels.

Executable teaching route: graph notebook and engineering notebook.

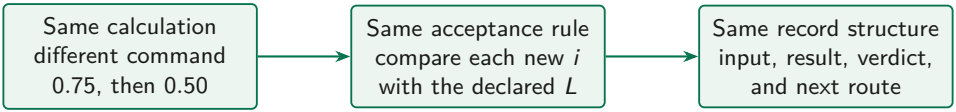
# Three hours: understand a process, then challenge it in code

| Minutes | Question we will answer | Hands-on checkpoint |
|---|---|---|
| 0–40 | How do state, tools, and decisions connect? | G0–G5; L0–L1 |
| 40–80 | Which inverter command meets the limits? | L2–L5; Llama evidence |
| 80–90 | Break | Keep the kernels available |
| 90–115 | Which permitted restoration choice serves priorities? | L6–L7 |
| 115–142 | Which measurements can support a response? | L8–L9 |
| 142–180 | What does the evidence justify for the operator? | L10–L12; capstone |

Every exercise follows **predict, run, explain**. A fresh numerical result, an archived actual Llama reply, and a new Llama call have different labels.

Executable teaching route: graph notebook and engineering notebook.

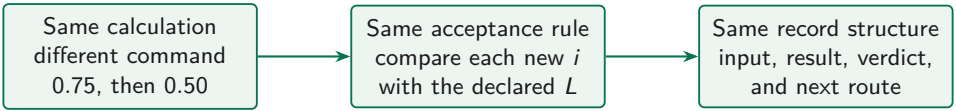
# Automate the repeated work; keep its evidence visible




A conventional program can perform all three responsibilities. An agentic design must earn its value through useful choices, clearer evidence, or adaptable task handling. We will compare numerical outcome, recorded checks, and elapsed time separately. More model calls do not by themselves establish an advantage.

Matched evaluation: PowerAgentBench-SS, 2026 preprint; tutorial baseline: G4.

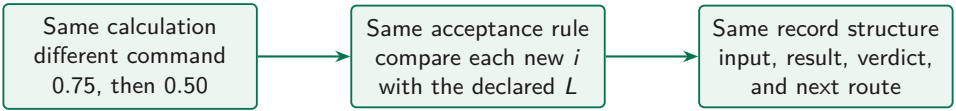
# Automate the repeated work; keep its evidence visible




A conventional program can perform all three responsibilities. An agentic design must earn its value through useful choices, clearer evidence, or adaptable task handling. We will compare numerical outcome, recorded checks, and elapsed time separately. More model calls do not by themselves establish an advantage.

Matched evaluation: PowerAgentBench-SS, 2026 preprint; tutorial baseline: G4.

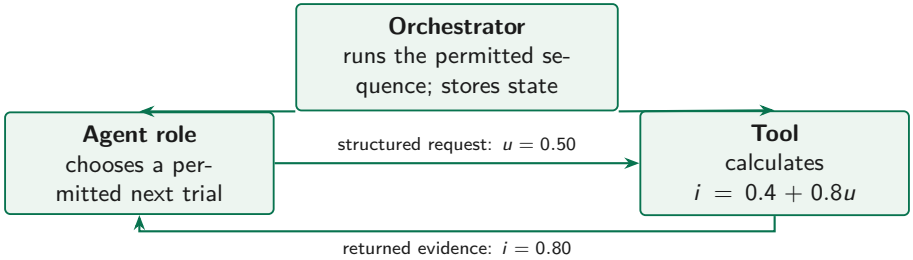
# Automate the repeated work; keep its evidence visible




A conventional program can perform all three responsibilities. An agentic design must earn its value through useful choices, clearer evidence, or adaptable task handling. We will compare numerical outcome, recorded checks, and elapsed time separately. More model calls do not by themselves establish an advantage.

Matched evaluation: PowerAgentBench-SS, 2026 preprint; tutorial baseline: G4.

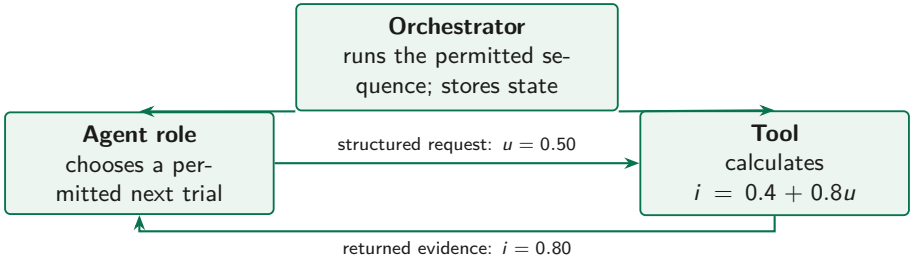
# Three names for three jobs we have already seen




A role is a responsibility, not necessarily a separate model. A numerical node can operate without an LLM; several roles can use the same Llama backbone. The graph lab uses deterministic choices. The engineering workflow later adds actual bounded Llama responses.

Role-based systems: AutoGen; tutorial separation: workflow implementation.

# Three names for three jobs we have already seen




A role is a responsibility, not necessarily a separate model. A numerical node can operate without an LLM; several roles can use the same Llama backbone. The graph lab uses deterministic choices. The engineering workflow later adds actual bounded Llama responses.

Role-based systems: AutoGen; tutorial separation: workflow implementation.

# Llama can choose follow-up work; the tool supplies the numbers

## Illustrative language interface

This candidate exceeds its limit. Choose an allowed repair or request a hold.

| Allowed operation | Meaning |
|---|---|
| `REPAIR` | try the declared repair rule |
| `HOLD` | stop and report the issue |

## What the tool still does

Calculate the repaired consequence and compare it with the unchanged limit.

## What we inspect

The chosen operation, returned result, verdict, and concise public explanation.

**This is an interface illustration, not a captured model reply.** The toy uses fixed routing; later voltage examples show actual Llama outputs and latency.

Observe–act feedback: ReAct; actual replies: L3, L10, L11.

# Llama can choose follow-up work; the tool supplies the numbers

## Illustrative language interface

This candidate exceeds its limit. Choose an allowed repair or request a hold.

| Allowed operation | Meaning |
|---|---|
| `REPAIR` | try the declared repair rule |
| `HOLD` | stop and report the issue |

## What the tool still does

Calculate the repaired consequence and compare it with the unchanged limit.

## What we inspect

The chosen operation, returned result, verdict, and concise public explanation.

**This is an interface illustration, not a captured model reply.** The toy uses fixed routing; later voltage examples show actual Llama outputs and latency.

Observe–act feedback: ReAct; actual replies: L3, L10, L11.

# Llama can choose follow-up work; the tool supplies the numbers

## Illustrative language interface

This candidate exceeds its limit. Choose an allowed repair or request a hold.

| Allowed operation | Meaning |
|---|---|
| `REPAIR` | try the declared repair rule |
| `HOLD` | stop and report the issue |

## What the tool still does

Calculate the repaired consequence and compare it with the unchanged limit.

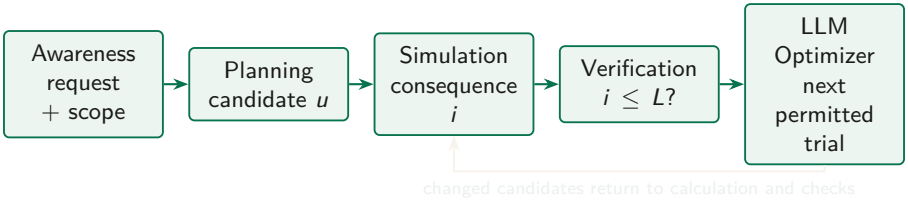
## What we inspect

The chosen operation, returned result, verdict, and concise public explanation.

**This is an interface illustration, not a captured model reply.** The toy uses fixed routing; later voltage examples show actual Llama outputs and latency.

Observe–act feedback: ReAct; actual replies: L3, L10, L11.

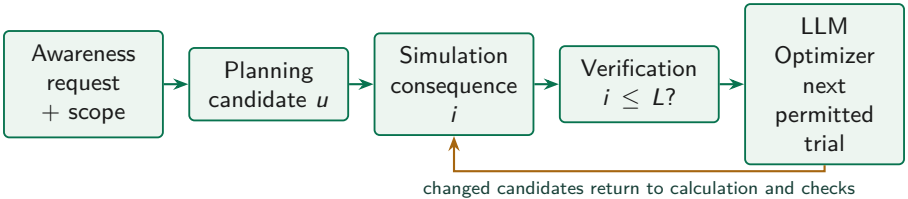
# All five proposed stages have a concrete return value




**Human review** receives the selected candidate, its evidence, alternatives, and unresolved assumptions. Planning uses a suitable numerical method for each case. The LLM Optimizer selects follow-up work; a new choice is a trial, not an already verified action.

All five stages and recorded events: executed tutorial trace.

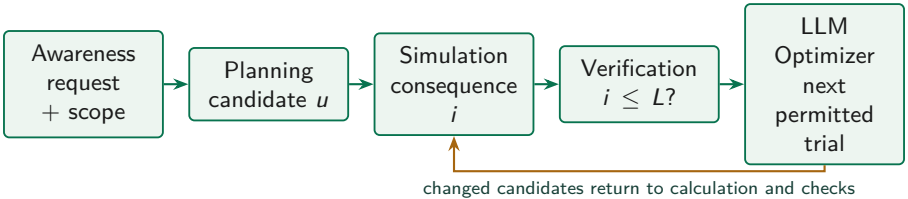
# All five proposed stages have a concrete return value




**Human review** receives the selected candidate, its evidence, alternatives, and unresolved assumptions. Planning uses a suitable numerical method for each case. The LLM Optimizer selects follow-up work; a new choice is a trial, not an already verified action.

All five stages and recorded events: executed tutorial trace.

# All five proposed stages have a concrete return value




**Human review** receives the selected candidate, its evidence, alternatives, and unresolved assumptions. Planning uses a suitable numerical method for each case. The LLM Optimizer selects follow-up work; a new choice is a trial, not an already verified action.

All five stages and recorded events: executed tutorial trace.

# The workflow reuses numerical methods; it does not replace them

| Case | Numerical work | What the workflow coordinates |
|---|---|---|
| Voltage support | MPC planning; SAC continuous-action proposal | Compare candidates, simulate, check, select a bounded trial |
| Restoration | Mixed-integer service allocation | Preserve priorities and feasible connectivity while evaluating options |
| Cyber response | State estimation, residual tests, constrained response | Distinguish suspicious evidence, supported action, and a hold |

A model-based method predicts behavior with an explicit model. A model-free policy learns an action rule from experience; training may still use a simulator. Every candidate faces declared checks. A language-model explanation is not a numerical certificate.

Methods: classroom tools; MPC; SAC.

# The workflow reuses numerical methods; it does not replace them

| Case | Numerical work | What the workflow coordinates |
|---|---|---|
| Voltage support | MPC planning; SAC continuous-action proposal | Compare candidates, simulate, check, select a bounded trial |
| Restoration | Mixed-integer service allocation | Preserve priorities and feasible connectivity while evaluating options |
| Cyber response | State estimation, residual tests, constrained response | Distinguish suspicious evidence, supported action, and a hold |

A model-based method predicts behavior with an explicit model. A model-free policy learns an action rule from experience; training may still use a simulator. Every candidate faces declared checks. A language-model explanation is not a numerical certificate.

Methods: classroom tools; MPC; SAC.

# The workflow reuses numerical methods; it does not replace them

| Case | Numerical work | What the workflow coordinates |
|---|---|---|
| Voltage support | MPC planning; SAC continuous-action proposal | Compare candidates, simulate, check, select a bounded trial |
| Restoration | Mixed-integer service allocation | Preserve priorities and feasible connectivity while evaluating options |
| Cyber response | State estimation, residual tests, constrained response | Distinguish suspicious evidence, supported action, and a hold |

A model-based method predicts behavior with an explicit model. A model-free policy learns an action rule from experience; training may still use a simulator. Every candidate faces declared checks. A language-model explanation is not a numerical certificate.

Methods: classroom tools; MPC; SAC.

# State is the record that keeps this decision coherent

**Before the repair**

| | |
|---|---|
| `command` | 0.75 |
| `current` | 1.00 |
| `limit` | 0.80 |
| `passed` | false |
| `attempts` | 0 |

**After repair and recalculation**

| | |
|---|---|
| `command` | 0.50 |
| `current` | 0.80 |
| `limit` | 0.80 |
| `passed` | true |
| `attempts` | 1 |

$s_t$ denotes the full record before workflow step $t$. A node reads that record and returns field updates; unchanged fields remain available. The limit remains 0.80. Repairing the command does not authorize relaxing its acceptance criterion.

Exact fields and updates: G1–G2.

# State is the record that keeps this decision coherent

| Before the repair | |
|---|---|
| `command` | 0.75 |
| `current` | 1.00 |
| `limit` | 0.80 |
| `passed` | false |
| `attempts` | 0 |

| After repair and recalculation | |
|---|---|
| `command` | 0.50 |
| `current` | 0.80 |
| `limit` | 0.80 |
| `passed` | true |
| `attempts` | 1 |

$s_t$ denotes the full record before workflow step $t$. A node reads that record and returns field updates; unchanged fields remain available. The limit remains 0.80. Repairing the command does not authorize relaxing its acceptance criterion.

Exact fields and updates: G1–G2.

# State is the record that keeps this decision coherent

| Before the repair | |
|---|---|
| `command` | 0.75 |
| `current` | 1.00 |
| `limit` | 0.80 |
| `passed` | false |
| `attempts` | 0 |

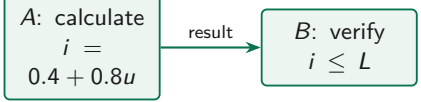
| After repair and recalculation | |
|---|---|
| `command` | 0.50 |
| `current` | 0.80 |
| `limit` | 0.80 |
| `passed` | true |
| `attempts` | 1 |

$s_t$ denotes the full record before workflow step $t$. A node reads that record and returns field updates; unchanged fields remain available. The limit remains 0.80. Repairing the command does not authorize relaxing its acceptance criterion.

Exact fields and updates: G1–G2.

# Two graphs answer different questions

**Workflow graph: what must run first?**




Nodes are operations; $A \rightarrow B$ means that $B$ requires a result from $A$.

**Electrical graph: what is connected?**



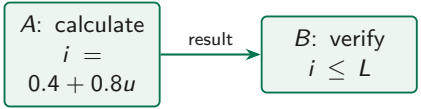

Nodes are buses; edges are lines or other network connections. Their equations describe electrical behavior.

A tool may solve the electrical graph while the workflow graph controls when that tool runs. The two drawings describe different systems. $A, B$: operations; $s$: shared record; $r(s)$: a rule returning the next operation label. The earlier $u, i, L$ remain the toy command, consequence, and limit.

LangGraph graph API; OpenDSS documentation.

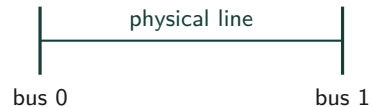
# Two graphs answer different questions

**Workflow graph: what must run first?**




Nodes are operations; $A \to B$ means that $B$ requires a result from $A$.

**Electrical graph: what is connected?**



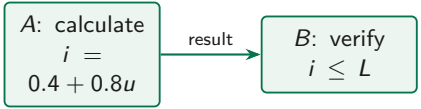

Nodes are buses; edges are lines or other network connections. Their equations describe electrical behavior.

A tool may solve the electrical graph while the workflow graph controls when that tool runs. The two drawings describe different systems. $A, B$: operations; $s$: shared record; $r(s)$: a rule returning the next operation label. The earlier $u, i, L$ remain the toy command, consequence, and limit.

LangGraph graph API; OpenDSS documentation.

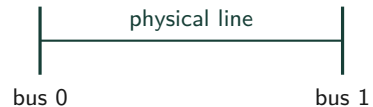
# Two graphs answer different questions

**Workflow graph: what must run first?**




Nodes are operations; $A \to B$ means that $B$ requires a result from $A$.

**Electrical graph: what is connected?**

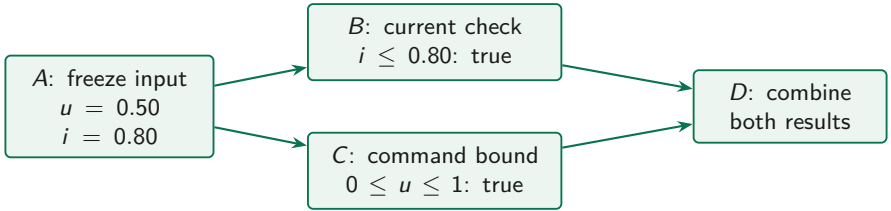



Nodes are buses; edges are lines or other network connections. Their equations describe electrical behavior.

A tool may solve the electrical graph while the workflow graph controls when that tool runs. The two drawings describe different systems. $A, B$: operations; $s$: shared record; $r(s)$: a rule returning the next operation label. The earlier $u, i, L$ remain the toy command, consequence, and limit.

LangGraph graph API; OpenDSS documentation.

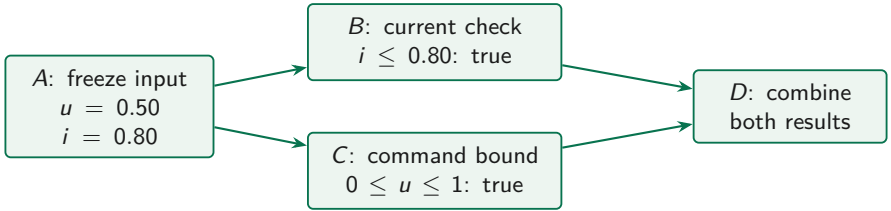
# A DAG can branch two checks and join their evidence




$\mathcal{G} = (N, E)$: $N = \{A, B, C, D\}$ is the operation set; $E$ contains the directed dependency arrows. **Directed**: arrows have direction. **Acyclic**: no directed path returns to its start. This is a DAG. This teaching example is not the G-lab topology. The join requires both results for the same frozen input.

DAG ordering: NetworkX; illustrative checks use the declared toy variables.

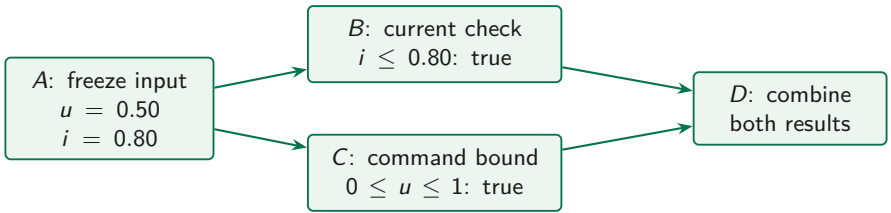
# A DAG can branch two checks and join their evidence




$\mathcal{G} = (N, E)$: $N = \{A, B, C, D\}$ is the operation set; $E$ contains the directed dependency arrows. **Directed**: arrows have direction. **Acyclic**: no directed path returns to its start. This is a DAG. This teaching example is not the G-lab topology. The join requires both results for the same frozen input.

DAG ordering: NetworkX; illustrative checks use the declared toy variables.

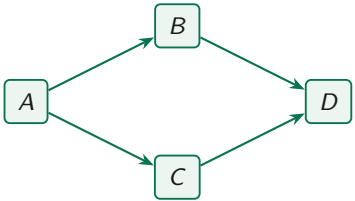
# A DAG can branch two checks and join their evidence




$\mathcal{G} = (N, E)$: $N = \{A, B, C, D\}$ is the operation set; $E$ contains the directed dependency arrows. **Directed**: arrows have direction. **Acyclic**: no directed path returns to its start. This is a DAG. This teaching example is not the G-lab topology. The join requires both results for the same frozen input.

DAG ordering: NetworkX; illustrative checks use the declared toy variables.

# A valid schedule puts both checks before the join

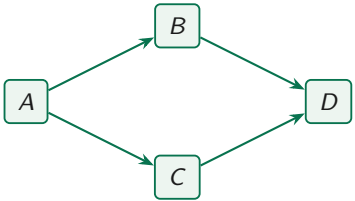



| Candidate schedule | Valid? |
|---|---|
| $A, B, C, D$ | Yes |
| $A, C, B, D$ | Yes |
| $A, B, D, C$ | No: $D$ lacks $C$ |

$A$: freeze input; $B$: current check; $C$: command bound; $D$: combine evidence. The two middle checks may exchange order. That does not establish measured speedup: execution resources and shared tool state still matter.

Topological ordering: NetworkX.

# A valid schedule puts both checks before the join

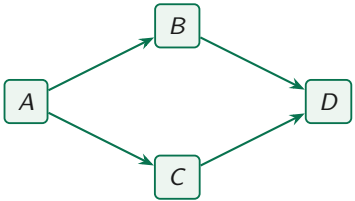



| Candidate schedule | Valid? |
|---|---|
| $A, B, C, D$ | Yes |
| $A, C, B, D$ | Yes |
| $A, B, D, C$ | No: $D$ lacks $C$ |

$A$: freeze input; $B$: current check; $C$: command bound; $D$: combine evidence. The two middle checks may exchange order. That does not establish measured speedup: execution resources and shared tool state still matter.

Topological ordering: NetworkX.

# A valid schedule puts both checks before the join

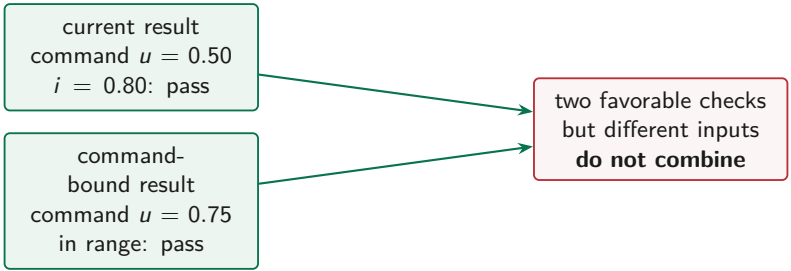



| Candidate schedule | Valid? |
|---|---|
| $A, B, C, D$ | Yes |
| $A, C, B, D$ | Yes |
| $A, B, D, C$ | No: $D$ lacks $C$ |

$A$: freeze input; $B$: current check; $C$: command bound; $D$: combine evidence. The two middle checks may exchange order. That does not establish measured speedup: execution resources and shared tool state still matter.

Topological ordering: NetworkX.

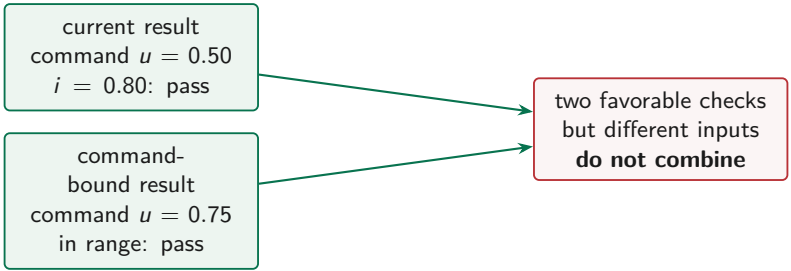
# The join must match the candidate, not merely collect two ticks




Both calculations can be individually correct. Together they do not establish the stated checks for one common candidate. Bind each result to the request revision and command. In L1, changing either makes the old binding fail.

Exact request/candidate binding exercise: L1.

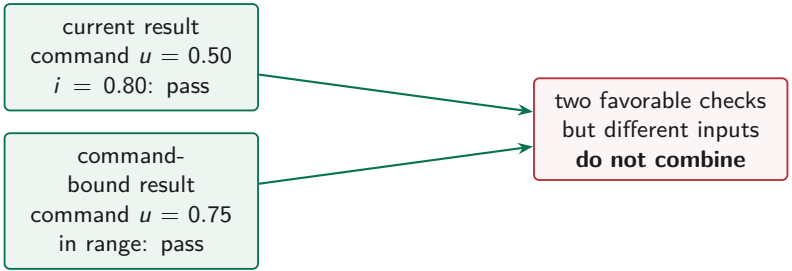
# The join must match the candidate, not merely collect two ticks




Both calculations can be individually correct. Together they do not establish the stated checks for one common candidate. Bind each result to the request revision and command. In L1, changing either makes the old binding fail.

Exact request/candidate binding exercise: L1.

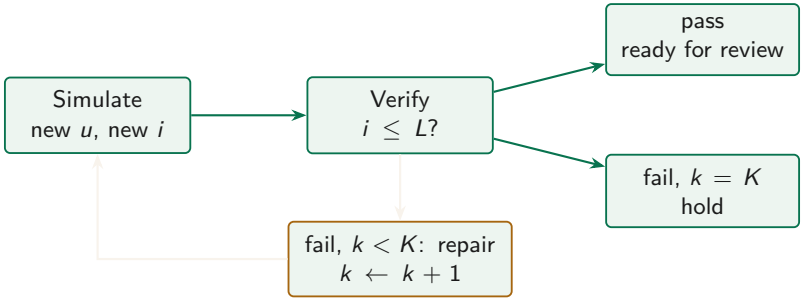
# The join must match the candidate, not merely collect two ticks




Both calculations can be individually correct. Together they do not establish the stated checks for one common candidate. Bind each result to the request revision and command. In L1, changing either makes the old binding fail.

Exact request/candidate binding exercise: L1.

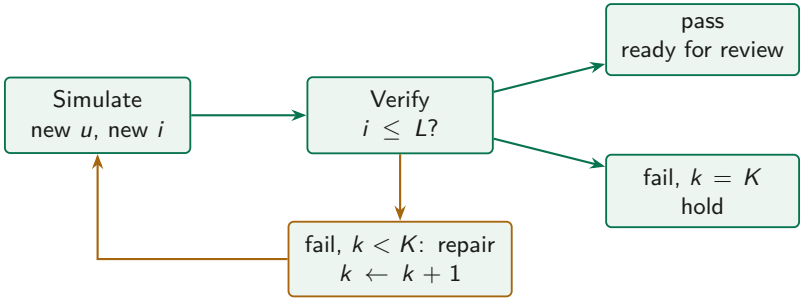
# A repair loop is a cycle, so give it a stopping rule




$k$: completed repairs; $K$: maximum repairs; both are nonnegative integers. The backward edge makes the program cyclic, not a DAG. The budget bounds attempts if tool calls return. It guarantees neither a feasible command nor a tool timeout.

Exact bounded policy: G2–G4; LangGraph.

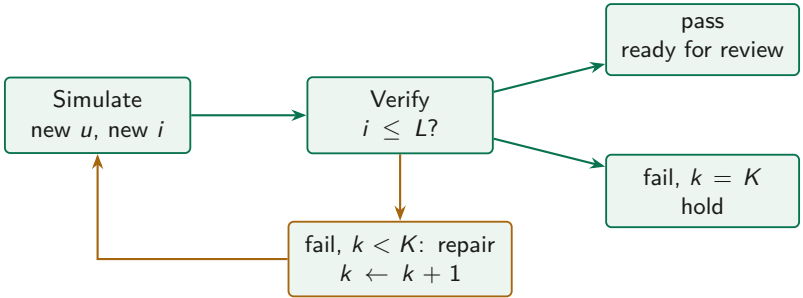
# A repair loop is a cycle, so give it a stopping rule




$k$: completed repairs; $K$: maximum repairs; both are nonnegative integers. The backward edge makes the program cyclic, not a DAG. The budget bounds attempts if tool calls return. It guarantees neither a feasible command nor a tool timeout.

Exact bounded policy: G2–G4; LangGraph.

# A repair loop is a cycle, so give it a stopping rule




$k$: completed repairs; $K$: maximum repairs; both are nonnegative integers. The backward edge makes the program cyclic, not a DAG. The budget bounds attempts if tool calls return. It guarantees neither a feasible command nor a tool timeout.

Exact bounded policy: G2–G4; LangGraph.

# Route from the verdict and the remaining budget

$$r(s) = \begin{cases} \texttt{report}, & \text{if passed is true,} \\ \texttt{repair}, & \text{if passed is false and } k < K, \\ \texttt{hold}, & \text{otherwise.} \end{cases}$$

Here $s$ is the current record; $r(s)$ returns one next-node label. $k$ is the completed repair count and $K$ is the repair budget.

**Predict:** A failed check has $k = 1$ and $K = 1$. Which route is permitted?

**Explanation:** `hold`. The budget is exhausted; equality does not permit another repair.
Verification computes the verdict. Routing decides what to do with that verdict.

The implemented route is four Python lines in G2.

# Route from the verdict and the remaining budget

$$r(s) = \begin{cases} \texttt{report}, & \text{if passed is true,} \\ \texttt{repair}, & \text{if passed is false and } k < K, \\ \texttt{hold}, & \text{otherwise.} \end{cases}$$

Here $s$ is the current record; $r(s)$ returns one next-node label. $k$ is the completed repair count and $K$ is the repair budget.

**Predict:** A failed check has $k = 1$ and $K = 1$. Which route is permitted?

**Explanation:** `hold`. The budget is exhausted; equality does not permit another repair.

Verification computes the verdict. Routing decides what to do with that verdict.

The implemented route is four Python lines in G2.

# Route from the verdict and the remaining budget

$$r(s) = \begin{cases} \texttt{report}, & \text{if passed is true,} \\ \texttt{repair}, & \text{if passed is false and } k < K, \\ \texttt{hold}, & \text{otherwise.} \end{cases}$$

Here $s$ is the current record; $r(s)$ returns one next-node label. $k$ is the completed repair count and $K$ is the repair budget.

**Predict:** A failed check has $k = 1$ and $K = 1$. Which route is permitted?

**Explanation:** `hold`. The budget is exhausted; equality does not permit another repair.
Verification computes the verdict. Routing decides what to do with that verdict.

The implemented route is four Python lines in G2.

# A state schema names the facts each node may need

| Fields | Our example | Purpose |
|---|---|---|
| `command`, `limit` | 0.75, 0.80 | identify the trial and unchanged criterion |
| `current`, `passed` | 1.00, `False` | store the latest calculation and comparison |
| `attempts`, `max_repairs` | 0, 2 | decide whether another repair is allowed |
| `trace` | ordered event list | explain how the current record was reached |

$s_{t+1} = \text{merge}(s_t, \Delta s_t)$. $s_t$: record before step $t$; $\Delta s_t$: returned field updates, not arithmetic subtraction. Python `TypedDict` describes expected keys and types. The tutorial's `initial_state` function performs explicit input-value checks.

Python TypedDict; validated inputs in graph-lab helper.

# A state schema names the facts each node may need

| Fields | Our example | Purpose |
|---|---|---|
| `command`, `limit` | 0.75, 0.80 | identify the trial and unchanged criterion |
| `current`, `passed` | 1.00, `False` | store the latest calculation and comparison |
| `attempts`, `max_repairs` | 0, 2 | decide whether another repair is allowed |
| `trace` | ordered event list | explain how the current record was reached |

$s_{t+1} = \text{merge}(s_t, \Delta s_t)$. $s_t$: record before step $t$; $\Delta s_t$: returned field updates, not arithmetic subtraction. Python `TypedDict` describes expected keys and types. The tutorial's `initial_state` function performs explicit input-value checks.

Python TypedDict; validated inputs in graph-lab helper.

# A state schema names the facts each node may need

| Fields | Our example | Purpose |
|---|---|---|
| `command`, `limit` | 0.75, 0.80 | identify the trial and unchanged criterion |
| `current`, `passed` | 1.00, `False` | store the latest calculation and comparison |
| `attempts`, `max_repairs` | 0, 2 | decide whether another repair is allowed |
| `trace` | ordered event list | explain how the current record was reached |

$s_{t+1} = \text{merge}(s_t, \Delta s_t)$. $s_t$: record before step $t$; $\Delta s_t$: returned field updates, not arithmetic subtraction. Python `TypedDict` describes expected keys and types. The tutorial's `initial_state` function performs explicit input-value checks.

Python TypedDict; validated inputs in graph-lab helper.

# A reducer states whether new information replaces or appends

**Replace the current result**

Old current 1.00; new current 0.80. Store 0.80 for the repaired command.

**Append the event history**

Old list [simulate]; new list [verify]. Store [simulate, verify].

$$R_{\text{current}}(a, b) = b \qquad R_{\text{trace}}(a, b) = a + b$$

$R$ is a reducer; $a$ is the stored value and $b$ the returned update. In the right-hand expression, $+$ means list concatenation. Preserve history without pretending an old current is the result for a new command. Parallel writes also need a meaningful merge rule.

List reducer in G1; LangGraph reducers.

# A reducer states whether new information replaces or appends

**Replace the current result**

Old current 1.00; new current 0.80. Store 0.80 for the repaired command.

**Append the event history**

Old list [simulate]; new list [verify]. Store [simulate, verify].

$$R_{\text{current}}(a, b) = b \qquad R_{\text{trace}}(a, b) = a + b$$

$R$ is a reducer; $a$ is the stored value and $b$ the returned update. In the right-hand expression, $+$ means list concatenation. Preserve history without pretending an old current is the result for a new command. Parallel writes also need a meaningful merge rule.

List reducer in G1; LangGraph reducers.

# A reducer states whether new information replaces or appends

**Replace the current result**

Old current 1.00; new current 0.80. Store 0.80 for the repaired command.

**Append the event history**

Old list [simulate]; new list [verify]. Store [simulate, verify].

$$R_{\text{current}}(a, b) = b \qquad R_{\text{trace}}(a, b) = a + b$$

$R$ is a reducer; $a$ is the stored value and $b$ the returned update. In the right-hand expression, $+$ means list concatenation. Preserve history without pretending an old current is the result for a new command. Parallel writes also need a meaningful merge rule.

List reducer in G1; LangGraph reducers.

# Three starting states test three distinct paths

| Start $(u, L, K)$ | First result | After one repair | Expected outcome |
|---|---|---|---|
| $(0.25, 0.80, 2)$ | $i = 0.60$ | not needed | ready immediately |
| $(0.75, 0.80, 2)$ | $i = 1.00$ | $u = 0.50,\ i = 0.80$ | ready after repair |
| $(0.90, 0.50, 1)$ | $i = 1.12$ | $u = 0.65,\ i = 0.92$ | hold: still above limit |

$u$: command; $L$: current limit; $K$: repair budget; $i = 0.4 + 0.8u$. Repair subtracts 0.25, bounded below by zero. The final case is useful: the process stops correctly even though it does not find an acceptable command within its budget.

These are the exact three cases executed in G4.

# Three starting states test three distinct paths

| Start $(u, L, K)$ | First result | After one repair | Expected outcome |
|---|---|---|---|
| $(0.25, 0.80, 2)$ | $i = 0.60$ | not needed | ready immediately |
| $(0.75, 0.80, 2)$ | $i = 1.00$ | $u = 0.50,\ i = 0.80$ | ready after repair |
| $(0.90, 0.50, 1)$ | $i = 1.12$ | $u = 0.65,\ i = 0.92$ | hold: still above limit |

$u$: command; $L$: current limit; $K$: repair budget; $i = 0.4 + 0.8u$. Repair subtracts 0.25, bounded below by zero. The final case is useful: the process stops correctly even though it does not find an acceptable command within its budget.

These are the exact three cases executed in G4.

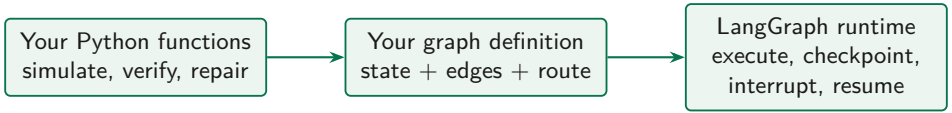
# Three starting states test three distinct paths

| Start $(u, L, K)$ | First result | After one repair | Expected outcome |
|---|---|---|---|
| $(0.25, 0.80, 2)$ | $i = 0.60$ | not needed | ready immediately |
| $(0.75, 0.80, 2)$ | $i = 1.00$ | $u = 0.50,\ i = 0.80$ | ready after repair |
| $(0.90, 0.50, 1)$ | $i = 1.12$ | $u = 0.65,\ i = 0.92$ | hold: still above limit |

$u$: command; $L$: current limit; $K$: repair budget; $i = 0.4 + 0.8u$. Repair subtracts 0.25, bounded below by zero. The final case is useful: the process stops correctly even though it does not find an acceptable command within its budget.

These are the exact three cases executed in G4.

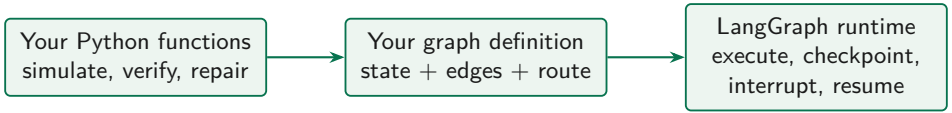
# LangGraph runs the stateful process we have already specified




The functions still contain the arithmetic. The route still contains the stopping policy. LangGraph makes their execution structure explicit. Our G notebook uses real LangGraph without an LLM or electrical simulator. The engineering notebooks remain custom Python; this is not a claim that they were migrated to LangGraph.

LangGraph overview; executed G notebook.

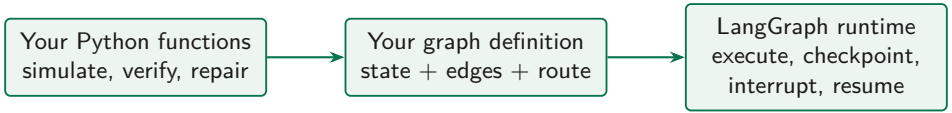
# LangGraph runs the stateful process we have already specified




The functions still contain the arithmetic. The route still contains the stopping policy. LangGraph makes their execution structure explicit. Our G notebook uses real LangGraph without an LLM or electrical simulator. The engineering notebooks remain custom Python; this is not a claim that they were migrated to LangGraph.

LangGraph overview; executed G notebook.

# LangGraph runs the stateful process we have already specified




The functions still contain the arithmetic. The route still contains the stopping policy. LangGraph makes their execution structure explicit. Our G notebook uses real LangGraph without an LLM or electrical simulator. The engineering notebooks remain custom Python; this is not a claim that they were migrated to LangGraph.

LangGraph overview; executed G notebook.

# G0: import the runtime and record what this experiment is

**Notebook G0** Run the full setup cell in `Graph_Workflow_Lab.ipynb`.

```
os.environ["LANGSMITH_TRACING"] = "false"
os.environ["LANGCHAIN_TRACING_V2"] = "false"
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command
from src.graph_workflow_lab import initial_state, recommendation_id, summary
versions = {name: version(name) for name in ("langgraph", "langgraph-checkpoint", "langchain-core")}
print(versions)
print("Rule-driven toy graph. No LLM, network call or electrical-model validation.")
```

The printed versions identify the environment. The last line identifies the scope: rule-driven toy execution, with no model call or electrical validation. After dependencies are installed, this exercise needs neither Ollama nor OpenDSS.

Exact notebook lines 5–13: G0; LangGraph.

# G0: import the runtime and record what this experiment is

**Notebook G0** Run the full setup cell in `Graph_Workflow_Lab.ipynb`.

```
os.environ["LANGSMITH_TRACING"] = "false"
os.environ["LANGCHAIN_TRACING_V2"] = "false"
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command
from src.graph_workflow_lab import initial_state, recommendation_id, summary
versions = {name: version(name) for name in ("langgraph", "langgraph-checkpoint", "langchain-core")}
print(versions)
print("Rule-driven toy graph. No LLM, network call or electrical-model validation.")
```

The printed versions identify the environment. The last line identifies the scope: rule-driven toy execution, with no model call or electrical validation. After dependencies are installed, this exercise needs neither Ollama nor OpenDSS.

Exact notebook lines 5–13: G0; LangGraph.

# G0: import the runtime and record what this experiment is

**Notebook G0** Run the full setup cell in `Graph_Workflow_Lab.ipynb`.

```
os.environ["LANGSMITH_TRACING"] = "false"
os.environ["LANGCHAIN_TRACING_V2"] = "false"
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command
from src.graph_workflow_lab import initial_state, recommendation_id, summary
versions = {name: version(name) for name in ("langgraph", "langgraph-checkpoint", "langchain-core")}
print(versions)
print("Rule-driven toy graph. No LLM, network call or electrical-model validation.")
```

The printed versions identify the environment. The last line identifies the scope: rule-driven toy execution, with no model call or electrical validation. After dependencies are installed, this exercise needs neither Ollama nor OpenDSS.

Exact notebook lines 5–13: G0; LangGraph.

# G1: the schema is our before-and-after table in Python

**Notebook G1** Run the full cell; connect each field to the worked example.

```python
class TrialState(TypedDict):
    command: float
    limit: float
    attempts: int
    max_repairs: int
    current: float
    passed: bool
    status: str
    acknowledged: bool
    trace: Annotated[list[str], add]
```

## Trial and gate

`command`, `limit`, `current`, and `passed` record the comparison.

## Execution history

`attempts` counts repairs. `trace` appends event lists using `add`.

`TypedDict` describes the schema; input validation is performed separately.

Exact state schema: G1; TypedDict.

# G1: the schema is our before-and-after table in Python

**Notebook G1** Run the full cell; connect each field to the worked example.

```python
class TrialState(TypedDict):
    command: float
    limit: float
    attempts: int
    max_repairs: int
    current: float
    passed: bool
    status: str
    acknowledged: bool
    trace: Annotated[list[str], add]
```

## Trial and gate

`command`, `limit`, `current`, and `passed` record the comparison.

## Execution history

`attempts` counts repairs. `trace` appends event lists using `add`.

`TypedDict` describes the schema; input validation is performed separately.

Exact state schema: G1; TypedDict.

# G1: the schema is our before-and-after table in Python

**Notebook G1** Run the full cell; connect each field to the worked example.

```python
class TrialState(TypedDict):
    command: float
    limit: float
    attempts: int
    max_repairs: int
    current: float
    passed: bool
    status: str
    acknowledged: bool
    trace: Annotated[list[str], add]
```

## Trial and gate

`command`, `limit`, `current`, and `passed` record the comparison.

## Execution history

`attempts` counts repairs. `trace` appends event lists using `add`.

`TypedDict` describes the schema; input validation is performed separately.

Exact state schema: G1; TypedDict.

# G2: simulation returns a number; verification returns a verdict

**Notebook G2** Run the whole cell; these are its first two node functions.

```
def simulate(s):
    value = 0.4 + 0.8 * s["command"]
    return {"current": value, "trace": [f"simulate: {value:.3f}"]}

def verify(s):
    ok = s["current"] <= s["limit"] + 1e-12
    return {"passed": ok, "trace": [f"verify: {ok}"]}
```

**Simulation update**

For $u = 0.75$, return `current = 1.00` and an event.

**Verification update**

Compare 1.00 with 0.80; return `passed = False` and another event.

The $10^{-12}$ code tolerance addresses floating-point comparison in this toy; it is not an engineering operating margin.

Exact functions: G2, lines 1–7.

# G2: simulation returns a number; verification returns a verdict

**Notebook G2** Run the whole cell; these are its first two node functions.

```
def simulate(s):
    value = 0.4 + 0.8 * s["command"]
    return {"current": value, "trace": [f"simulate: {value:.3f}"]}

def verify(s):
    ok = s["current"] <= s["limit"] + 1e-12
    return {"passed": ok, "trace": [f"verify: {ok}"]}
```

## Simulation update

For $u = 0.75$, return `current = 1.00` and an event.

## Verification update

Compare 1.00 with 0.80; return `passed = False` and another event.

The $10^{-12}$ code tolerance addresses floating-point comparison in this toy; it is not an engineering operating margin.

Exact functions: G2, lines 1–7.

# G2: simulation returns a number; verification returns a verdict

**Notebook G2** Run the whole cell; these are its first two node functions.

```
def simulate(s):
    value = 0.4 + 0.8 * s["command"]
    return {"current": value, "trace": [f"simulate: {value:.3f}"]}

def verify(s):
    ok = s["current"] <= s["limit"] + 1e-12
    return {"passed": ok, "trace": [f"verify: {ok}"]}
```

## Simulation update

For $u = 0.75$, return `current = 1.00` and an event.

## Verification update

Compare 1.00 with 0.80; return `passed = False` and another event.

The $10^{-12}$ code tolerance addresses floating-point comparison in this toy; it is not an engineering operating margin.

Exact functions: G2, lines 1–7.

# G2: repair changes the command; routing checks the budget

## Change the command and count the repair

```
def repair(s):
    return {"command": max(0., s["command"] - 0.25),
            "attempts": s["attempts"] + 1, "trace": [
     "repair: -0.25"]}
```

## Choose the permitted next node

```
def route(s):
    if s["passed"]:
        return "report"
    return "repair" if s["attempts"] < s["
     max_repairs"] else "hold"
```

The command drops by 0.25, bounded below by zero; `attempts` increases by one. Neither line changes the limit. A pass returns `report`. Otherwise, the route returns `repair` only while the counter is smaller than its maximum.

Exact repair and route: G2, lines 9–11 and 19–22.

# G2: repair changes the command; routing checks the budget

### Change the command and count the repair

```
def repair(s):
    return {"command": max(0., s["command"] - 0.25),
            "attempts": s["attempts"] + 1, "trace": [
     "repair: -0.25"]}
```

### Choose the permitted next node

```
def route(s):
    if s["passed"]:
        return "report"
    return "repair" if s["attempts"] < s["
     max_repairs"] else "hold"
```

The command drops by 0.25, bounded below by zero; `attempts` increases by one. Neither line changes the limit. A pass returns `report`. Otherwise, the route returns `repair` only while the counter is smaller than its maximum.

Exact repair and route: G2, lines 9–11 and 19–22.

# G2: repair changes the command; routing checks the budget

### Change the command and count the repair

```
def repair(s):
    return {"command": max(0., s["command"] - 0.25),
            "attempts": s["attempts"] + 1, "trace": [
     "repair: -0.25"]}
```

### Choose the permitted next node

```
def route(s):
    if s["passed"]:
        return "report"
    return "repair" if s["attempts"] < s["
     max_repairs"] else "hold"
```

The command drops by 0.25, bounded below by zero; `attempts` increases by one. Neither line changes the limit. A pass returns `report`. Otherwise, the route returns `repair` only while the counter is smaller than its maximum.

Exact repair and route: G2, lines 9–11 and 19–22.

# G3: the backward edge makes every repair face a fresh calculation

**Notebook G3** Run the cell and inspect the compiled node list.

```
    builder.add_edge(START, "simulate")
    builder.add_edge("simulate", "verify")
    builder.add_conditional_edges("verify", route,
        {"report": "report", "repair": "repair", "hold": "hold"})
    builder.add_edge("repair", "simulate")
    builder.add_edge("report", END)
    builder.add_edge("hold", END)
    return builder.compile(checkpointer=checkpointer)
```

`simulate` → `verify` always calculates before checking. The conditional edge reads the label returned by `route`. `repair` → `simulate` is the return loop. Compilation checks structure; it does not prove the response law or its physical assumptions.

Exact graph wiring: G3; LangGraph construction.

# G3: the backward edge makes every repair face a fresh calculation

**Notebook G3** Run the cell and inspect the compiled node list.

```
    builder.add_edge(START, "simulate")
    builder.add_edge("simulate", "verify")
    builder.add_conditional_edges("verify", route,
        {"report": "report", "repair": "repair", "hold": "hold"})
    builder.add_edge("repair", "simulate")
    builder.add_edge("report", END)
    builder.add_edge("hold", END)
    return builder.compile(checkpointer=checkpointer)
```

`simulate` → `verify` always calculates before checking. The conditional edge reads the label returned by `route`. `repair` → `simulate` is the return loop. Compilation checks structure; it does not prove the response law or its physical assumptions.

Exact graph wiring: G3; LangGraph construction.

# G3: the backward edge makes every repair face a fresh calculation

**Notebook G3** Run the cell and inspect the compiled node list.

```
    builder.add_edge(START, "simulate")
    builder.add_edge("simulate", "verify")
    builder.add_conditional_edges("verify", route,
        {"report": "report", "repair": "repair", "hold": "hold"})
    builder.add_edge("repair", "simulate")
    builder.add_edge("report", END)
    builder.add_edge("hold", END)
    return builder.compile(checkpointer=checkpointer)
```

`simulate` → `verify` always calculates before checking. The conditional edge reads the label returned by `route`. `repair` → `simulate` is the return loop. Compilation checks structure; it does not prove the response law or its physical assumptions.

Exact graph wiring: G3; LangGraph construction.

# G4: predict, run, then explain one complete trace

**Notebook G4** Run unchanged, then use a separate scratch cell for a new budget.

| Case | Final command | Final current | Status |
|---|---|---|---|
| Already admissible | 0.25 | 0.60 | `TOY_READY` |
| One repair | 0.50 | 0.80 | `TOY_READY` |
| Budget exhausted | 0.65 | 0.92 | `TOY_HOLD` |

**Repair trace:** simulate 1.00 → verify false → repair → simulate 0.80 → verify true → report.

**Predict:** What if the exhausted case gets four repairs instead of one?

**Explanation:** It reaches $u = 0$, hence $i = 0.40 \leq 0.50$, and can report ready.

Exact outputs, trace assertion, and cases: G4.

# G4: predict, run, then explain one complete trace

**Notebook G4** Run unchanged, then use a separate scratch cell for a new budget.

| Case | Final command | Final current | Status |
|---|---|---|---|
| Already admissible | 0.25 | 0.60 | `TOY_READY` |
| One repair | 0.50 | 0.80 | `TOY_READY` |
| Budget exhausted | 0.65 | 0.92 | `TOY_HOLD` |

**Repair trace:** simulate 1.00 $\rightarrow$ verify false $\rightarrow$ repair $\rightarrow$ simulate 0.80 $\rightarrow$ verify true $\rightarrow$ report.

**Predict:** What if the exhausted case gets four repairs instead of one?

**Explanation:** It reaches $u = 0$, hence $i = 0.40 \leq 0.50$, and can report ready.

Exact outputs, trace assertion, and cases: G4.

# G4: hold the functions fixed and compare the supervisor

```
from src.graph_workflow_lab import fixed_baseline
for name, args in cases.items():
    baseline = fixed_baseline(initial_state(*args), simulate, verify, repair, route, ready, hold)
    assert summary(baseline) == summary(results[name])
print("Direct Python baseline matches every command, outcome and trace: 3/3.")
```

## Controlled comparison

Same initial states, response law, repair rule, verification, and route.

## Observed result

All three commands, statuses, and event histories match the direct-Python baseline.

The graph did not improve this toy control result. Its explicit structure supports inspection and the pause/resume mechanism we will test next.

Executable baseline equality assertion: G4, lines 15–19.

# G4: hold the functions fixed and compare the supervisor

```
from src.graph_workflow_lab import fixed_baseline
for name, args in cases.items():
    baseline = fixed_baseline(initial_state(*args), simulate, verify, repair, route, ready, hold)
    assert summary(baseline) == summary(results[name])
print("Direct Python baseline matches every command, outcome and trace: 3/3.")
```

## Controlled comparison

Same initial states, response law, repair rule, verification, and route.

## Observed result

All three commands, statuses, and event histories match the direct-Python baseline.

The graph did not improve this toy control result. Its explicit structure supports inspection and the pause/resume mechanism we will test next.

Executable baseline equality assertion: G4, lines 15–19.

# G4: hold the functions fixed and compare the supervisor

```
from src.graph_workflow_lab import fixed_baseline
for name, args in cases.items():
    baseline = fixed_baseline(initial_state(*args), simulate, verify, repair, route, ready, hold)
    assert summary(baseline) == summary(results[name])
print("Direct Python baseline matches every command, outcome and trace: 3/3.")
```

## Controlled comparison

Same initial states, response law, repair rule, verification, and route.

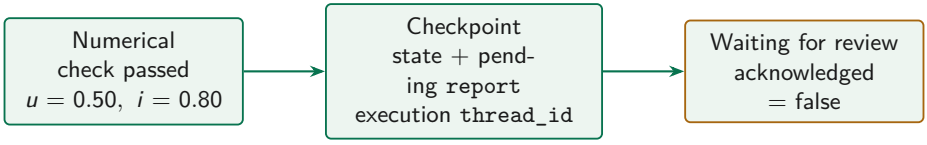
## Observed result

All three commands, statuses, and event histories match the direct-Python baseline.

The graph did not improve this toy control result. Its explicit structure supports inspection and the pause/resume mechanism we will test next.

Executable baseline equality assertion: G4, lines 15–19.

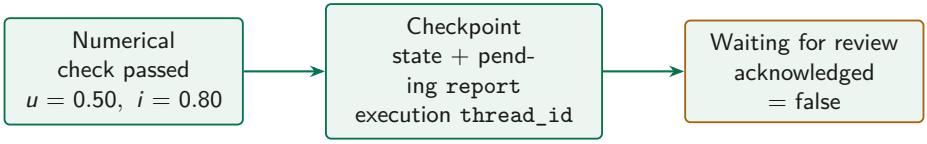
# A checkpoint preserves the record and the next unfinished step




A checkpoint is a saved execution record. A thread identifier associates a later response with that execution; it is not an operator credential. The lab uses `InMemorySaver`. It supports a pause within the running kernel, not durable recovery after the process is lost.

LangGraph persistence; actual checkpoint inspection: G5.

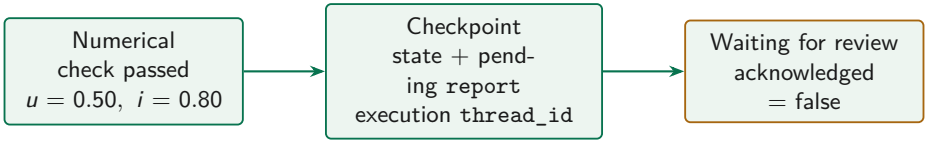
# A checkpoint preserves the record and the next unfinished step




A checkpoint is a saved execution record. A thread identifier associates a later response with that execution; it is not an operator credential. The lab uses `InMemorySaver`. It supports a pause within the running kernel, not durable recovery after the process is lost.

LangGraph persistence; actual checkpoint inspection: G5.

# A checkpoint preserves the record and the next unfinished step




A checkpoint is a saved execution record. A thread identifier associates a later response with that execution; it is not an operator credential. The lab uses `InMemorySaver`. It supports a pause within the running kernel, not durable recovery after the process is lost.

LangGraph persistence; actual checkpoint inspection: G5.

# G5: interrupt with the recommendation that needs a response

```python
def human_review(s):
    token = recommendation_id(s)
    reply = interrupt({"recommendation_id": token, "command": s["command"],
                       "toy_current": s["current"], "scope": "invented toy equation only"})
    accepted = (isinstance(reply, dict) and reply.get("approve") is True
                and reply.get("recommendation_id") == token)
    return {"acknowledged": accepted,
            "status": "TOY_ACKNOWLEDGED" if accepted else "TOY_HOLD",
            "trace": [f"human review: {accepted}"]}
```

The payload states the command, toy consequence, recommendation identity, and scope. A matching approval is required to record acknowledgment. On resume, the node can run again from its beginning. Do not put an uncontrolled equipment action before the interrupt.

Exact review function: G5, lines 1–9; interrupt semantics.

# G5: interrupt with the recommendation that needs a response

```python
def human_review(s):
    token = recommendation_id(s)
    reply = interrupt({"recommendation_id": token, "command": s["command"],
                       "toy_current": s["current"], "scope": "invented toy equation only"})
    accepted = (isinstance(reply, dict) and reply.get("approve") is True
                and reply.get("recommendation_id") == token)
    return {"acknowledged": accepted,
            "status": "TOY_ACKNOWLEDGED" if accepted else "TOY_HOLD",
            "trace": [f"human review: {accepted}"]}
```

The payload states the command, toy consequence, recommendation identity, and scope. A matching approval is required to record acknowledgment. On resume, the node can run again from its beginning. Do not put an uncontrolled equipment action before the interrupt.

Exact review function: G5, lines 1–9; interrupt semantics.

# G5: interrupt with the recommendation that needs a response

```python
def human_review(s):
    token = recommendation_id(s)
    reply = interrupt({"recommendation_id": token, "command": s["command"],
                       "toy_current": s["current"], "scope": "invented toy equation only"})
    accepted = (isinstance(reply, dict) and reply.get("approve") is True
                and reply.get("recommendation_id") == token)
    return {"acknowledged": accepted,
            "status": "TOY_ACKNOWLEDGED" if accepted else "TOY_HOLD",
            "trace": [f"human review: {accepted}"]}
```

The payload states the command, toy consequence, recommendation identity, and scope. A matching approval is required to record acknowledgment. On resume, the node can run again from its beginning. Do not put an uncontrolled equipment action before the interrupt.

Exact review function: G5, lines 1–9; interrupt semantics.

# G5: pause and inspect before resuming

**Notebook G5** For a visible pause, execute lines 1–17 first; then inspect the output.

```
review_app = build_graph(human_review, InMemorySaver())
config = {"configurable": {"thread_id": "student-" + uuid.uuid4().hex}, "recursion_limit": 40}
pause = review_app.invoke(initial_state(0.75, 0.80, 2), config)
snapshot = review_app.get_state(config)
print("Pending node:", snapshot.next, "| acknowledged:", snapshot.values["acknowledged"])
print("Interrupt payload:", pause["__interrupt__"][0].value)
assert snapshot.next == ("report",) and not snapshot.values["acknowledged"]
```

**Expected pre-resume output:** pending node `('report',)`; `acknowledged: False`. The payload describes the exact toy recommendation. Running the full cell also supplies a scripted response afterward. Inspect both printed stages rather than overlooking the intermediate pause.

Exact pause and snapshot: G5, lines 11–17.

# G5: pause and inspect before resuming

**Notebook G5** For a visible pause, execute lines 1–17 first; then inspect the output.

```
review_app = build_graph(human_review, InMemorySaver())
config = {"configurable": {"thread_id": "student-" + uuid.uuid4().hex}, "recursion_limit": 40}
pause = review_app.invoke(initial_state(0.75, 0.80, 2), config)
snapshot = review_app.get_state(config)
print("Pending node:", snapshot.next, "| acknowledged:", snapshot.values["acknowledged"])
print("Interrupt payload:", pause["__interrupt__"][0].value)
assert snapshot.next == ("report",) and not snapshot.values["acknowledged"]
```

**Expected pre-resume output:** pending node `('report',)`; `acknowledged: False`. The payload describes the exact toy recommendation. Running the full cell also supplies a scripted response afterward. Inspect both printed stages rather than overlooking the intermediate pause.

Exact pause and snapshot: G5, lines 11–17.

# G5: pause and inspect before resuming

**Notebook G5** For a visible pause, execute lines 1–17 first; then inspect the output.

```
review_app = build_graph(human_review, InMemorySaver())
config = {"configurable": {"thread_id": "student-" + uuid.uuid4().hex}, "recursion_limit": 40}
pause = review_app.invoke(initial_state(0.75, 0.80, 2), config)
snapshot = review_app.get_state(config)
print("Pending node:", snapshot.next, "| acknowledged:", snapshot.values["acknowledged"])
print("Interrupt payload:", pause["__interrupt__"][0].value)
assert snapshot.next == ("report",) and not snapshot.values["acknowledged"]
```

**Expected pre-resume output:** pending node `('report',)`; `acknowledged: False`. The payload describes the exact toy recommendation. Running the full cell also supplies a scripted response afterward. Inspect both printed stages rather than overlooking the intermediate pause.

Exact pause and snapshot: G5, lines 11–17.

# G5: resume the same execution with a matching readback

```
response = {"approve": True, "recommendation_id": recommendation_id(snapshot.values)}
resumed = review_app.invoke(Command(resume=response), config)
print("After simulated readback:", summary(resumed))
assert resumed["acknowledged"] and review_app.get_state(config).next == ()
```

**Matching simulated approval**

Status becomes `TOY_ACKNOWLEDGED`; no node remains pending.

**Decline or wrong identity**

The review function returns `TOY_HOLD`; acknowledgment remains false.

This is a state-management exercise. The scripted response does not authenticate a person, and the example sends no equipment command.

Exact resume and final assertion: G5; LangGraph interrupts.

# G5: resume the same execution with a matching readback

```
response = {"approve": True, "recommendation_id": recommendation_id(snapshot.values)}
resumed = review_app.invoke(Command(resume=response), config)
print("After simulated readback:", summary(resumed))
assert resumed["acknowledged"] and review_app.get_state(config).next == ()
```

## Matching simulated approval

Status becomes `TOY_ACKNOWLEDGED`; no node remains pending.

## Decline or wrong identity

The review function returns `TOY_HOLD`; acknowledgment remains false.

This is a state-management exercise. The scripted response does not authenticate a person, and the example sends no equipment command.

Exact resume and final assertion: G5; LangGraph interrupts.

# G5: resume the same execution with a matching readback

```
response = {"approve": True, "recommendation_id": recommendation_id(snapshot.values)}
resumed = review_app.invoke(Command(resume=response), config)
print("After simulated readback:", summary(resumed))
assert resumed["acknowledged"] and review_app.get_state(config).next == ()
```

## Matching simulated approval

Status becomes `TOY_ACKNOWLEDGED`; no node remains pending.

## Decline or wrong identity

The review function returns `TOY_HOLD`; acknowledgment remains false.

This is a state-management exercise. The scripted response does not authenticate a person, and the example sends no equipment command.

Exact resume and final assertion: G5; LangGraph interrupts.

# L1: a changed request must not reuse old evidence

**Notebook L0–L1** Open `Classroom_Labs.ipynb`; run setup and the binding experiment.

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
changed_request = {**request, 'load_scale': 1.10, 'revision': 2}
```

| | |
|---|---|
| Same request and same command | `True` |
| Changed load; old evidence | `False` |
| Changed command; old evidence | `False` |

The payload is explicitly `example_only`. Hash binding checks consistency; it does not prove physical feasibility or source authenticity.

Exact inputs and expected outputs: L1; binding implementation.

# L1: a changed request must not reuse old evidence

**Notebook L0–L1** Open `Classroom_Labs.ipynb`; run setup and the binding experiment.

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
changed_request = {**request, 'load_scale': 1.10, 'revision': 2}
```

| | |
|---|---|
| Same request and same command | `True` |
| Changed load; old evidence | `False` |
| Changed command; old evidence | `False` |

The payload is explicitly `example_only`. Hash binding checks consistency; it does not prove physical feasibility or source authenticity.

Exact inputs and expected outputs: L1; binding implementation.

# L1: a changed request must not reuse old evidence

**Notebook L0–L1** Open `Classroom_Labs.ipynb`; run setup and the binding experiment.

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
changed_request = {**request, 'load_scale': 1.10, 'revision': 2}
```

| | |
|---|---|
| Same request and same command | `True` |
| Changed load; old evidence | `False` |
| Changed command; old evidence | `False` |

The payload is explicitly `example_only`. Hash binding checks consistency; it does not prove physical feasibility or source authenticity.

Exact inputs and expected outputs: L1; binding implementation.

# Schema, binding, and engineering checks answer different questions

| Check | Question | A pass does not establish |
|---|---|---|
| Schema | Are required fields and values valid? | Correct interpretation of the operator's intent |
| Binding | Does this result belong to this candidate? | Authenticity of its source |
| Engineering | Do modeled limits pass? | Adequate model scope or field authorization |

**Predict:** A solver converges at 112 A; the declared limit is 100 A. Two LLMs favor the candidate. Accept it?

**Explanation:** No. The current check fails by 12 A. A vote cannot raise the declared limit.

Tutorial evidence and limit checks: classroom tools; case adapters.

# Schema, binding, and engineering checks answer different questions

| Check | Question | A pass does not establish |
|---|---|---|
| Schema | Are required fields and values valid? | Correct interpretation of the operator's intent |
| Binding | Does this result belong to this candidate? | Authenticity of its source |
| Engineering | Do modeled limits pass? | Adequate model scope or field authorization |

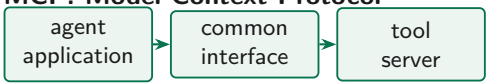
**Predict:** A solver converges at 112 A; the declared limit is 100 A. Two LLMs favor the candidate. Accept it?

**Explanation:** No. The current check fails by 12 A. A vote cannot raise the declared limit.

Tutorial evidence and limit checks: classroom tools; case adapters.

# MCP connects tools; MPC computes a control plan

## MCP: Model Context Protocol




It standardizes tool discovery, arguments, and results. It does not determine whether a current is acceptable.

## MPC: Model Predictive Control



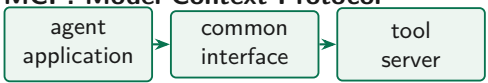

It uses a predictive model, objective, and constraints to plan control, then replans after the next observation.

An MPC solver could be exposed through MCP. Our classroom functions are called directly in Python; the tutorial does not require an MCP server.

MCP tools specification; Rawlings et al., MPC.

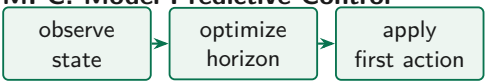
# MCP connects tools; MPC computes a control plan

**MCP: Model Context Protocol**




It standardizes tool discovery, arguments, and results. It does not determine whether a current is acceptable.

**MPC: Model Predictive Control**



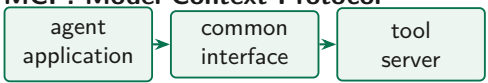

It uses a predictive model, objective, and constraints to plan control, then replans after the next observation.

An MPC solver could be exposed through MCP. Our classroom functions are called directly in Python; the tutorial does not require an MCP server.

MCP tools specification; Rawlings et al., MPC.

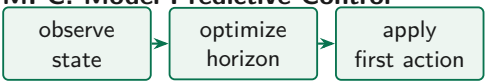
# MCP connects tools; MPC computes a control plan

## MCP: Model Context Protocol




It standardizes tool discovery, arguments, and results. It does not determine whether a current is acceptable.

## MPC: Model Predictive Control




It uses a predictive model, objective, and constraints to plan control, then replans after the next observation.

An MPC solver could be exposed through MCP. Our classroom functions are called directly in Python; the tutorial does not require an MCP server.

MCP tools specification; Rawlings et al., MPC.

# MPC plans with a model; SAC supplies a learned continuous action

## Model-based planning

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k)$$

subject to a prediction model and operating constraints. Apply the first action, observe again, and replan.

## Model-free policy inference

$$u_t = \pi_\theta(o_t).$$

The trained policy maps an observation to a continuous command. Class uses saved SAC weights; it does not train a new actor.

$H$: planning horizon; $k$: predicted step; $x_k$: predicted state; $u_k$: control; $\ell$: stage cost. $t$: current decision time; $o_t$: observation; $\pi_\theta$: policy with learned parameters $\theta$. **Same acceptance process:** simulate both candidates and check the same declared limits. PPO is discussed as an alternative; it is not trained here.

MPC; SAC; PPO.

# MPC plans with a model; SAC supplies a learned continuous action

### Model-based planning

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k)$$

subject to a prediction model and operating constraints. Apply the first action, observe again, and replan.

### Model-free policy inference

$$u_t = \pi_\theta(o_t).$$

The trained policy maps an observation to a continuous command. Class uses saved SAC weights; it does not train a new actor.

$H$: planning horizon; $k$: predicted step; $x_k$: predicted state; $u_k$: control; $\ell$: stage cost. $t$: current decision time; $o_t$: observation; $\pi_\theta$: policy with learned parameters $\theta$. **Same acceptance process:** simulate both candidates and check the same declared limits. PPO is discussed as an alternative; it is not trained here.

MPC; SAC; PPO.

# MPC plans with a model; SAC supplies a learned continuous action

### Model-based planning

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k)$$

subject to a prediction model and operating constraints. Apply the first action, observe again, and replan.

### Model-free policy inference

$$u_t = \pi_\theta(o_t).$$

The trained policy maps an observation to a continuous command. Class uses saved SAC weights; it does not train a new actor.

$H$: planning horizon; $k$: predicted step; $x_k$: predicted state; $u_k$: control; $\ell$: stage cost. $t$: current decision time; $o_t$: observation; $\pi_\theta$: policy with learned parameters $\theta$. **Same acceptance process:** simulate both candidates and check the same declared limits. PPO is discussed as an alternative; it is not trained here.

MPC; SAC; PPO.

# Prior frameworks contribute mechanisms we can teach and test

| Framework | Useful mechanism | Connection to our example |
|---|---|---|
| PowerAgent ecosystem | reusable engineering interfaces and workflows | expose the calculation with inspectable inputs and outputs |
| PowerDAG | exemplar retrieval; prerequisite advisory | detect a covered dependency problem before a tool call |
| Grid-Agent v3 | specialized roles; sandbox validation and rollback | keep a failed trial separate from the incumbent |
| X-GridAgent | planning–coordination–action hierarchy | preserve task context across handoffs |

PowerDAG covers selected rules and permits repeated calls after its one-shot advisory. That mechanism is not an unconditional hard safety gate. These are close prior works, not a leaderboard. Our tutorial does not claim the first multi-agent grid workflow or first use of numerical verification.

PowerAgent; PowerDAG; Grid-Agent v3; X-GridAgent.

# Prior frameworks contribute mechanisms we can teach and test

| Framework | Useful mechanism | Connection to our example |
|---|---|---|
| PowerAgent ecosystem | reusable engineering interfaces and workflows | expose the calculation with inspectable inputs and outputs |
| PowerDAG | exemplar retrieval; prerequisite advisory | detect a covered dependency problem before a tool call |
| Grid-Agent v3 | specialized roles; sandbox validation and rollback | keep a failed trial separate from the incumbent |
| X-GridAgent | planning–coordination–action hierarchy | preserve task context across handoffs |

PowerDAG covers selected rules and permits repeated calls after its one-shot advisory. That mechanism is not an unconditional hard safety gate. These are close prior works, not a leaderboard. Our tutorial does not claim the first multi-agent grid workflow or first use of numerical verification.

PowerAgent; PowerDAG; Grid-Agent v3; X-GridAgent.

# Prior frameworks contribute mechanisms we can teach and test

| Framework | Useful mechanism | Connection to our example |
|---|---|---|
| PowerAgent ecosystem | reusable engineering interfaces and workflows | expose the calculation with inspectable inputs and outputs |
| PowerDAG | exemplar retrieval; prerequisite advisory | detect a covered dependency problem before a tool call |
| Grid-Agent v3 | specialized roles; sandbox validation and rollback | keep a failed trial separate from the incumbent |
| X-GridAgent | planning–coordination–action hierarchy | preserve task context across handoffs |

PowerDAG covers selected rules and permits repeated calls after its one-shot advisory. That mechanism is not an unconditional hard safety gate. These are close prior works, not a leaderboard. Our tutorial does not claim the first multi-agent grid workflow or first use of numerical verification.

PowerAgent; PowerDAG; Grid-Agent v3; X-GridAgent.

# A capability tick describes documented scope, not superiority

| Work | Tools | Feedback | Checks | Review | Qualification |
|---|---|---|---|---|---|
| PowerAgent | ✓ | ✓ | △ | ✓ | capabilities depend on workflow |
| PowerDAG | ✓ | ✓ | △ | NR | selected rules; one-shot advisory |
| Grid-Agent v3 | ✓ | ✓ | ✓ | △ | sandbox; deployment oversight proposed |
| X-GridAgent | ✓ | ✓ | △ | △ | solver checks; prompt refinement |
| PowerAgentBench-SS | ✓ | ✓ | ✓ | △ | DC benchmark; hidden revalidation |
| Tutorial | ✓ | ✓ | ✓ | ✓ | declared models; local readback |

✓: documented; △: qualified or proposed; NR: not reported in the inspected source. Benchmark work evaluates agents; it is not itself a controller. Common scenarios, tools, budgets, and outcome definitions are needed before claiming an advantage.

PowerAgent; PowerDAG; Grid-Agent v3; X-GridAgent; PowerAgentBench-SS.

# A capability tick describes documented scope, not superiority

| Work | Tools | Feedback | Checks | Review | Qualification |
|---|---|---|---|---|---|
| PowerAgent | ✓ | ✓ | △ | ✓ | capabilities depend on workflow |
| PowerDAG | ✓ | ✓ | △ | NR | selected rules; one-shot advisory |
| Grid-Agent v3 | ✓ | ✓ | ✓ | △ | sandbox; deployment oversight proposed |
| X-GridAgent | ✓ | ✓ | △ | △ | solver checks; prompt refinement |
| PowerAgentBench-SS | ✓ | ✓ | ✓ | △ | DC benchmark; hidden revalidation |
| Tutorial | ✓ | ✓ | ✓ | ✓ | declared models; local readback |

✓: documented; △: qualified or proposed; NR: not reported in the inspected source. Benchmark work evaluates agents; it is not itself a controller. Common scenarios, tools, budgets, and outcome definitions are needed before claiming an advantage.

PowerAgent; PowerDAG; Grid-Agent v3; X-GridAgent; PowerAgentBench-SS.

# Compare benefit in three columns, with the same experiment

**Numerical outcome**

Objective value, served load, constraint violations, and model scope.

**Workflow outcome**

Stale evidence rejected, required checks recorded, and recommendation identity preserved.

**Cost of the decision**

Solver time, model-call time, number of trials, and training cost reported separately.

The G baseline already gives a useful null result: the commands tie exactly. That result demonstrates graph mechanics, not better physical control. For the engineering comparison, hold the scenario, available controls, objective, constraints, and trial budget fixed where a paired claim is made.

Tutorial matched baseline: G4; evaluation design: PowerAgentBench-SS.

# Compare benefit in three columns, with the same experiment

**Numerical outcome**

Objective value, served load, constraint violations, and model scope.

**Workflow outcome**

Stale evidence rejected, required checks recorded, and recommendation identity preserved.

**Cost of the decision**

Solver time, model-call time, number of trials, and training cost reported separately.

The G baseline already gives a useful null result: the commands tie exactly. That result demonstrates graph mechanics, not better physical control. For the engineering comparison, hold the scenario, available controls, objective, constraints, and trial budget fixed where a paired claim is made.

Tutorial matched baseline: G4; evaluation design: PowerAgentBench-SS.

# Compare benefit in three columns, with the same experiment

**Numerical outcome**

Objective value, served load, constraint violations, and model scope.

**Workflow outcome**

Stale evidence rejected, required checks recorded, and recommendation identity preserved.

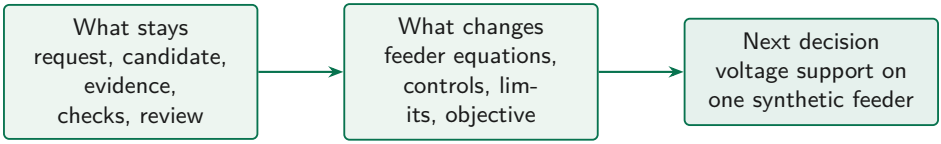
**Cost of the decision**

Solver time, model-call time, number of trials, and training cost reported separately.

The G baseline already gives a useful null result: the commands tie exactly. That result demonstrates graph mechanics, not better physical control. For the engineering comparison, hold the scenario, available controls, objective, constraints, and trial budget fixed where a paired claim is made.

Tutorial matched baseline: G4; evaluation design: PowerAgentBench-SS.

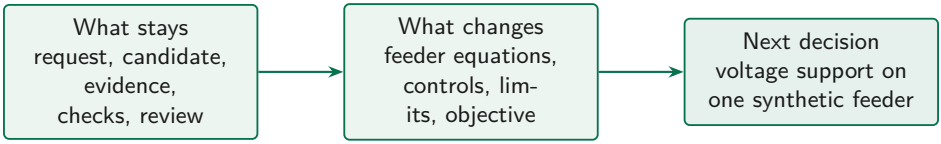
# Keep the process; replace the toy law with electrical models




MPC and SAC will propose inverter commands. Three-phase LinDistFlow and OpenDSS will expose differences in modeled consequences and scope. Llama's bounded choices, public explanations, and latency will be inspected alongside numerical baselines. A favorable explanation never substitutes for a fresh check.

Voltage experiments and exact outputs: L2–L5; response traces: five-stage archive.

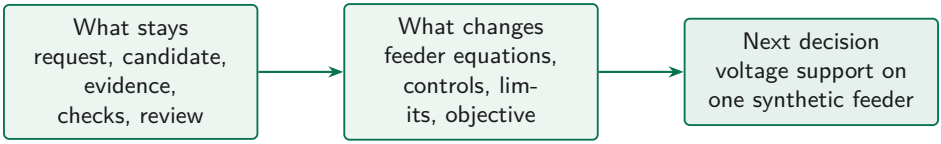
# Keep the process; replace the toy law with electrical models




MPC and SAC will propose inverter commands. Three-phase LinDistFlow and OpenDSS will expose differences in modeled consequences and scope. Llama's bounded choices, public explanations, and latency will be inspected alongside numerical baselines. A favorable explanation never substitutes for a fresh check.

Voltage experiments and exact outputs: L2–L5; response traces: five-stage archive.

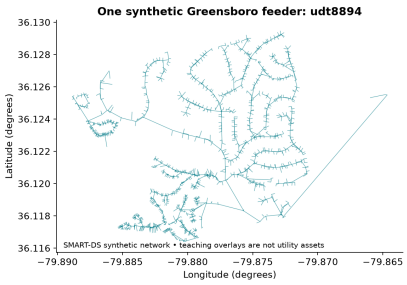
# Keep the process; replace the toy law with electrical models




MPC and SAC will propose inverter commands. Three-phase LinDistFlow and OpenDSS will expose differences in modeled consequences and scope. Llama's bounded choices, public explanations, and latency will be inspected alongside numerical baselines. A favorable explanation never substitutes for a fresh check.

Voltage experiments and exact outputs: L2–L5; response traces: five-stage archive.

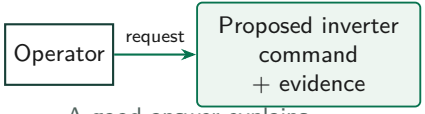
# The operator's question: how much PV can this feeder accept?




One SMART-DS synthetic Greensboro feeder: `uhs20_1247-udt8894`.

**An operating request**

Use available PV while keeping modeled voltages and currents within their limits.



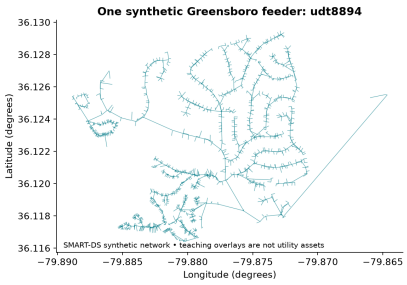

A good answer explains what was checked and what was omitted.

**Predict:** Which could limit additional PV: voltage, current, or inverter capability?

**Explore:** Any of the three. We must inspect their margins together.

SMART-DS synthetic distribution models; Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

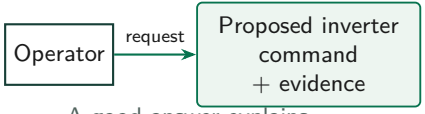
# The operator's question: how much PV can this feeder accept?




One SMART-DS synthetic Greensboro feeder: `uhs20_1247-udt8894`.

**An operating request**

Use available PV while keeping modeled voltages and currents within their limits.



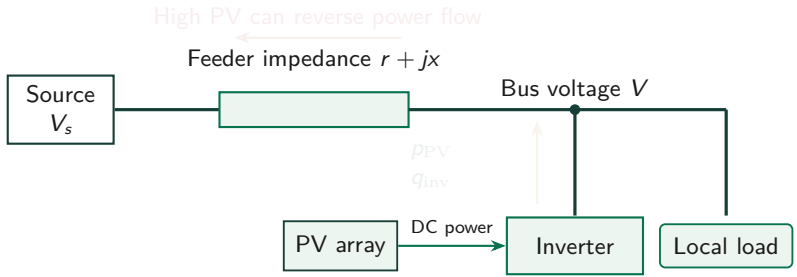

A good answer explains what was checked and what was omitted.

**Predict:** Which could limit additional PV: voltage, current, or inverter capability?

**Explanation:** Any of the three. We must inspect their margins together.

SMART-DS synthetic distribution models; Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

# See the electrical path before choosing an algorithm




$r$: resistance; $x$: reactance; $\mathrm{j}^2 = -1$. $p_{\mathrm{PV}}, q_{\mathrm{inv}}$: active/reactive inverter injection.

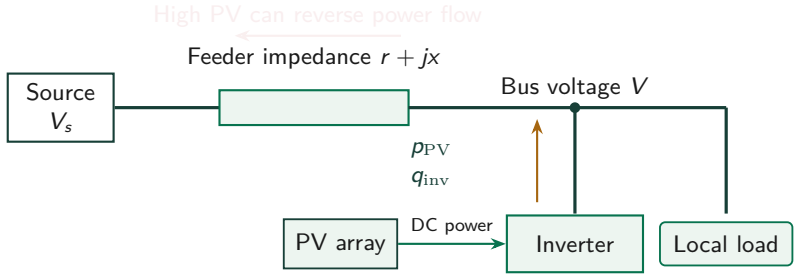
**Voltage**
The line impedance turns power flow into a voltage change.

**Current**
Both active and reactive flow use current capacity.

**Inverter**
Active and reactive commands share one kVA rating.

Gan & Low: three-phase distribution optimization (2014)

# See the electrical path before choosing an algorithm




$r$: resistance; $x$: reactance; $\mathrm{j}^2 = -1$. $p_{\mathrm{PV}}, q_{\mathrm{inv}}$: active/reactive inverter injection.

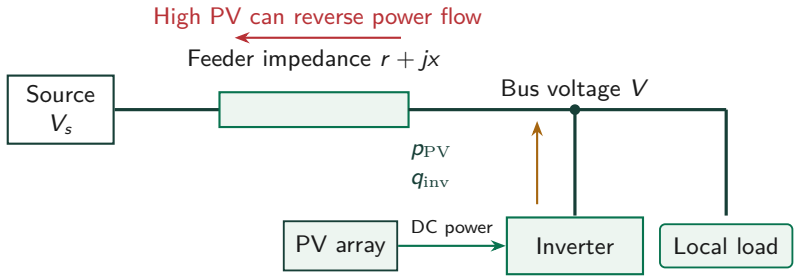
**Voltage**
The line impedance turns power flow into a voltage change.

**Current**
Both active and reactive flow use current capacity.

**Inverter**
Active and reactive commands share one kVA rating.

Gan & Low: three-phase distribution optimization (2014)

# See the electrical path before choosing an algorithm




$r$: resistance; $x$: reactance; $\mathrm{j}^2 = -1$. $p_{\mathrm{PV}}, q_{\mathrm{inv}}$: active/reactive inverter injection.

**Voltage**
The line impedance turns power flow into a voltage change.

**Current**
Both active and reactive flow use current capacity.

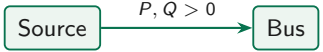
**Inverter**
Active and reactive commands share one kVA rating.

Gan & Low: three-phase distribution optimization (2014)

# Keep net demand and inverter injection signs distinct

**The feeder sees net demand**
Positive $P, Q$ mean net consumption downstream.

$$P = P_{\text{load}} - p_{\text{PV}}, \qquad Q = Q_{\text{load}} - q_{\text{inv}}.$$




$P, Q$: net active/reactive power; $P_{\text{load}}, Q_{\text{load}}$: demand; $p_{\text{PV}}, q_{\text{inv}}$: inverter injections. Use one consistent power base.

**The inverter absorbs reactive power**
$q_{\text{inv}} < 0$ increases net $Q$.

$$\Delta V \approx rP + xQ, \qquad V \approx V_s - \Delta V.$$

High $V$ → Absorb $Q$ → Lower $V$

For positive line reactance $x$, more net reactive consumption increases the approximate drop.

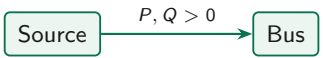
**Physical tradeoff**
Lower voltage can require more current. We must check both.

Gan & Low: three-phase distribution optimization (2014); illustrative single-branch linearization, not a feeder measurement

# Keep net demand and inverter injection signs distinct

**The feeder sees net demand**
Positive $P, Q$ mean net consumption downstream.

$$P = P_{\text{load}} - p_{\text{PV}}, \qquad Q = Q_{\text{load}} - q_{\text{inv}}.$$




$P, Q$: net active/reactive power; $P_{\text{load}}, Q_{\text{load}}$: demand; $p_{\text{PV}}, q_{\text{inv}}$: inverter injections. Use one consistent power base.

**The inverter absorbs reactive power**
$q_{\text{inv}} < 0$ increases net $Q$.

$$\Delta V \approx rP + xQ, \qquad V \approx V_s - \Delta V.$$




For positive line reactance $x$, more net reactive consumption increases the approximate drop.

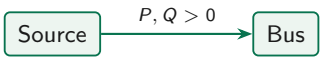
**Physical tradeoff**
Lower voltage can require more current. We must check both.

Gan & Low: three-phase distribution optimization (2014); illustrative single-branch linearization, not a feeder measurement

# Keep net demand and inverter injection signs distinct

**The feeder sees net demand**
Positive $P, Q$ mean net consumption downstream.

$$P = P_{\text{load}} - p_{\text{PV}}, \qquad Q = Q_{\text{load}} - q_{\text{inv}}.$$




$P, Q$: net active/reactive power; $P_{\text{load}}, Q_{\text{load}}$: demand; $p_{\text{PV}}, q_{\text{inv}}$: inverter injections. Use one consistent power base.

**The inverter absorbs reactive power**
$q_{\text{inv}} < 0$ increases net $Q$.

$$\Delta V \approx rP + xQ, \qquad V \approx V_s - \Delta V.$$




For positive line reactance $x$, more net reactive consumption increases the approximate drop.

**Physical tradeoff**
Lower voltage can require more current. We must check both.

Gan & Low: three-phase distribution optimization (2014); illustrative single-branch linearization, not a feeder measurement

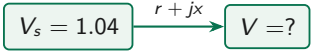
# A small calculation: reactive absorption lowers high voltage

**Illustrative single branch**
All quantities are per unit on consistent bases.

$$V_s = 1.04, \quad r = 0.02, \quad x = 0.04,$$

$$P = -0.60.$$

Negative $P$ means net PV export. $Q$ is net reactive consumption. This is arithmetic for intuition, not output from the Greensboro model.




$$V \approx V_s - rP - xQ.$$

**No reactive absorption:** $Q = 0$
$V \approx 1.04 - 0.02(-0.60) = 1.052$ p.u.
Above the illustrative 1.05 p.u. upper bound.

**Absorb reactive power:** $Q = 0.20$
$V \approx 1.04 + 0.012 - 0.008 = 1.044$ p.u.
The voltage now passes that bound.

Illustrative arithmetic using a linear voltage-drop approximation; Gan & Low: three-phase distribution optimization (2014)

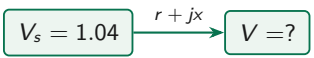
# A small calculation: reactive absorption lowers high voltage

**Illustrative single branch**
All quantities are per unit on consistent bases.

$$V_s = 1.04, \quad r = 0.02, \quad x = 0.04,$$

$$P = -0.60.$$

Negative $P$ means net PV export. $Q$ is net reactive consumption. This is arithmetic for intuition, not output from the Greensboro model.




$$V \approx V_s - rP - xQ.$$

**No reactive absorption:** $Q = 0$
$V \approx 1.04 - 0.02(-0.60) = 1.052$ p.u.
Above the illustrative 1.05 p.u. upper bound.

**Absorb reactive power:** $Q = 0.20$
$V \approx 1.04 + 0.012 - 0.008 = 1.044$ p.u.
The voltage now passes that bound.

Illustrative arithmetic using a linear voltage-drop approximation; Gan & Low: three-phase distribution optimization (2014)

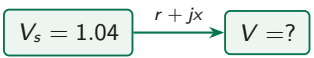
# A small calculation: reactive absorption lowers high voltage

**Illustrative single branch**
All quantities are per unit on consistent bases.

$$V_s = 1.04, \quad r = 0.02, \quad x = 0.04,$$

$$P = -0.60.$$

Negative $P$ means net PV export. $Q$ is net reactive consumption. This is arithmetic for intuition, not output from the Greensboro model.




$$V \approx V_s - rP - xQ.$$

**No reactive absorption:** $Q = 0$
$V \approx 1.04 - 0.02(-0.60) = 1.052$ p.u.
Above the illustrative 1.05 p.u. upper bound.

**Absorb reactive power:** $Q = 0.20$
$V \approx 1.04 + 0.012 - 0.008 = 1.044$ p.u.
The voltage now passes that bound.

Illustrative arithmetic using a linear voltage-drop approximation; Gan & Low: three-phase distribution optimization (2014)

# Voltage passes, but current can still fail

$$I \approx \frac{\sqrt{P^2 + Q^2}}{V_s}, \qquad V_{\max} = 1.05, \quad I_{\max} = 0.60 \text{ p.u.}$$

$I$: approximate current magnitude; $P, Q$: net power; $V_s = 1.04$ p.u. fixed reference. The limits are illustrative.

| Action | $P$ | $Q$ | $V$ | $I$ | Checks |
|---|---|---|---|---|---|
| PV export only | −0.60 | 0 | 1.052 | 0.577 | Voltage fails |
| Add absorption | −0.60 | 0.20 | 1.044 | 0.608 | Current fails |
| Also reduce export | −0.55 | 0.20 | 1.043 | 0.563 | Both pass |

**Predict:** Which action would a voltage-only checker mistakenly accept?

**Why we need optimization and verification**

Balance active power, reactive effort and every declared limit. A single improved metric is insufficient.

Explicitly illustrative per-unit arithmetic; no measured or simulated feeder result

# Voltage passes, but current can still fail

$$I \approx \frac{\sqrt{P^2 + Q^2}}{V_s}, \qquad V_{\max} = 1.05, \quad I_{\max} = 0.60 \text{ p.u.}$$

$I$: approximate current magnitude; $P, Q$: net power; $V_s = 1.04$ p.u. fixed reference. The limits are illustrative.

| Action | $P$ | $Q$ | $V$ | $I$ | Checks |
|---|---|---|---|---|---|
| PV export only | −0.60 | 0 | 1.052 | 0.577 | Voltage fails |
| Add absorption | −0.60 | 0.20 | 1.044 | 0.608 | Current fails |
| Also reduce export | −0.55 | 0.20 | 1.043 | 0.563 | Both pass |

**Predict:** Which action would a voltage-only checker mistakenly accept?

**Why we need optimization and verification**

Balance active power, reactive effort and every declared limit. A single improved metric is insufficient.

Explicitly illustrative per-unit arithmetic; no measured or simulated feeder result

# Voltage passes, but current can still fail

$$I \approx \frac{\sqrt{P^2 + Q^2}}{V_s}, \qquad V_{\max} = 1.05, \quad I_{\max} = 0.60 \text{ p.u.}$$

$I$: approximate current magnitude; $P, Q$: net power; $V_s = 1.04$ p.u. fixed reference. The limits are illustrative.

| Action | $P$ | $Q$ | $V$ | $I$ | Checks |
|---|---|---|---|---|---|
| PV export only | −0.60 | 0 | 1.052 | 0.577 | Voltage fails |
| Add absorption | −0.60 | 0.20 | 1.044 | 0.608 | Current fails |
| Also reduce export | −0.55 | 0.20 | 1.043 | 0.563 | Both pass |

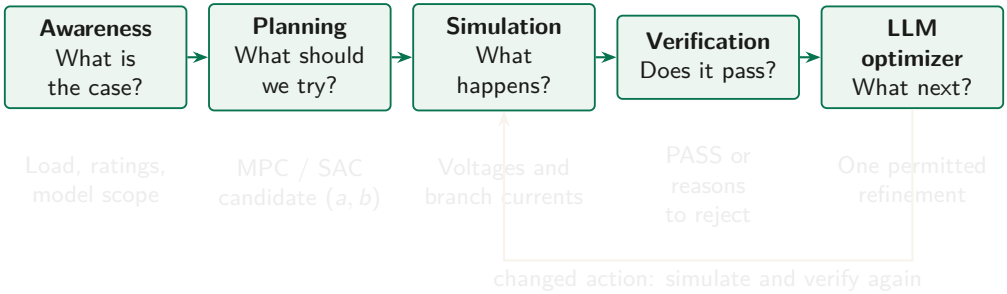
**Predict:** Which action would a voltage-only checker mistakenly accept?

**Why we need optimization and verification**

Balance active power, reactive effort and every declared limit. A single improved metric is insufficient.

Explicitly illustrative per-unit arithmetic; no measured or simulated feeder result

# The five stages turn an action into reviewable evidence

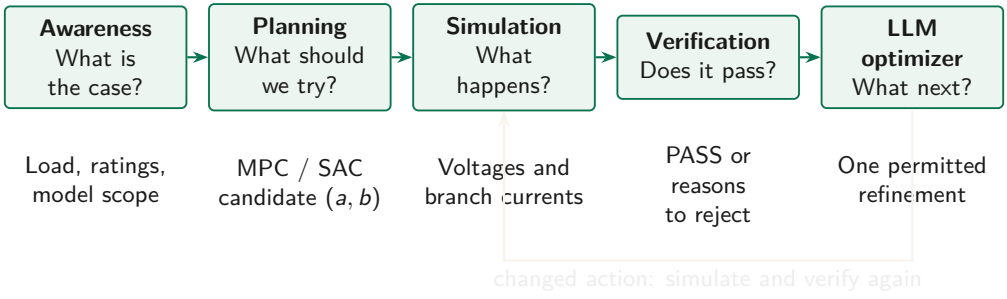



**The lesson from the arithmetic**

The optimizer may suggest reducing export. The numerical checker must establish whether that new command meets the declared limits.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; five-stage proposal architecture

# The five stages turn an action into reviewable evidence

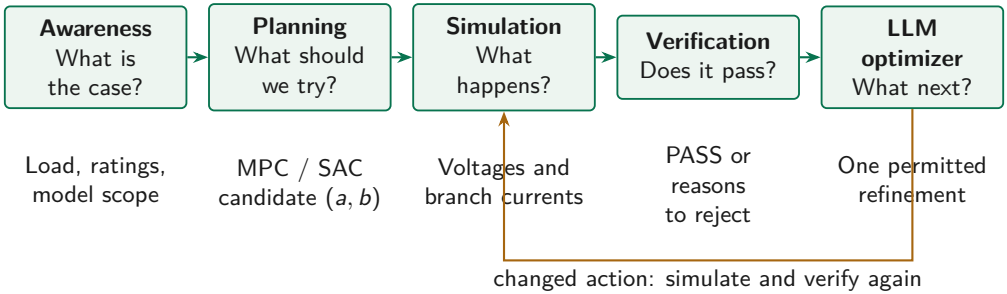



**The lesson from the arithmetic**

The optimizer may suggest reducing export. The numerical checker must establish whether that new command meets the declared limits.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; five-stage proposal architecture

# The five stages turn an action into reviewable evidence

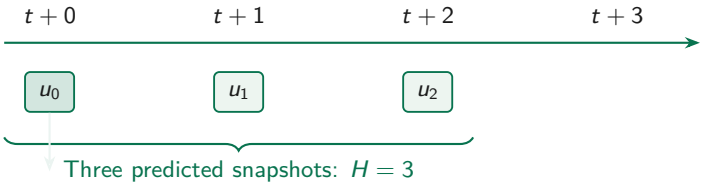



**The lesson from the arithmetic**

The optimizer may suggest reducing export. The numerical checker must establish whether that new command meets the declared limits.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; five-stage proposal architecture

# MPC uses a forecast, then commits only the first move

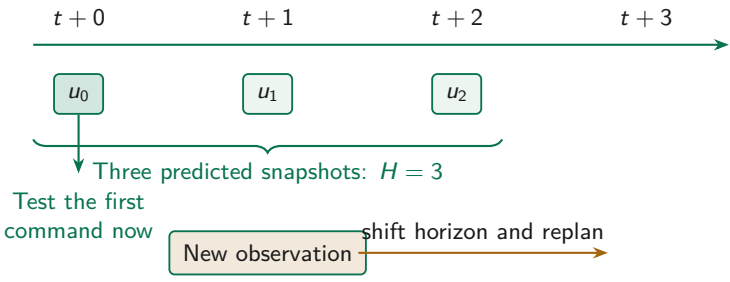



$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \gamma^k \ell(x_k, u_k) \quad \text{s.t. prediction model and operating limits.}$$

$u_k$: control; $x_k$: predicted state; $H$: horizon; $\gamma = 0.98$: discount; $\ell$: one-step cost. Here: algebraic power-flow snapshots linked by command-movement cost.

Rawlings, Mayne & Diehl: MPC (2017)

# MPC uses a forecast, then commits only the first move




$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \gamma^k \ell(x_k, u_k) \quad \text{s.t. prediction model and operating limits.}$$

$u_k$: control; $x_k$: predicted state; $H$: horizon; $\gamma = 0.98$: discount; $\ell$: one-step cost. Here: algebraic power-flow snapshots linked by command-movement cost.

Rawlings, Mayne & Diehl: MPC (2017)

# The objective makes engineering preferences explicit

**One fleet command**
$u = (a, b)$ is dimensionless.

$$p_i = aS_i, \qquad q_i = bS_i.$$

$S_i$: inverter rating (kVA); $p_i$: active power (kW); $q_i$: reactive injection (kvar).

$b < 0$ means absorption. $u_{\text{prev}}$ is the previous command.

$$\ell = \underbrace{(1-a)^2}_{\text{unused active capability}} + \underbrace{0.05b^2}_{\text{reactive effort}} + \underbrace{10\,\text{mean}[(V-1.01)^2]}_{\text{voltage deviation}} + \underbrace{0.05\|u-u_{\text{prev}}\|_2^2}_{\text{command movement}}.$$

$V$: modeled voltage magnitudes (p.u.); $\ell$: dimensionless weighted teaching cost.

**Predict:** If $|b|$ decreases, which cost term certainly decreases?

**Explanation:** Reactive effort. The voltage and movement terms can also change.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# The objective makes engineering preferences explicit

**One fleet command**
$u = (a, b)$ is dimensionless.

$$p_i = aS_i, \qquad q_i = bS_i.$$

$S_i$: inverter rating (kVA); $p_i$: active power (kW); $q_i$: reactive injection (kvar).

$b < 0$ means absorption. $u_{\text{prev}}$ is the previous command.

$$\ell = \underbrace{(1-a)^2}_{\text{unused active capability}} + \underbrace{0.05b^2}_{\text{reactive effort}} + \underbrace{10\,\text{mean}[(V-1.01)^2]}_{\text{voltage deviation}} + \underbrace{0.05\|u-u_{\text{prev}}\|_2^2}_{\text{command movement}}.$$

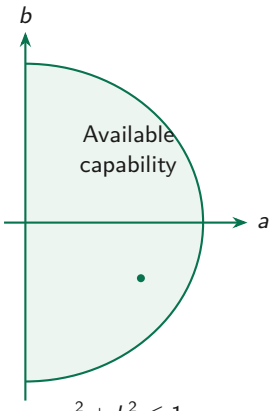
$V$: modeled voltage magnitudes (p.u.); $\ell$: dimensionless weighted teaching cost.

**Predict:** If $|b|$ decreases, which cost term certainly decreases?

**Explanation:** Reactive effort. The voltage and movement terms can still worsen.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# The admissible region is a gate, not another preference




**Network limits**

$V_{\min} \leq |V_i| \leq V_{\max}, \quad |I_e| \leq I_e^{\max}.$

$V_i$: bus-phase voltage; $I_e$: branch current; bounds belong to the request.

**Inverter limits**

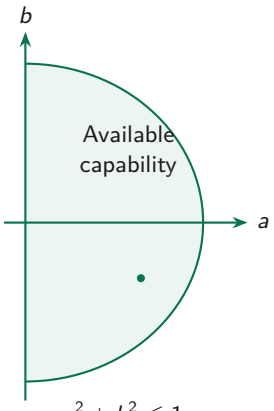
$p_i^2 + q_i^2 \leq S_i^2$ gives the capability circle; $0 \leq a \leq 1$ and declared bounds on $b$ also apply.

**Evidence limits**

Finite inputs, solver completion, dispatch tracking and matching request/candidate/model identifiers.

Rawlings, Mayne & Diehl: MPC (2017); Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# The admissible region is a gate, not another preference




**Network limits**

$V_{\min} \leq |V_i| \leq V_{\max}, \quad |I_e| \leq I_e^{\max}.$

$V_i$: bus-phase voltage; $I_e$: branch current; bounds belong to the request.

**Inverter limits**

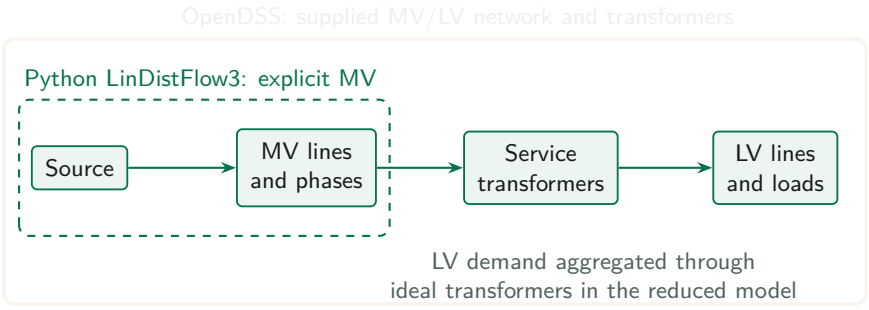
$p_i^2 + q_i^2 \leq S_i^2$ gives the capability circle; $0 \leq a \leq 1$ and declared bounds on $b$ also apply.

**Evidence limits**

Finite inputs, solver completion, dispatch tracking and matching request/candidate/model identifiers.

Rawlings, Mayne & Diehl: MPC (2017); Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# The two models cover different parts of the feeder



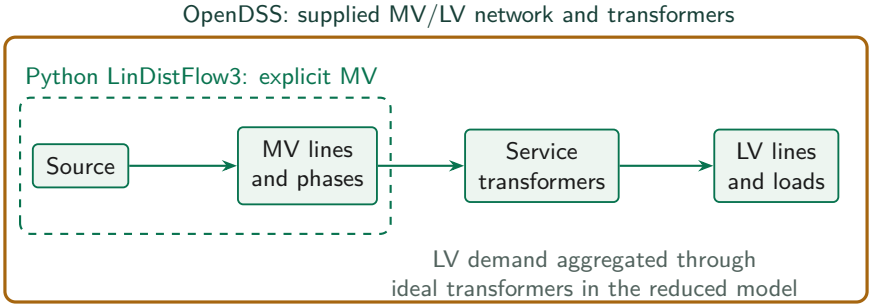

**Reduced-model PASS**
Within the MV-only approximation and its checked limits.

**Full-model PASS**
Within the broader unbalanced snapshot contract.

**Comparison boundary:** equations *and* constraint coverage change; this is not an accuracy-only test.

Gan & Low: three-phase distribution optimization (2014); EPRI OpenDSS documentation

# The two models cover different parts of the feeder



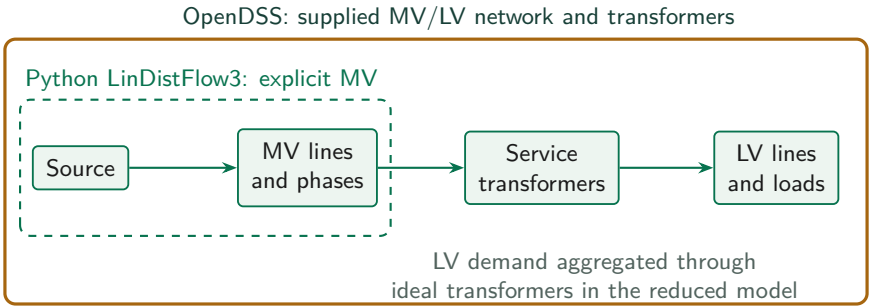

**Reduced-model PASS**
Within the MV-only approximation and its checked limits.

**Full-model PASS**
Within the broader unbalanced snapshot contract.

**Comparison boundary:** equations *and* constraint coverage change; this is not an accuracy-only test.

Gan & Low: three-phase distribution optimization (2014); EPRI OpenDSS documentation

# The two models cover different parts of the feeder




**Reduced-model PASS**
Within the MV-only approximation and its checked limits.

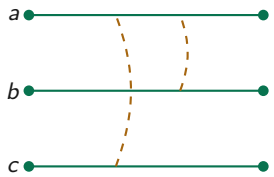
**Full-model PASS**
Within the broader unbalanced snapshot contract.

**Comparison boundary:** equations *and* constraint coverage change; this is not an accuracy-only test.

Gan & Low: three-phase distribution optimization (2014); EPRI OpenDSS documentation

# Three-phase LinDistFlow keeps mutual phase coupling

Parent $i$ Child $j$




$Z_{ij}$ contains self and mutual impedance

$$\mathbf{w}_j = \mathbf{w}_i - 2\,\mathrm{Re}\left[\overline{\beta} \odot \left(Z_{ij}(\beta \odot \overline{\mathbf{S}}_{ij})\right)\right].$$

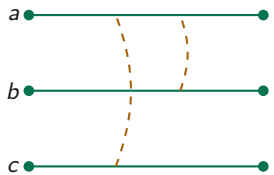
$\mathbf{w}_i, \mathbf{w}_j$: squared phase voltages ($\mathrm{V}^2$);
$Z_{ij}$: coupled phase-impedance matrix ($\Omega$);
$\mathbf{S}_{ij}$: downstream phase powers (VA);
$\beta = (1, e^{-\mathrm{j}2\pi/3}, e^{-\mathrm{j}4\pi/3})$: nominal phase ratios.
$\overline{\cdot}$: conjugate; $\odot$: entrywise multiplication; Re: real part; $\mathrm{j}^2 = -1$.

**The approximation**
Nominal angle ratios and lossless branch balances. Current uses a fixed 1.03-p.u. reference; losses are diagnostic.

Gan & Low: three-phase distribution optimization (2014); exact adapter: `src/lindistflow3.py`

# Three-phase LinDistFlow keeps mutual phase coupling

Parent $i$ Child $j$




$Z_{ij}$ contains self and mutual impedance

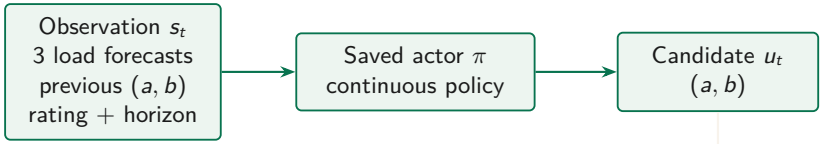
$$\mathbf{w}_j = \mathbf{w}_i - 2\,\mathrm{Re}\left[\overline{\beta} \odot \left(Z_{ij}(\beta \odot \overline{\mathbf{S}}_{ij})\right)\right].$$

$\mathbf{w}_i, \mathbf{w}_j$: squared phase voltages ($\mathrm{V}^2$);
$Z_{ij}$: coupled phase-impedance matrix ($\Omega$);
$\mathbf{S}_{ij}$: downstream phase powers (VA);
$\beta = (1, e^{-\mathrm{j}2\pi/3}, e^{-\mathrm{j}4\pi/3})$: nominal phase ratios.
$\overline{\cdot}$: conjugate; $\odot$: entrywise multiplication; Re: real part; $\mathrm{j}^2 = -1$.

**The approximation**
Nominal angle ratios and lossless branch balances. Current uses a fixed 1.03-p.u. reference; losses are diagnostic.

Gan & Low: three-phase distribution optimization (2014); exact adapter: `src/lindistflow3.py`

# SAC learns a continuous action rule; checking remains separate

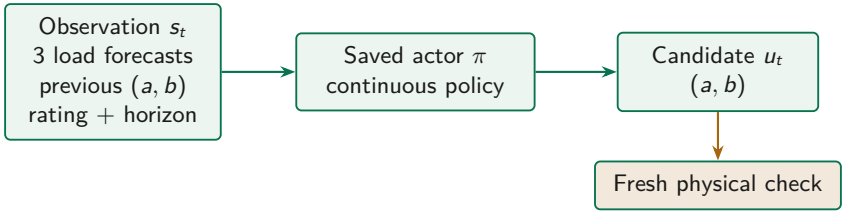



$$\max_{\pi} \mathbb{E}\left[\sum_{t\geq 0} \gamma^t \left\{ \underbrace{r(s_t, u_t)}_{\text{task reward}} + \underbrace{\alpha\, \mathcal{H}(\pi(\cdot \mid s_t))}_{\text{exploration during learning}} \right\}\right].$$

$s_t$: observation; $u_t$: action; $r$: reward; $\pi$: policy; $\mathcal{H}$: entropy; $\alpha$: exploration weight; $\gamma$: discount; $t$: time step; $\mathbb{E}$: expectation over trajectories. **Executed: SAC.** PPO is a continuous-action alternative discussed here, not another trained result.

Haarnoja et al.: SAC, ICML 2018; Schulman et al.: PPO (2017)

# SAC learns a continuous action rule; checking remains separate

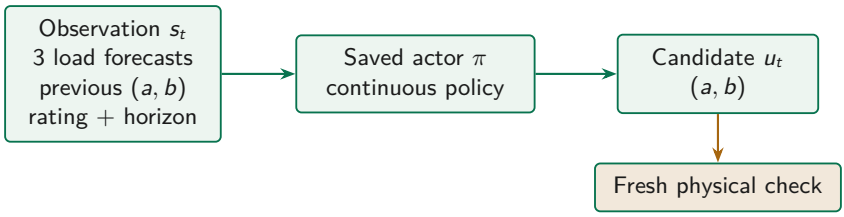



$$\max_{\pi} \mathbb{E}\left[\sum_{t\geq 0} \gamma^t \left\{ \underbrace{r(s_t, u_t)}_{\text{task reward}} + \underbrace{\alpha\, \mathcal{H}(\pi(\cdot \mid s_t))}_{\text{exploration during learning}} \right\}\right].$$

$s_t$: observation; $u_t$: action; $r$: reward; $\pi$: policy; $\mathcal{H}$: entropy; $\alpha$: exploration weight; $\gamma$: discount; $t$: time step; $\mathbb{E}$: expectation over trajectories. **Executed: SAC.** PPO is a continuous-action alternative discussed here, not another trained result.

Haarnoja et al.: SAC, ICML 2018; Schulman et al.: PPO (2017)

# SAC learns a continuous action rule; checking remains separate

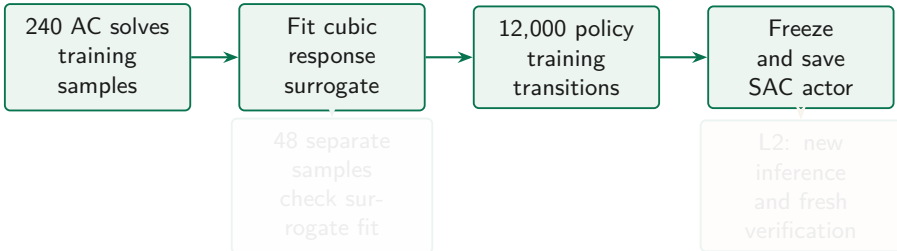



$$\max_{\pi} \mathbb{E}\left[\sum_{t\geq 0} \gamma^t \left\{ \underbrace{r(s_t, u_t)}_{\text{task reward}} + \underbrace{\alpha\, \mathcal{H}(\pi(\cdot \mid s_t))}_{\text{exploration during learning}} \right\}\right].$$

$s_t$: observation; $u_t$: action; $r$: reward; $\pi$: policy; $\mathcal{H}$: entropy; $\alpha$: exploration weight; $\gamma$: discount; $t$: time step; $\mathbb{E}$: expectation over trajectories. **Executed: SAC.** PPO is a continuous-action alternative discussed here, not another trained result.

Haarnoja et al.: SAC, ICML 2018; Schulman et al.: PPO (2017)

# Training provenance prevents a misleading speed claim

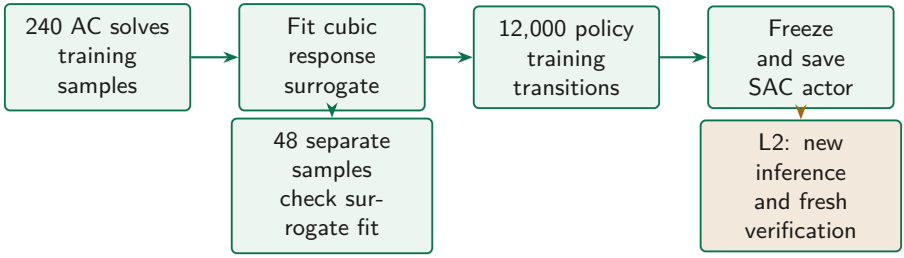



**What the count means**
12,000 transitions on the fitted surrogate, not 12,000 full AC solves.

**What transfer means**
A new load or rating condition can challenge the frozen actor; L2 does not retrain it.

Haarnoja et al.: SAC, ICML 2018; executed training artifact: `Continuous_SAC_Training.ipynb`

# Training provenance prevents a misleading speed claim




**What the count means**

12,000 transitions on the fitted surrogate, not 12,000 full AC solves.

**What transfer means**

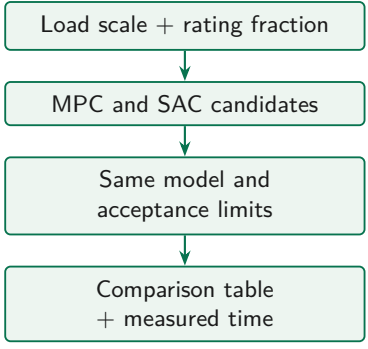
A new load or rating condition can challenge the frozen actor; L2 does not retrain it.

Haarnoja et al.: SAC, ICML 2018; executed training artifact: `Continuous_SAC_Training.ipynb`

# L2 recomputes both proposals under one declared scenario

**Notebook L2** Fresh MPC solve + saved SAC inference + fresh numerical checks.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
timings['L2 fresh MPC + SAC + checks'] = time.perf_counter()
    - started
print(voltage['mode'])
display(voltage['table'])
```




**Read the timer correctly**

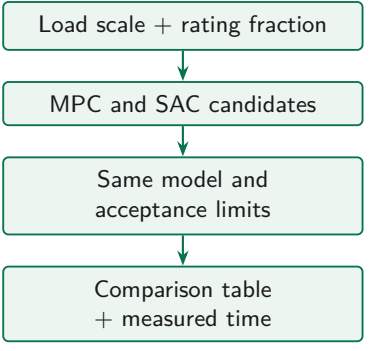
It encloses the helper call: planning, actor inference and checks. It is not isolated SAC inference latency.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

# L2 recomputes both proposals under one declared scenario

**Notebook L2** Fresh MPC solve + saved SAC inference + fresh numerical checks.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
timings['L2 fresh MPC + SAC + checks'] = time.perf_counter()
    - started
print(voltage['mode'])
display(voltage['table'])
```




**Read the timer correctly**

It encloses the helper call: planning, actor inference and checks. It is not isolated SAC inference latency.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

# Lab: predict, change demand, then explain the response

**Notebook L2** Seven minutes in pairs. Run the full cell, not only this excerpt.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE,
     RATING_FRACTION)
```

1. Predict the effect of higher demand.
2. Run the default case.
3. Change only `LOAD_SCALE`; rerun L2.
4. Explain the limiting margin, not just the winner.

| Record | Default | Changed |
|---|---|---|
| Feasible? | _____ | _____ |
| Objective | _____ | _____ |
| Voltage margin | _____ | _____ |
| Current margin | _____ | _____ |

**Predict:** Must higher demand increase PV curtailment?
**Expected:** No. Voltage and current constraints can move in different directions.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

# Lab: predict, change demand, then explain the response

**Notebook L2** Seven minutes in pairs. Run the full cell, not only this excerpt.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE,
     RATING_FRACTION)
```

1. Predict the effect of higher demand.
2. Run the default case.
3. Change only `LOAD_SCALE`; rerun L2.
4. Explain the limiting margin, not just the winner.

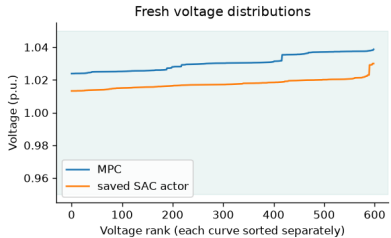
| Record | Default | Changed |
|---|---|---|
| Feasible? | _____ | _____ |
| Objective | _____ | _____ |
| Voltage margin | _____ | _____ |
| Current margin | _____ | _____ |

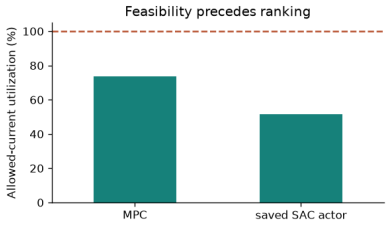
**Predict:** Must higher demand increase PV curtailment?

**Explanation:** No. Voltage and current constraints can move in different directions.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

# Inspect the physical margins before ranking the controllers




| Fresh L2 default | $J$ | MV gate |
|---|---|---|
| MPC | 0.031104 | PASS |
| Saved SAC actor | 0.436428 | PASS |

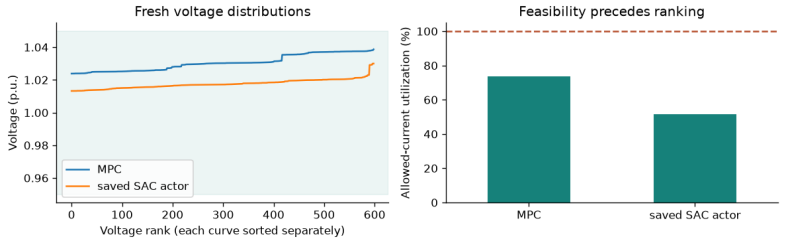
$J$: current-step teaching cost. Both actions pass this reduced-model contract. The voltage curves are sorted separately: equal rank is not equal bus identity.

**Attribute the result correctly**

The cost gap is a controller result. It does not, by itself, measure the benefit of agent orchestration.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

# Inspect the physical margins before ranking the controllers




| Fresh L2 default | $J$ | MV gate |
|---|---|---|
| MPC | 0.031104 | PASS |
| Saved SAC actor | 0.436428 | PASS |

$J$: current-step teaching cost. Both actions pass this reduced-model contract. The voltage curves are sorted separately: equal rank is not equal bus identity.

**Attribute the result correctly**

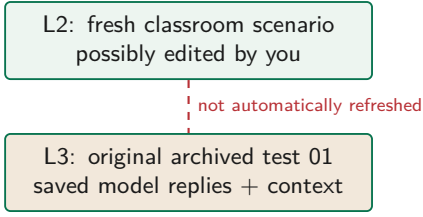
The cost gap is a controller result. It does not, by itself, measure the benefit of agent orchestration.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

# L3 opens a different, explicitly archived context

**Notebook L3** Inspect original `test_01`; these are actual recorded model responses.

```
pool = lab.saved_pool('test_01')
display(pd.DataFrame(pool['responses'])[['model_id', '
    operation_id', 'reason', 'seconds', 'source']])
display(pd.DataFrame([{'operation': ', '.join(c['
    operation_ids']),
   'votes': c['vote_count'], 'share': c['vote_share'],
    'parameters': c['operation']['parameters']}
   for c in pool['pool']['candidates']]))
print('ARCHIVED ACTUAL LLM; original context:', pool['
    context']['id'])
print('Distinct physical candidates:', len(pool['pool']['
    candidates']))
print('Malformed/duplicate votes rejected:', len(pool['
    pool']['rejected']))
```

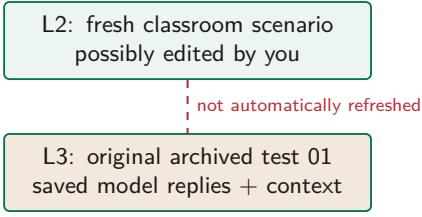



Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# L3 opens a different, explicitly archived context

**Notebook L3** Inspect original `test_01`; these are actual recorded model responses.

```
pool = lab.saved_pool('test_01')
display(pd.DataFrame(pool['responses'])[['model_id', '
    operation_id', 'reason', 'seconds', 'source']])
display(pd.DataFrame([{'operation': ', '.join(c['
    operation_ids']),
   'votes': c['vote_count'], 'share': c['vote_share'],
    'parameters': c['operation']['parameters']}
   for c in pool['pool']['candidates']]))
print('ARCHIVED ACTUAL LLM; original context:', pool['
    context']['id'])
print('Distinct physical candidates:', len(pool['pool']['
    candidates']))
print('Malformed/duplicate votes rejected:', len(pool['
    pool']['rejected']))
```




**Read the provenance**

`reason`: exact public response.
`seconds`: recorded call time.
Printing the table makes no new LLM call.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# Read the actual replies, then challenge their claims

Exact public excerpts, archived original test 01. Llama is the model family; Ollama is the local runtime.

**Llama 3.1 8B 1.867 s**
Trial: increase $a$ by 0.0025.

*Increasing active participation by 0.0025 may help reduce curtailment and improve the overall performance.*

**Llama 3.2 3B 0.905 s**
Trial: reduce $|b|$ by 10%.

*Reducing reactive effort magnitude by ten percent may help improve the overall system performance and reduce curtailment*

**Qwen 2.5 7B 1.844 s**
Trial: reduce $|b|$ by 10%.

*may help improve voltage stability without significantly degrading active power performance*

**Predict:** Which phrases need evidence beyond a plausible explanation?

**Reactive effort ≠ curtailment**
Reducing $|b|$ does not itself prove higher active generation.

**Snapshot ≠ stability proof**
Steady-state power flow does not establish voltage stability.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# Read the actual replies, then challenge their claims

Exact public excerpts, archived original test 01. Llama is the model family; Ollama is the local runtime.

**Llama 3.1 8B 1.867 s**
Trial: increase $a$ by 0.0025.

*Increasing active participation by 0.0025 may help reduce curtailment and improve the overall performance.*

**Llama 3.2 3B 0.905 s**
Trial: reduce $|b|$ by 10%.

*Reducing reactive effort magnitude by ten percent may help improve the overall system performance and reduce curtailment*

**Qwen 2.5 7B 1.844 s**
Trial: reduce $|b|$ by 10%.

*may help improve voltage stability without significantly degrading active power performance*

**Predict:** Which phrases need evidence beyond a plausible explanation?

**Reactive effort $\neq$ curtailment**
Reducing $|b|$ does not itself prove higher active generation.

**Snapshot $\neq$ stability proof**
Steady-state power flow does not establish voltage stability.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# Read the actual replies, then challenge their claims

Exact public excerpts, archived original test 01. Llama is the model family; Ollama is the local runtime.

**Llama 3.1 8B 1.867 s**
Trial: increase $a$ by 0.0025.

*Increasing active participation by 0.0025 may help reduce curtailment and improve the overall performance.*

**Llama 3.2 3B 0.905 s**
Trial: reduce $|b|$ by 10%.

*Reducing reactive effort magnitude by ten percent may help improve the overall system performance and reduce curtailment*

**Qwen 2.5 7B 1.844 s**
Trial: reduce $|b|$ by 10%.

*may help improve voltage stability without significantly degrading active power performance*

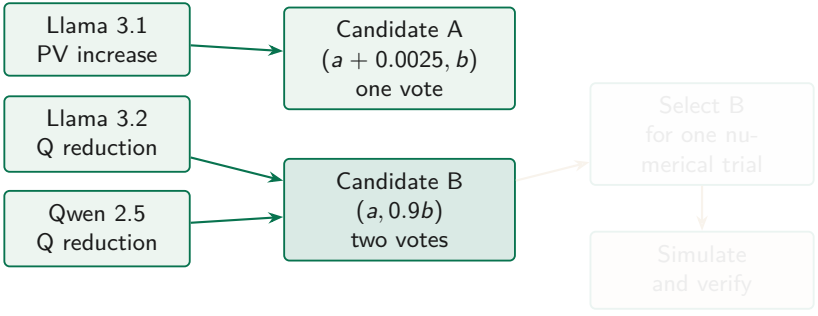
**Predict:** Which phrases need evidence beyond a plausible explanation?

**Reactive effort $\neq$ curtailment**
Reducing $|b|$ does not itself prove higher active generation.

**Snapshot $\neq$ stability proof**
Steady-state power flow does not establish voltage stability.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# Canonicalize actions before counting votes



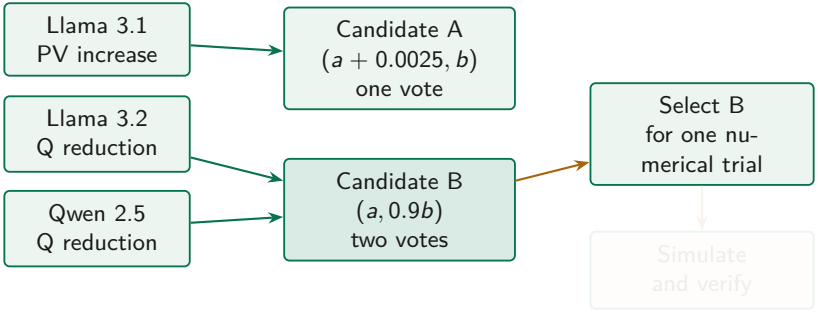

$a, b$: incumbent fleet command. Tool name and parameters identify the same physical operation; prose style does not create a new action.

**A majority is a selection rule**

Correlated language models can share an error. Agreement is not a calibrated probability of safety.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# Canonicalize actions before counting votes



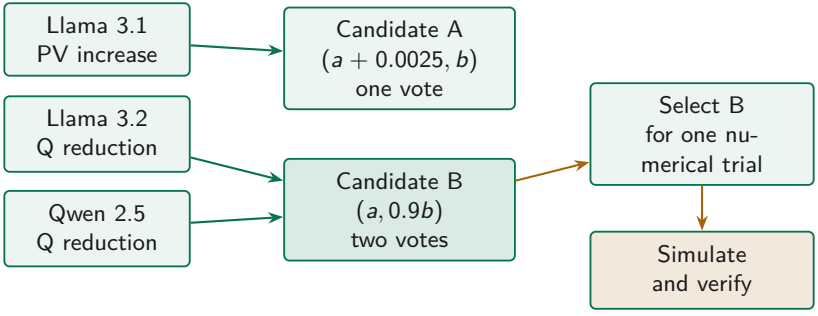

$a, b$: incumbent fleet command. Tool name and parameters identify the same physical operation; prose style does not create a new action.

**A majority is a selection rule**

Correlated language models can share an error. Agreement is not a calibrated probability of safety.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# Canonicalize actions before counting votes



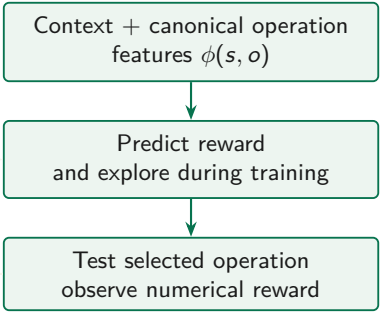

$a, b$: incumbent fleet command. Tool name and parameters identify the same physical operation; prose style does not create a new action.

**A majority is a selection rule**

Correlated language models can share an error. Agreement is not a calibrated probability of safety.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# LinUCB learns which proposed operation is worth testing




$$q(o \mid s) = \underbrace{\theta^\top \phi(s,o)}_{\text{predicted reward}} + \underbrace{\beta\sqrt{\phi(s,o)^\top A^{-1}\phi(s,o)}}_{\text{training exploration bonus}},$$

$$o^* = \arg\max_o q(o \mid s).$$

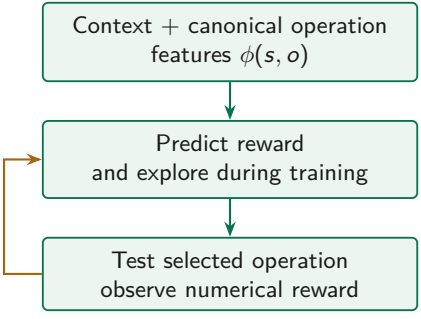
$s$: evidence context; $o$: canonical operation; $\phi$: features; $\theta$: learned coefficients; $A$: regularized feature information; $\beta$: exploration weight; $q$: score; $o^*$: selected operation.

**Frozen classroom evaluation**

Use predicted reward only, with no updates. All LLM replies already exist: this learner does not choose which model to call or train LLM weights.

Li et al.: LinUCB, WWW 2010; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# LinUCB learns which proposed operation is worth testing




$$q(o \mid s) = \underbrace{\theta^\top \phi(s,o)}_{\text{predicted reward}} + \underbrace{\beta\sqrt{\phi(s,o)^\top A^{-1}\phi(s,o)}}_{\text{training exploration bonus}},$$

$$o^* = \arg\max_o q(o \mid s).$$

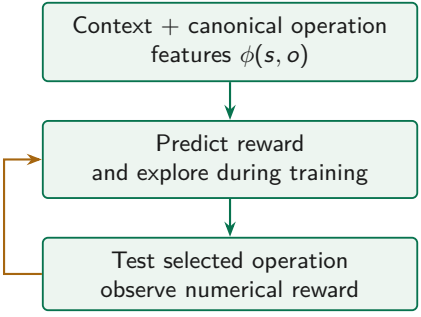
$s$: evidence context; $o$: canonical operation; $\phi$: features; $\theta$: learned coefficients; $A$: regularized feature information; $\beta$: exploration weight; $q$: score; $o^*$: selected operation.

**Frozen classroom evaluation**

Use predicted reward only, with no updates. All LLM replies already exist: this learner does not choose which model to call or train LLM weights.

Li et al.: LinUCB, WWW 2010; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# LinUCB learns which proposed operation is worth testing




$$q(o \mid s) = \underbrace{\theta^\top \phi(s,o)}_{\text{predicted reward}} + \underbrace{\beta\sqrt{\phi(s,o)^\top A^{-1}\phi(s,o)}}_{\text{training exploration bonus}},$$

$$o^* = \arg\max_o q(o \mid s).$$

$s$: evidence context; $o$: canonical operation; $\phi$: features; $\theta$: learned coefficients; $A$: regularized feature information; $\beta$: exploration weight; $q$: score; $o^*$: selected operation.

**Frozen classroom evaluation**

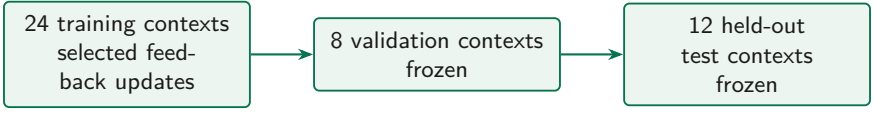
Use predicted reward only, with no updates. All LLM replies already exist: this learner does not choose which model to call or train LLM weights.

Li et al.: LinUCB, WWW 2010; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# Reward learning and physical acceptance have different jobs

$$r = \begin{cases} \min\left(1, \dfrac{\max(0, J_{\text{inc}} - J_{\text{trial}})}{0.01}\right) - 0.01, & \text{trial passes,} \\ -1.01, & \text{trial rejected.} \end{cases}$$

$r$: post-trial reward; $J_{\text{inc}}, J_{\text{trial}}$: incumbent and trial current-step costs. The 0.01 charge is a fixed trial penalty, *not seconds*.




**Predict:** The learner predicts improvement, but the trial violates current. Accept?

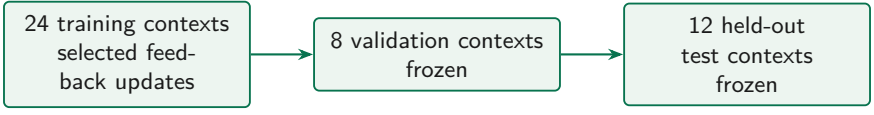
Reject the trial. Keep a valid incumbent or recorded reward. Does not grant permission.

Li et al.: LinUCB; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# Reward learning and physical acceptance have different jobs

$$r = \begin{cases} \min\left(1, \dfrac{\max(0, J_{\text{inc}} - J_{\text{trial}})}{0.01}\right) - 0.01, & \text{trial passes,} \\ -1.01, & \text{trial rejected.} \end{cases}$$

$r$: post-trial reward; $J_{\text{inc}}, J_{\text{trial}}$: incumbent and trial current-step costs. The 0.01 charge is a fixed trial penalty, *not seconds*.




**Predict:** The learner predicts improvement, but the trial violates current. Accept?

**Explanation:** Reject the trial. Keep a valid incumbent or escalate; reward does not grant permission.

Li et al.: LinUCB; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# L4: verify the frozen choice with a new numerical trial

**Notebook L4** Three minutes: run the full cell and explain the output.

```
decision = pool['selection']
print('Frozen:', pool['frozen'], '| selected-feedback training
    updates:', pool['training_updates'])
print('Bandit operation:', decision.operation_id, '| predicted
    reward:', round(decision.predicted_reward, 4))
print('Same canonical choice as majority:', decision.
    candidate_id == pool['vote_id'])
started = time.perf_counter()
selected_trials = lab.trial_saved_selection(pool)
timings['L4 new verification of archived choices'] = time.
    perf_counter() - started
display(selected_trials['table'])
```

**Before the trial**
Which canonical operation did the learner choose? Does it match the majority?

**After the trial**
Did the fresh gate pass? What current-step cost was obtained?

**Default:** learner and vote select Q reduction; original-context $J \approx 0.031082$.

This evaluates archived original choices in their original context. Do not subtract this cost from a changed L2 scenario's cost.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# L4: verify the frozen choice with a new numerical trial

**Notebook L4** Three minutes: run the full cell and explain the output.

```
decision = pool['selection']
print('Frozen:', pool['frozen'], '| selected-feedback training
    updates:', pool['training_updates'])
print('Bandit operation:', decision.operation_id, '| predicted
    reward:', round(decision.predicted_reward, 4))
print('Same canonical choice as majority:', decision.
    candidate_id == pool['vote_id'])
started = time.perf_counter()
selected_trials = lab.trial_saved_selection(pool)
timings['L4 new verification of archived choices'] = time.
    perf_counter() - started
display(selected_trials['table'])
```

**Before the trial**
Which canonical operation did the learner choose? Does it match the majority?

**After the trial**
Did the fresh gate pass? What current-step cost was obtained?

**Default:** learner and vote select Q reduction; original-context $J \approx 0.031082$.

This evaluates archived original choices in their original context. Do not subtract this cost from a changed L2 scenario's cost.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# L4: verify the frozen choice with a new numerical trial

**Notebook L4** Three minutes: run the full cell and explain the output.

```
decision = pool['selection']
print('Frozen:', pool['frozen'], '| selected-feedback training
    updates:', pool['training_updates'])
print('Bandit operation:', decision.operation_id, '| predicted
    reward:', round(decision.predicted_reward, 4))
print('Same canonical choice as majority:', decision.
    candidate_id == pool['vote_id'])
started = time.perf_counter()
selected_trials = lab.trial_saved_selection(pool)
timings['L4 new verification of archived choices'] = time.
    perf_counter() - started
display(selected_trials['table'])
```

**Before the trial**
Which canonical operation did the learner choose? Does it match the majority?

**After the trial**
Did the fresh gate pass? What current-step cost was obtained?

**Default:** learner and vote select Q reduction; original-context $J \approx 0.031082$.

This evaluates archived original choices in their original context. Do not subtract this cost from a changed L2 scenario's cost.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# A fair experiment can find no learned advantage

Recorded held-out MV pilot: 12 scenarios; lower $J$ is better under the same reduced-model contract.

| Selection method | Mean $J$ | Same-action full AC PASS |
|---|---|---|
| Fixed numerical refinement | 0.03400656 | 0/12 |
| Majority vote | 0.03398479 | 0/12 |
| Frozen learned selector | 0.03398479 | 0/12 |

**Observed**
Vote and learner tie on every held-out case.

**Diagnostic**
A posthoc fixed Q-reduction rule reproduces their choices without LLM calls.

**What this supports**
Reproducible trials and explicit checking are demonstrated. Superior control from extra LLMs or a learned selector is not established.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# A fair experiment can find no learned advantage

Recorded held-out MV pilot: 12 scenarios; lower $J$ is better under the same reduced-model contract.

| Selection method | Mean $J$ | Same-action full AC PASS |
|---|---|---|
| Fixed numerical refinement | 0.03400656 | 0/12 |
| Majority vote | 0.03398479 | 0/12 |
| Frozen learned selector | 0.03398479 | 0/12 |

**Observed**
Vote and learner tie on every held-out case.

**Diagnostic**
A posthoc fixed Q-reduction rule reproduces their choices without LLM calls.

**What this supports**
Reproducible trials and explicit checking are demonstrated. Superior control from extra LLMs or a learned selector is not established.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# A fair experiment can find no learned advantage

Recorded held-out MV pilot: 12 scenarios; lower $J$ is better under the same reduced-model contract.

| Selection method | Mean $J$ | Same-action full AC PASS |
|---|---|---|
| Fixed numerical refinement | 0.03400656 | 0/12 |
| Majority vote | 0.03398479 | 0/12 |
| Frozen learned selector | 0.03398479 | 0/12 |

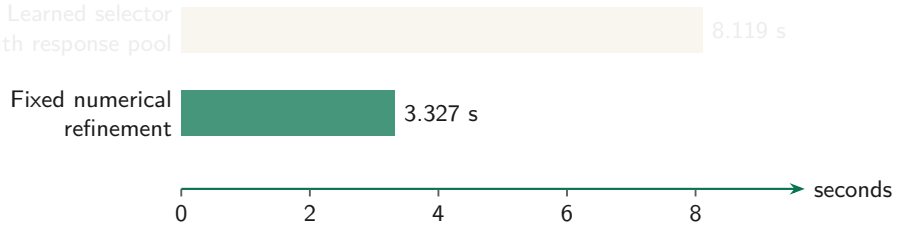
**Observed**
Vote and learner tie on every held-out case.

**Diagnostic**
A posthoc fixed Q-reduction rule reproduces their choices without LLM calls.

**What this supports**
Reproducible trials and explicit checking are demonstrated. Superior control from extra LLMs or a learned selector is not established.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# Report the latency of the entire decision path



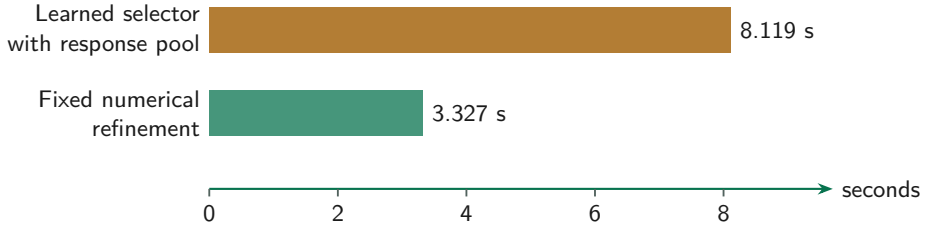

Recorded component times, reconstructed serial accounting for the held-out MV pilot. These are not guarantees for a deployed control room.

**Count all work**
Every pool selector pays for all three LLM responses, plus numerical trials and checks.

**Separate timing boundaries**
Training, model loading, token generation, actor inference and end-to-end execution are different quantities.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`; Ollama chat API timing fields

# Report the latency of the entire decision path




Recorded component times, reconstructed serial accounting for the held-out MV pilot. These are not guarantees for a deployed control room.

**Count all work**
Every pool selector pays for all three LLM responses, plus numerical trials and checks.

**Separate timing boundaries**
Training, model loading, token generation, actor inference and end-to-end execution are different quantities.

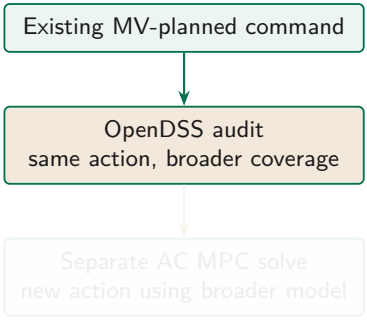
Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`; Ollama chat API timing fields

# L5 separates same-action auditing from AC replanning

**Notebook L5** Default: fresh MV audit. Optional: full AC audit and a separate AC MPC solve.

```
started = time.perf_counter()
scope_audit = lab.backend_audit(pool, run_opendss=
    RUN_OPENDSS)
timings['L5 selected model scope checks'] = time.
    perf_counter() - started
display(scope_audit['table'])
print(scope_audit['optional_mode'])
print('An MV-only PASS cannot acknowledge a request
    requiring full MV/LV coverage.')
```

Enable `RUN_OPENDSS` in L0 for the optional path. Run OpenDSS evaluations sequentially.




**Two questions**

Does the old recommendation survive the audit? What would a planner using the broader model recommend instead?

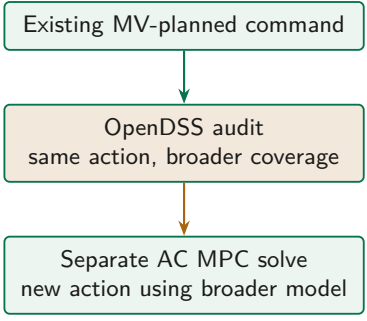
Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; EPRI OpenDSS documentation

# L5 separates same-action auditing from AC replanning

**Notebook L5** Default: fresh MV audit. Optional: full AC audit and a separate AC MPC solve.

```
started = time.perf_counter()
scope_audit = lab.backend_audit(pool, run_opendss=
    RUN_OPENDSS)
timings['L5 selected model scope checks'] = time.
    perf_counter() - started
display(scope_audit['table'])
print(scope_audit['optional_mode'])
print('An MV-only PASS cannot acknowledge a request
    requiring full MV/LV coverage.')
```

Enable `RUN_OPENDSS` in L0 for the optional path. Run OpenDSS evaluations sequentially.




**Two questions**

Does the old recommendation survive the audit? What would a planner using the broader model recommend instead?

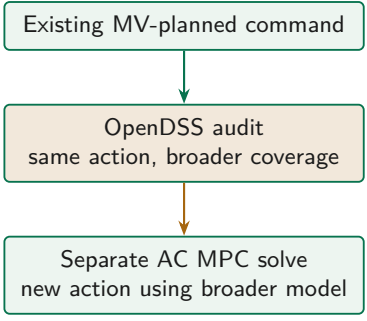
Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; EPRI OpenDSS documentation

# L5 separates same-action auditing from AC replanning

**Notebook L5** Default: fresh MV audit. Optional: full AC audit and a separate AC MPC solve.

```
started = time.perf_counter()
scope_audit = lab.backend_audit(pool, run_opendss=
    RUN_OPENDSS)
timings['L5 selected model scope checks'] = time.
    perf_counter() - started
display(scope_audit['table'])
print(scope_audit['optional_mode'])
print('An MV-only PASS cannot acknowledge a request
    requiring full MV/LV coverage.')
```

Enable `RUN_OPENDSS` in L0 for the optional path. Run OpenDSS evaluations sequentially.

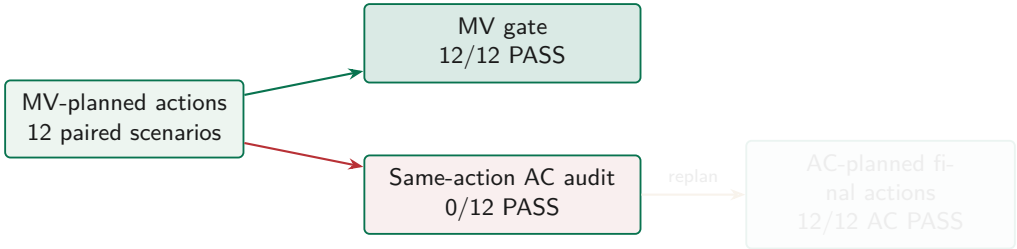



**Two questions**

Does the old recommendation survive the audit? What would a planner using the broader model recommend instead?

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; EPRI OpenDSS documentation

# A reduced-model PASS can fail the full-feeder audit



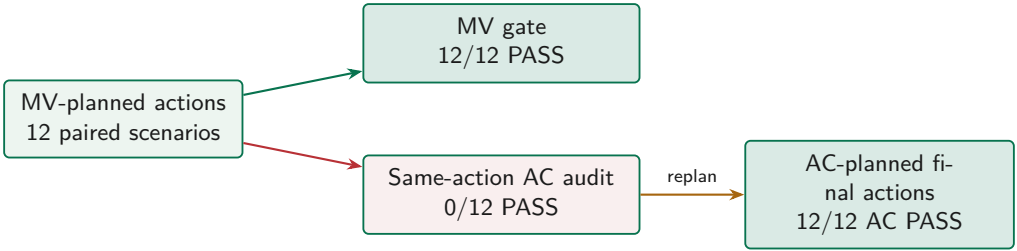

**What changed**
Numerical equations and network/constraint coverage. The selector remained frozen.

**What did not follow**
This does not prove an LLM caused the better physical result, nor does PASS constitute human approval.

Separately recorded 12-case pilot; not the output of one classroom L5 execution.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# A reduced-model PASS can fail the full-feeder audit




**What changed**
Numerical equations and network/constraint coverage. The selector remained frozen.

**What did not follow**
This does not prove an LLM caused the better physical result, nor does PASS constitute human approval.

Separately recorded 12-case pilot; not the output of one classroom L5 execution.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# The AC comparison still needs a strong numerical baseline

Recorded paired AC pilot, 12 scenarios.

| Method | Mean current-step $J$ |
|---|---|
| MPC first action | 0.47172417 |
| Fixed refinement | 0.47112138 |
| Uniform pool selection | 0.47110513 |
| Majority vote | 0.47138488 |
| Frozen learned selection | 0.47138488 |

**Interpretation**

Vote and learner tie. The learned selector does not beat the fixed or uniform comparators here.

**Check the raw policy too**

Only 1/12 raw SAC actions passes the AC gate in this pilot. A fast proposal is not automatically usable.

**Why this is a scientific result**

Hold scenario, feasible set, budget and evaluation boundary constant. Attribute changes to the controller, model, information or orchestration mechanism that actually changed.

Do not compare AC and MV objective means as a pure solver ranking: the modeled voltage sets differ.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# The AC comparison still needs a strong numerical baseline

Recorded paired AC pilot, 12 scenarios.

| Method | Mean current-step $J$ |
|---|---|
| MPC first action | 0.47172417 |
| Fixed refinement | 0.47112138 |
| Uniform pool selection | 0.47110513 |
| Majority vote | 0.47138488 |
| Frozen learned selection | 0.47138488 |

**Interpretation**

Vote and learner tie. The learned selector does not beat the fixed or uniform comparators here.

**Check the raw policy too**

Only 1/12 raw SAC actions passes the AC gate in this pilot. A fast proposal is not automatically usable.

**Why this is a scientific result**

Hold scenario, feasible set, budget and evaluation boundary constant. Attribute changes to the controller, model, information or orchestration mechanism that actually changed.

Do not compare AC and MV objective means as a pure solver ranking: the modeled voltage sets differ.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# The AC comparison still needs a strong numerical baseline

Recorded paired AC pilot, 12 scenarios.

| Method | Mean current-step $J$ |
|---|---|
| MPC first action | 0.47172417 |
| Fixed refinement | 0.47112138 |
| Uniform pool selection | 0.47110513 |
| Majority vote | 0.47138488 |
| Frozen learned selection | 0.47138488 |

**Interpretation**

Vote and learner tie. The learned selector does not beat the fixed or uniform comparators here.

**Check the raw policy too**

Only 1/12 raw SAC actions passes the AC gate in this pilot. A fast proposal is not automatically usable.

**Why this is a scientific result**

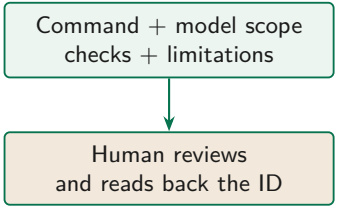
Hold scenario, feasible set, budget and evaluation boundary constant. Attribute changes to the controller, model, information or orchestration mechanism that actually changed.

Do not compare AC and MV objective means as a pure solver ranking: the modeled voltage sets differ.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# The operator receives a scoped recommendation and its evidence

| Stage | Returned evidence in this lesson |
|---|---|
| Awareness | Load, rating fraction, requested scope, revision |
| Planning | MPC and SAC candidate commands |
| Simulation | New voltages, currents and objective |
| Verification | PASS / failures; candidate-bound model evidence |
| LLM optimizer | Permitted trial and recorded public rationale |




**Full scope requested?**
An MV-only PASS remains HOLD for a request requiring full MV/LV coverage.

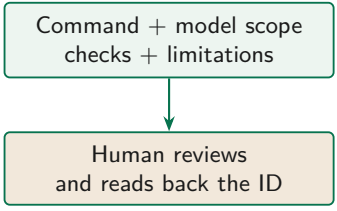
**Demonstrated benefit and boundary**
The workflow makes assumptions, trials and failures inspectable. Extra LLMs did not demonstrate better control here. Readback is advisory, not equipment dispatch.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`, L1–L5 and L12; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

**Next:** After the break: what changes when the action is a topology rather than a setpoint?

# The operator receives a scoped recommendation and its evidence

| Stage | Returned evidence in this lesson |
|---|---|
| Awareness | Load, rating fraction, requested scope, revision |
| Planning | MPC and SAC candidate commands |
| Simulation | New voltages, currents and objective |
| Verification | PASS / failures; candidate-bound model evidence |
| LLM optimizer | Permitted trial and recorded public rationale |



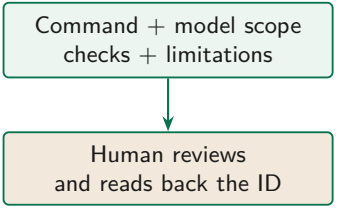

**Full scope requested?**
An MV-only PASS remains HOLD for a request requiring full MV/LV coverage.

**Demonstrated benefit and boundary**
The workflow makes assumptions, trials and failures inspectable. Extra LLMs did not demonstrate better control here. Readback is advisory, not equipment dispatch.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`, L1–L5 and L12; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# The operator receives a scoped recommendation and its evidence

| Stage | Returned evidence in this lesson |
|---|---|
| Awareness | Load, rating fraction, requested scope, revision |
| Planning | MPC and SAC candidate commands |
| Simulation | New voltages, currents and objective |
| Verification | PASS / failures; candidate-bound model evidence |
| LLM optimizer | Permitted trial and recorded public rationale |




**Full scope requested?**
An MV-only PASS remains HOLD for a request requiring full MV/LV coverage.

**Demonstrated benefit and boundary**
The workflow makes assumptions, trials and failures inspectable. Extra LLMs did not demonstrate better control here. Readback is advisory, not equipment dispatch.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`, L1–L5 and L12; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# Ten-minute break: 80–90 minutes

Keep your voltage result and its model scope.

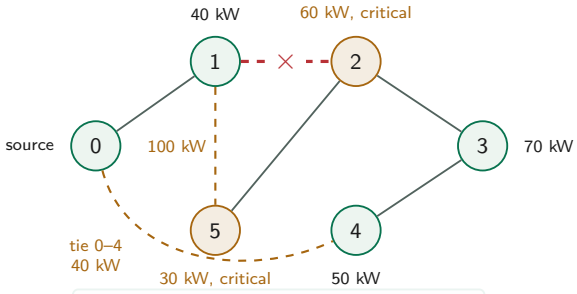
**What changes after the break?**

For voltage support, we adjusted inverter controls on a given network. After a fault, some paths are unavailable. We must choose which loads the remaining paths can serve.

**Predict:** Would the largest number of restored kilowatts always be the right objective?

Workshop timetable. Take the full ten-minute break.

# A fault changes the paths that are available




Fault 1–2 stays isolated.
Check ties only after isolation.

## A small, transparent fixture

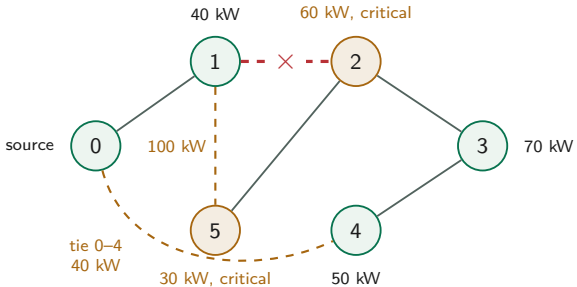
Six nodes; five loads; one failed segment; two optional ties. Amber nodes have higher service priority.

**Predict:** Can we simply reconnect the failed segment?

**Explanation:** No. Its availability is fixed to zero. Planning must find another admissible path.

Fresh classroom fixture: Classroom_Labs L6; src/standard_formulations.py. Restoration context: Chen et al., IEEE TPWRS, 2018.

# A fault changes the paths that are available




**A small, transparent fixture**

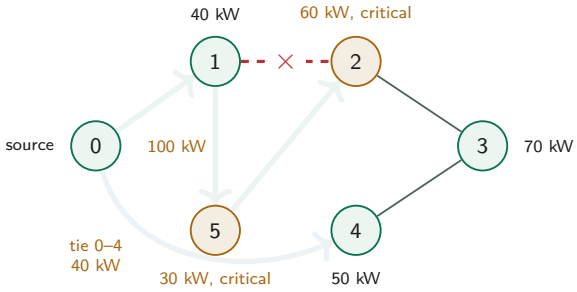
Six nodes; five loads; one failed segment; two optional ties. Amber nodes have higher service priority.

**Predict:** Can we simply reconnect the failed segment?

**Explanation:** No. Its availability is fixed to zero. Planning must find another admissible path.

Fresh classroom fixture: Classroom_Labs L6; src/standard_formulations.py. Restoration context: Chen et al., IEEE TPWRS, 2018.

# Two ties create choices, with different bottlenecks




**Follow the power**

The route $0 \to 1 \to 5 \to 2$ uses the 100 kW tie. The other tie has only 40 kW in this first case.

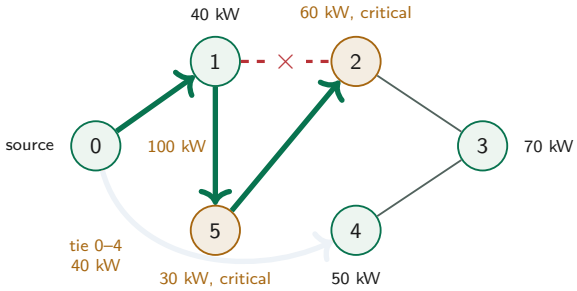
**Predict:** Can the 40 kW tie pick up the entire 50 kW load at node 4?

No: $50 > 40$. Loads are indivisible here; partial pickup is excluded.

Fixture and binary pickup assumption: Classroom_Labs L6; standard_formulations.restoration_graph. Blue and teal paths are explanatory alternatives, not a switching sequence.

**Next:** Does an energized node have to serve its local load?

# Two ties create choices, with different bottlenecks




**Follow the power**

The route $0 \to 1 \to 5 \to 2$ uses the 100 kW tie. The other tie has only 40 kW in this first case.

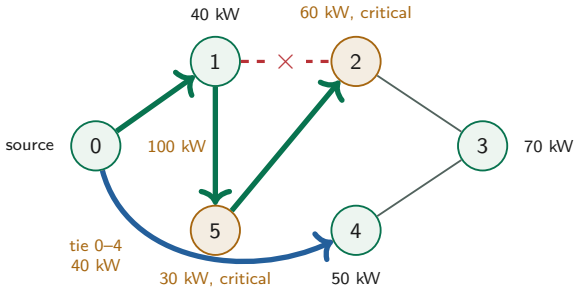
**Predict:** Can the 40 kW tie pick up the entire 50 kW load at node 4?

No: $50 > 40$. Loads are indivisible here; partial pickup is excluded.

Fixture and binary pickup assumption: Classroom_Labs L6; standard_formulations.restoration_graph. Blue and teal paths are explanatory alternatives, not a switching sequence.

**Next:** Does an energized node have to serve its local load?

# Two ties create choices, with different bottlenecks




**Follow the power**

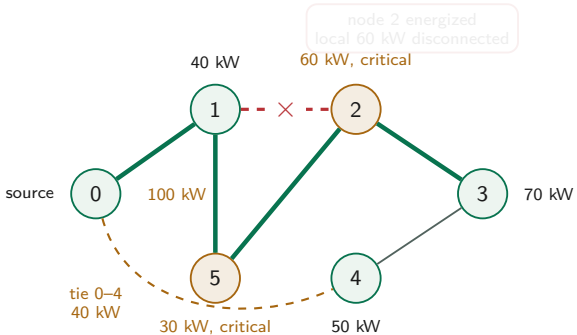
The route $0 \to 1 \to 5 \to 2$ uses the 100 kW tie. The other tie has only 40 kW in this first case.

**Predict:** Can the 40 kW tie pick up the entire 50 kW load at node 4? No: $50 > 40$. Loads are indivisible here; partial pickup is excluded.

Fixture and binary pickup assumption: Classroom_Labs L6; standard_formulations.restoration_graph. Blue and teal paths are explanatory alternatives, not a switching sequence.

**Next:** Does an energized node have to serve its local load?

# A node can carry power while its load stays disconnected




Example: serve nodes 1, 3, 5; pass through node 2.

## Two different binary decisions

$y_i = 1$: node $i$ is energized.
$s_i = 1$: its load is picked up.

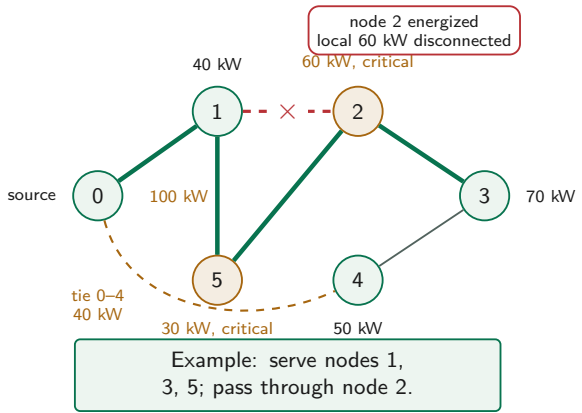
We require $s_i \le y_i$.

**Predict:** Why is equality $s_i = y_i$ too restrictive?

**Explanation:** It would forbid a useful transit node whose own load remains shed.

Original schematic of a feasible six-node plan; binary pickup/energization model in src/standard_formulations.py.

# A node can carry power while its load stays disconnected




**Two different binary decisions**

$y_i = 1$: node $i$ is energized.
$s_i = 1$: its load is picked up.

We require $s_i \le y_i$.

**Predict:** Why is equality $s_i = y_i$ too restrictive?

**Explanation:** It would forbid a useful transit node whose own load remains shed.

Original schematic of a feasible six-node plan; binary pickup/energization model in src/standard_formulations.py.

# A capacity budget turns priorities into a real choice

Critical pair: $60 + 30 = 90$ kW







Largest total: $70 + 30 = 100$ kW

**The 100 kW tie is the budget**

Both choices also serve node 1's 40 kW upstream.
Critical pair: total 130; critical 90.
Largest total: total 140; critical 30.

**Predict:** Which outcome should a critical-service policy prefer?

**Explanation:** The answer depends on the stated objective, not on how persuasive its explanation sounds.

Hand-checkable choices at tie 0–4 capacity 40 kW; exact optima confirmed by Classroom_Labs L6. Bars represent kW, not physical line lengths.

# A capacity budget turns priorities into a real choice

Critical pair: $60 + 30 = 90$ kW







Largest total: $70 + 30 = 100$ kW

**The 100 kW tie is the budget**

Both choices also serve node 1's 40 kW upstream.
Critical pair: total 130; critical 90.
Largest total: total 140; critical 30.

**Predict:** Which outcome should a critical-service policy prefer?

**Explanation:** The answer depends on the stated objective, not on how persuasive its explanation sounds.

Hand-checkable choices at tie 0–4 capacity 40 kW; exact optima confirmed by Classroom_Labs L6. Bars represent kW, not physical line lengths.

# Compare outcomes before comparing algorithms

| Same 40 kW tie case | Total served | Critical served | Critical share served |
|---|---|---|---|
| Maximize total kW | 140 kW | 30 kW | 30/90 |
| Maximize weighted kW | 130 kW | 90 kW | 90/90 |

**What the experiment establishes**

Changing the service objective changes who benefits. It does not establish an advantage caused by an LLM or by the word "agentic."

**Predict:** What must stay matched to isolate an agentic workflow effect?

**Explanation:** The objective, candidate tools, physical limits, information and verification rules.

Fresh six-node experiment: Classroom_Labs L6, weighted MILP versus unweighted exact enumeration; both use the same feasible constraints.

# Compare outcomes before comparing algorithms

| Same 40 kW tie case | Total served | Critical served | Critical share served |
|---|---|---|---|
| Maximize total kW | 140 kW | 30 kW | 30/90 |
| Maximize weighted kW | 130 kW | 90 kW | 90/90 |

**What the experiment establishes**

Changing the service objective changes who benefits. It does not establish an advantage caused by an LLM or by the word "agentic."

**Predict:** What must stay matched to isolate an agentic workflow effect?

**Explanation:** The objective, candidate tools, physical limits, information and verification rules.

Fresh six-node experiment: Classroom_Labs L6, weighted MILP versus unweighted exact enumeration; both use the same feasible constraints.

# The objective is an explicit, inspectable service policy

$$\max \sum_{i=1}^{5} w_i d_i s_i, \qquad w_2 = w_5 = 10, \qquad w_1 = w_3 = w_4 = 1.$$

**Critical-pair choice**

$10(60 + 30) + 40 = \mathbf{940}$
weighted kW; 130 actual kW.

**Maximum-total choice**

$10(30) + 70 + 40 = \mathbf{410}$
weighted kW; 140 actual kW.

$d_i$: load demand in kW; $w_i$: dimensionless priority; $s_i \in \{0, 1\}$: pickup.

**Predict:** Does weight ten guarantee critical-first service on every possible network?

**Explanation:** No. Strict priority requires a lexicographic objective or a proven sufficiently large weight.

Classroom priority policy: standard_formulations.restoration_graph. Weighted restoration objectives: Chen et al., 2018.

# The objective is an explicit, inspectable service policy

$$\max \sum_{i=1}^{5} w_i d_i s_i, \qquad w_2 = w_5 = 10, \qquad w_1 = w_3 = w_4 = 1.$$

**Critical-pair choice**

$10(60 + 30) + 40 = \mathbf{940}$
weighted kW; 130 actual kW.

**Maximum-total choice**

$10(30) + 70 + 40 = \mathbf{410}$
weighted kW; 140 actual kW.

$d_i$: load demand in kW; $w_i$: dimensionless priority; $s_i \in \{0, 1\}$: pickup.

**Predict:** Does weight ten guarantee critical-first service on every possible network?

**Explanation:** No. Strict priority requires a lexicographic objective or a proven sufficiently large weight.

Classroom priority policy: standard_formulations.restoration_graph. Weighted restoration objectives: Chen et al., 2018.

# Planning: binary pickup and topology decisions

| Symbol | Meaning |
|---|---|
| $s_i \in \{0,1\}$ | load $i$ picked up |
| $y_i \in \{0,1\}$ | node $i$ energized |
| $z_e \in \{0,1\}$ | edge $e$ closed |
| $P_e \in \mathbb{R}$ | signed active flow, kW |
| $\bar{P}_e$ | edge capacity, kW |

$$s_i \le y_i, \qquad y_0 = 1,$$
$$z_{ij} \le y_i, \qquad z_{ij} \le y_j,$$
$$z_{1,2} = 0.$$

**Interpret each line**

Picked-up loads need energized nodes; closed edges need energized endpoints; the fault stays isolated.

**Predict:** Which of these rules prevents a disconnected energized island?

**Explanation:** None yet. Connectivity must be imposed separately.

Fresh MILP implementation: src/standard_formulations.py. Radiality and connectivity: Wang et al., 2020.

# Planning: binary pickup and topology decisions

| Symbol | Meaning |
|---|---|
| $s_i \in \{0,1\}$ | load $i$ picked up |
| $y_i \in \{0,1\}$ | node $i$ energized |
| $z_e \in \{0,1\}$ | edge $e$ closed |
| $P_e \in \mathbb{R}$ | signed active flow, kW |
| $\bar{P}_e$ | edge capacity, kW |

$$s_i \le y_i, \qquad y_0 = 1,$$
$$z_{ij} \le y_i, \qquad z_{ij} \le y_j,$$
$$z_{1,2} = 0.$$

**Interpret each line**

Picked-up loads need energized nodes; closed edges need energized endpoints; the fault stays isolated.

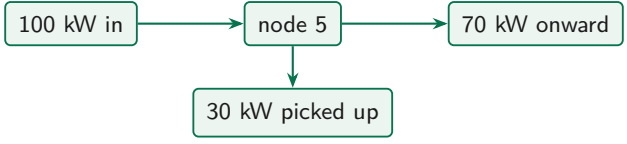
**Predict:** Which of these rules prevents a disconnected energized island?

**Explanation:** None yet. Connectivity must be imposed separately.

Fresh MILP implementation: src/standard_formulations.py. Radiality and connectivity: Wang et al., 2020.

# Power balance says where every picked-up kilowatt comes from

$$\underbrace{\sum_{e} A_{ie} P_e}_{\text{net active power entering node } i} = \underbrace{d_i s_i}_{\text{picked-up demand}}, \quad i \neq 0, \qquad -\bar{P}_e z_e \leq P_e \leq \bar{P}_e z_e.$$




$A_{ie} = +1$ for an edge entering $i$, $-1$ leaving it, zero otherwise.

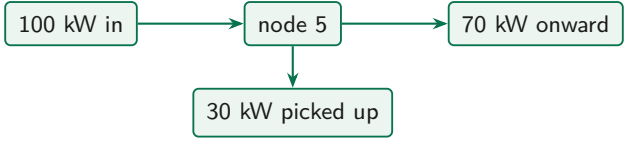
**Know the model boundary**

This is lossless active-power transport. Voltage, reactive power, AC losses and protection are not certified.

Classroom_Labs L6; active-transport formulation in src/standard_formulations.py. This six-node model is separate from the archived feeder power-flow checks.

# Power balance says where every picked-up kilowatt comes from

$$\underbrace{\sum_{e} A_{ie} P_e}_{\text{net active power entering node } i} = \underbrace{d_i s_i}_{\text{picked-up demand}}, \quad i \neq 0, \qquad -\bar{P}_e z_e \leq P_e \leq \bar{P}_e z_e.$$



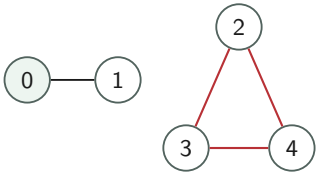

$A_{ie} = +1$ for an edge entering $i$, $-1$ leaving it, zero otherwise.

**Know the model boundary**

This is lossless active-power transport. Voltage, reactive power, AC losses and protection are not certified.

Classroom_Labs L6; active-transport formulation in src/standard_formulations.py. This six-node model is separate from the archived feeder power-flow checks.

# Edge counts alone do not prove a source-connected tree




5 nodes; 4 edges
but an unsupplied cycle

$$\sum_e A_{ie} f_e = y_i \qquad (i \neq 0),$$

$$-5z_e \le f_e \le 5z_e,$$

$$\sum_e z_e = \sum_i y_i - 1.$$

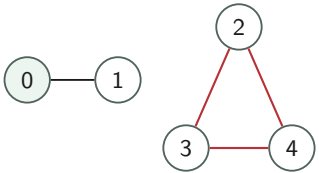
$f_e$: fictitious connectivity flow, not electrical kW. Every energized non-source node consumes one unit. Five is the maximum number of such nodes.

## Two jobs, two conditions

Commodity flow proves reachability. Reachability plus the edge count gives a tree on the energized nodes.

Original counterexample; single-commodity connectivity and radiality: Wang et al., 2020; classroom MILP.

# Edge counts alone do not prove a source-connected tree




5 nodes; 4 edges
but an unsupplied cycle

$$\sum_e A_{ie} f_e = y_i \qquad (i \neq 0),$$

$$-5z_e \leq f_e \leq 5z_e,$$

$$\sum_e z_e = \sum_i y_i - 1.$$

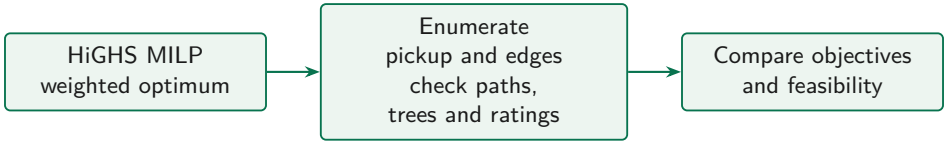
$f_e$: fictitious connectivity flow, not electrical kW. Every energized non-source node consumes one unit. Five is the maximum number of such nodes.

## Two jobs, two conditions

Commodity flow proves reachability. Reachability plus the edge count gives a tree on the energized nodes.

Original counterexample; single-commodity connectivity and radiality: Wang et al., 2020; classroom MILP.

# Verification uses an independent, exhaustive small-case check




**Two solves within the MILP method**

First maximize weighted service. Then preserve that optimum and minimize selected edge count, with a small edge-index tie-breaker.

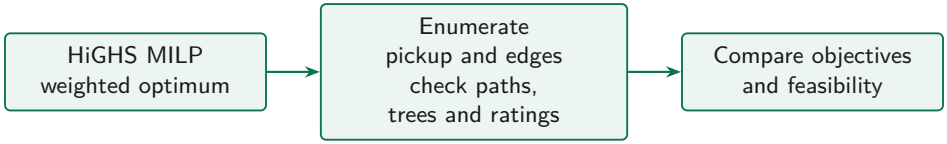
**Predict:** Does minimizing selected edges minimize switching operations?

**Explanation:** No. Operation count needs the initial switch states and a sequence-aware model.

Exhaustive enumeration is practical because this fixture has only six nodes. It is a verification aid, not a proposed large-feeder algorithm.

Classroom_Labs L6; standard_formulations.solve_restoration_milp and _enumerate_restoration.

# Verification uses an independent, exhaustive small-case check




## Two solves within the MILP method

First maximize weighted service. Then preserve that optimum and minimize selected edge count, with a small edge-index tie-breaker.

**Predict:** Does minimizing selected edges minimize switching operations?

**Explanation:** No. Operation count needs the initial switch states and a sequence-aware model. Exhaustive enumeration is practical because this fixture has only six nodes. It is a verification aid, not a proposed large-feeder algorithm.

Classroom_Labs L6; standard_formulations.solve_restoration_milp and _enumerate_restoration.

# Hands-on: vary the bottleneck, keep the policy visible

**Notebook L6** Eight minutes: run the full cell, then compare 40, 80 and 100 kW.

```
CAPACITIES_KW = (40., 60., 80., 100., 120.)
started = time.perf_counter()
restoration = lab.restoration_sweep(CAPACITIES_KW)
timings['L6 fresh MILP sweep + independent enumeration'] = time.perf_counter() - started
display(restoration['table'].drop(columns='pickup'))
```

## Before running

Predict total and critical pickup at 40 kW. Then predict whether every increase in capacity must increase service.

## Read the result

Confirm MILP/enumeration agreement. Record both service measures and the elapsed solve/check time. Explain one plateau.

Run the entire L6 cell; the displayed excerpt only locates the experiment. Samples are discrete capacities, not a continuous response curve.

Exact excerpt: Classroom_Labs.ipynb, L6 lines 1–5. Fresh results use the six-node active-transport fixture.

# Hands-on: vary the bottleneck, keep the policy visible

**Notebook L6** Eight minutes: run the full cell, then compare 40, 80 and 100 kW.

```
CAPACITIES_KW = (40., 60., 80., 100., 120.)
started = time.perf_counter()
restoration = lab.restoration_sweep(CAPACITIES_KW)
timings['L6 fresh MILP sweep + independent enumeration'] = time.perf_counter() - started
display(restoration['table'].drop(columns='pickup'))
```

## Before running

Predict total and critical pickup at 40 kW. Then predict whether every increase in capacity must increase service.

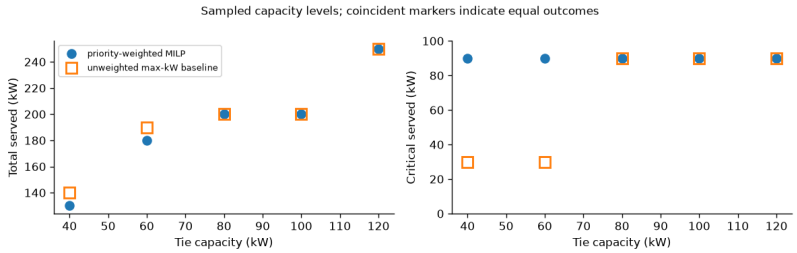
## Read the result

Confirm MILP/enumeration agreement. Record both service measures and the elapsed solve/check time. Explain one plateau.

Run the entire L6 cell; the displayed excerpt only locates the experiment. Samples are discrete capacities, not a continuous response curve.

Exact excerpt: Classroom_Labs.ipynb, L6 lines 1–5. Fresh results use the six-node active-transport fixture.

# More capacity can produce the same discrete pickup




## Two ratings, same pickup

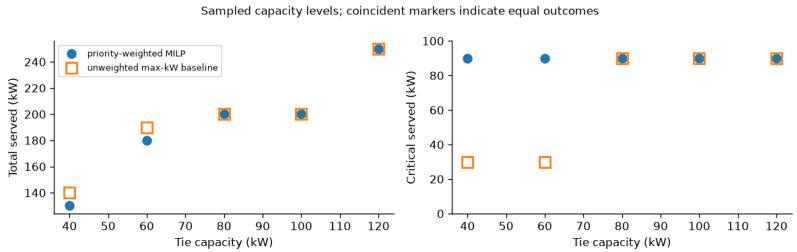
At 80 and 100 kW: weighted service is 200 kW total, including all 90 critical kW.

**Predict:** Is the flat step evidence that capacity has no value?

**Explanation:** No. It reflects indivisible load blocks and the available routes at these sampled ratings.

Original computed figure: Classroom_Labs L6 / classroom_restoration.png. Same feasible constraints; weighted versus unweighted objectives.

# More capacity can produce the same discrete pickup




## Two ratings, same pickup

At 80 and 100 kW: weighted service is 200 kW total, including all 90 critical kW.

**Predict:** Is the flat step evidence that capacity has no value?

**Explanation:** No. It reflects indivisible load blocks and the available routes at these sampled ratings.

Original computed figure: Classroom_Labs L6 / classroom_restoration.png. Same feasible constraints; weighted versus unweighted objectives.

# The workflow surrounds the solver with evidence and decisions

| Stage | Specific method | Output that can be checked |
|---|---|---|
| **Awareness** | Topology and status checks | Fault 1–2; ratings; critical nodes 2, 5 |
| **Planning** | Weighted restoration MILP | Pickup vector and selected edges |
| **Simulation** | Active-power transport | Per-edge demand and capacity margins |
| **Verification** | Independent enumeration | Feasible and objective agreement |
| **LLM Optimizer** | Allowed refinement request | Proposed next calculation, with rationale |

**Scope matters**

L6–L7 run fresh numerical and binding checks. L10 replays the full five-stage language trace from the separate archived feeder experiment.

Classroom_Labs L6–L7, L10; proposal_complete_workflow.json. This table describes stage roles; L6 does not make five live LLM calls.

# The workflow surrounds the solver with evidence and decisions

| Stage | Specific method | Output that can be checked |
|---|---|---|
| **Awareness** | Topology and status checks | Fault 1–2; ratings; critical nodes 2, 5 |
| **Planning** | Weighted restoration MILP | Pickup vector and selected edges |
| **Simulation** | Active-power transport | Per-edge demand and capacity margins |
| **Verification** | Independent enumeration | Feasible and objective agreement |
| **LLM Optimizer** | Allowed refinement request | Proposed next calculation, with rationale |

## Scope matters

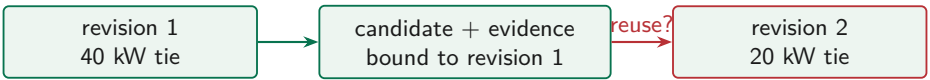
L6–L7 run fresh numerical and binding checks. L10 replays the full five-stage language trace from the separate archived feeder experiment.

Classroom_Labs L6–L7, L10; proposal_complete_workflow.json. This table describes stage roles; L6 does not make five live LLM calls.

# Hands-on: old evidence does not certify a changed rating

**Notebook L7** Four minutes: run the full cell, then explain the two Boolean results.

```
new_rest_request = {**rest_request, 'capacity_kw': 20., 'revision': 2}
print('Original contract:', lab.bound_to(rest_request, rest_candidate, rest_evidence))
print('Changed rating with old evidence:', lab.bound_to(new_rest_request, rest_candidate, rest_evidence))
print('Required response: HOLD old recommendation; request a fresh constrained solve.')
```



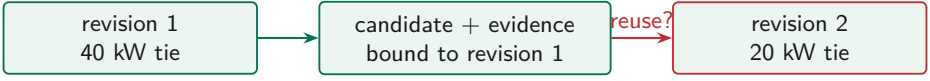

**Explanation:** Original binding: True. Changed request with old evidence: False. HOLD and re-solve.

Exact excerpt: Classroom_Labs L7 lines 7–10; lab.bind_evidence and lab.bound_to. Binding checks identity, not physical feasibility by itself.

# Hands-on: old evidence does not certify a changed rating

**Notebook L7** Four minutes: run the full cell, then explain the two Boolean results.

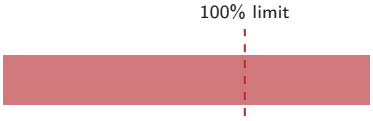
```
new_rest_request = {**rest_request, 'capacity_kw': 20., 'revision': 2}
print('Original contract:', lab.bound_to(rest_request, rest_candidate, rest_evidence))
print('Changed rating with old evidence:', lab.bound_to(new_rest_request, rest_candidate, rest_evidence))
print('Required response: HOLD old recommendation; request a fresh constrained solve.')
```




**Explanation:** Original binding: True. Changed request with old evidence: False. HOLD and re-solve.

Exact excerpt: Classroom_Labs L7 lines 7–10; lab.bind_evidence and lab.bound_to. Binding checks identity, not physical feasibility by itself.

# Archived feeder: an attractive pickup can fail a thermal check




E01: 151.64% loading

Verification rejects the initial high-pickup proposal.

**Different fixture; explicit scope**

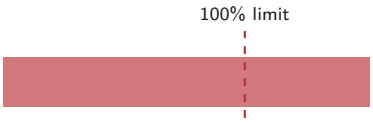
Archived MV feeder; synthetic short bypass; 100 A limit; ideal LV aggregation. No time-ordered switching certificate.

**Refine, simulate, verify again**

A permitted shedding refinement produces a passing candidate: 1843.61 kW served, including 474.04 kW critical.

ARCHIVED ACTUAL: proposal_complete_workflow.json, restoration / lindistflow3. Initial E01 loading 151.639%; final verified SHED_075.

# Archived feeder: an attractive pickup can fail a thermal check




Verification rejects the initial high-pickup proposal.

## Different fixture; explicit scope

Archived MV feeder; synthetic short bypass; 100 A limit; ideal LV aggregation. No time-ordered switching certificate.

## Refine, simulate, verify again

A permitted shedding refinement produces a passing candidate: 1843.61 kW served, including 474.04 kW critical.

ARCHIVED ACTUAL: proposal_complete_workflow.json, restoration / lindistflow3. Initial E01 loading 151.639%; final verified SHED_075.

# Llama can request a useful check and still rank results incorrectly

**Actual Planning response: 0.874 s**

*"Requesting both initial simulations as per the Planning Agent's role and the contract's operational limit."*

**Actual LLM ranking: numerical guard required**

Returned ranking: E02, E03.
Returned scores: E02 = −5509.14; E03 = −6109.97.
The task minimizes cost, so **E03 must precede E02**.

**Predict:** Should a majority of similar language responses overrule the score?

No. Recompute the ordering from verified numerical evidence.
The accepted proposal is tied to a fresh final verification.

Exact Planning quote and returned ranking: archived restoration / lindistflow3 trace, proposal_complete_workflow.json. Times are recorded model-call wall times.

# Llama can request a useful check and still rank results incorrectly

**Actual Planning response: 0.874 s**

*"Requesting both initial simulations as per the Planning Agent's role and the contract's operational limit."*

**Actual LLM ranking: numerical guard required**

Returned ranking: E02, E03.
Returned scores: E02 $= -5509.14$; E03 $= -6109.97$.
The task minimizes cost, so **E03 must precede E02**.

**Predict:** Should a majority of similar language responses overrule the score?
No. Recompute the ordering from verified numerical evidence.
The accepted proposal is tied to a fresh final verification.

Exact Planning quote and returned ranking: archived restoration / lindistflow3 trace, proposal_complete_workflow.json. Times are recorded model-call wall times.

# Llama can request a useful check and still rank results incorrectly

**Actual Planning response: 0.874 s**

*"Requesting both initial simulations as per the Planning Agent's role and the contract's operational limit."*

**Actual LLM ranking: numerical guard required**

Returned ranking: E02, E03.
Returned scores: E02 $= -5509.14$; E03 $= -6109.97$.
The task minimizes cost, so **E03 must precede E02**.

**Predict:** Should a majority of similar language responses overrule the score?
No. Recompute the ordering from verified numerical evidence.
The accepted proposal is tied to a fresh final verification.

Exact Planning quote and returned ranking: archived restoration / lindistflow3 trace, proposal_complete_workflow.json. Times are recorded model-call wall times.

# Matched supervision: equal service, different latency

| Archived LinDistFlow3 restoration | Fixed supervisor | Llama supervisor |
|---|---|---|
| Total served (kW) | 1843.61 | 1843.61 |
| Critical served (kW) | 474.04 | 474.04 |
| Objective cost | −6109.97 | −6109.97 |
| Total elapsed time (s) | 0.306 | 5.232 |

**A result worth teaching honestly**

This run shows no service improvement from Llama. Its visible contribution is a tool-selection rationale and trace; it adds latency and needs numerical guards.

**Predict:** What would demonstrate an additional agentic benefit?

**Explanation:** Matched tests of changed requests, tool failures and stale evidence, with success rate and human effort measured.

ARCHIVED ACTUAL: fixed versus agentic, restoration / lindistflow3, proposal_complete_workflow.json. Single recorded case; four Llama calls.

# Matched supervision: equal service, different latency

| Archived LinDistFlow3 restoration | Fixed supervisor | Llama supervisor |
|---|---|---|
| Total served (kW) | 1843.61 | 1843.61 |
| Critical served (kW) | 474.04 | 474.04 |
| Objective cost | −6109.97 | −6109.97 |
| Total elapsed time (s) | 0.306 | 5.232 |

## A result worth teaching honestly

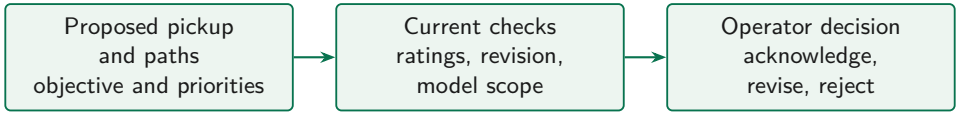
This run shows no service improvement from Llama. Its visible contribution is a tool-selection rationale and trace; it adds latency and needs numerical guards.

**Predict:** What would demonstrate an additional agentic benefit?

**Explanation:** Matched tests of changed requests, tool failures and stale evidence, with success rate and human effort measured.

ARCHIVED ACTUAL: fixed versus agentic, restoration / lindistflow3, proposal_complete_workflow.json. Single recorded case; four Llama calls.

# Handoff: show the action, the evidence and the remaining decision




## A complete recommendation is conditional

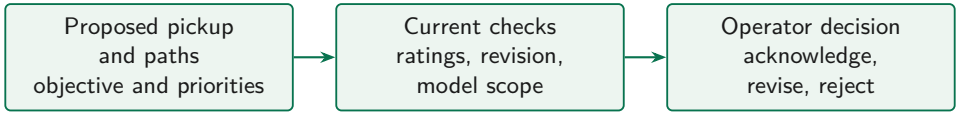
State which measurements and equipment statuses were trusted. A passing calculation is tied to that information and that model.

**Predict:** What if the network model is correct but the measurements are deceptive?

**Explanation:** The planner may optimize the wrong state. The next case tests consistency, observability and trusted replacement evidence.

Restoration lessons: Classroom_Labs L6–L7 and archived L10 trace. Operator acknowledgment is a tutorial workflow, not a field execution system.

# Handoff: show the action, the evidence and the remaining decision




## A complete recommendation is conditional

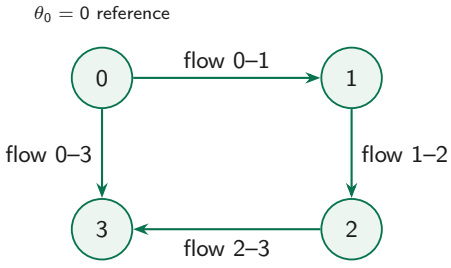
State which measurements and equipment statuses were trusted. A passing calculation is tied to that information and that model.

**Predict:** What if the network model is correct but the measurements are deceptive?

**Explanation:** The planner may optimize the wrong state. The next case tests consistency, observability and trusted replacement evidence.

Restoration lessons: Classroom_Labs L6–L7 and archived L10 trace. Operator acknowledgment is a tutorial workflow, not a field execution system.

# Sensors report numbers; the state is what we infer




**Fresh four-bus DC fixture**

Eight measurements: four flows, three injections, one angle. Three unknown angles. Redundancy permits a consistency test.

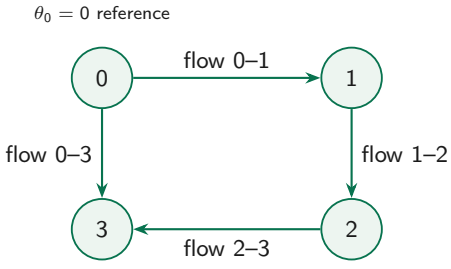
**Predict:** Is a small residual the same as a trustworthy state?

**Explanation:** No. We will construct two different states with the same residual.

This identifiable DC example is separate from the three-phase distribution-feeder models.

Classroom_Labs L8–L9; standard_formulations.run_wls_demo. State estimation: Schweppe and Wildes, 1970.

# Sensors report numbers; the state is what we infer




**Fresh four-bus DC fixture**

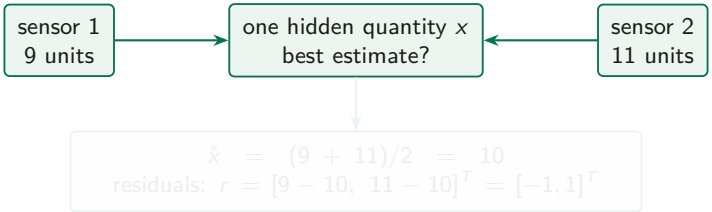
Eight measurements: four flows, three injections, one angle. Three unknown angles. Redundancy permits a consistency test.

**Predict:** Is a small residual the same as a trustworthy state?

**Explanation:** No. We will construct two different states with the same residual.
This identifiable DC example is separate from the three-phase distribution-feeder models.

Classroom_Labs L8–L9; standard_formulations.run_wls_demo. State estimation: Schweppe and Wildes, 1970.

# Two equally precise sensors: find their common explanation



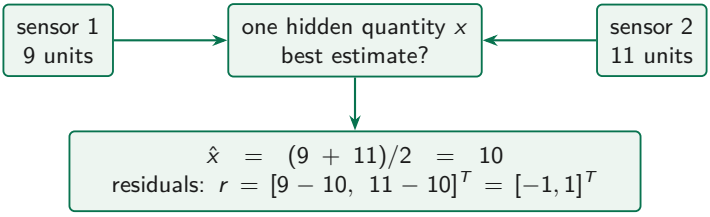

**Predict:** What mismatch remains after choosing the best common value?

**Explanation:** $J = (-1)^2 + 1^2 = 2$. A residual measures mismatch to the fitted model.

Illustrative scalar example: $H = [1, 1]^T$, unit variance for each sensor. This is WLS with equal weights, not the four-bus run.

Original arithmetic teaching example; least-squares principle: Schweppe and Wildes, 1970.

# Two equally precise sensors: find their common explanation




**Predict:** What mismatch remains after choosing the best common value?

**Explanation:** $J = (-1)^2 + 1^2 = 2$. A residual measures mismatch to the fitted model. Illustrative scalar example: $H = [1, 1]^T$, unit variance for each sensor. This is WLS with equal weights, not the four-bus run.

Original arithmetic teaching example; least-squares principle: Schweppe and Wildes, 1970.

# An isolated error increases disagreement between sensors

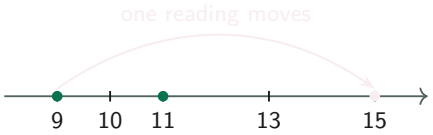
## Before corruption

$z = [9, 11]^T$
$\hat{x} = 10$
$r = [-1, 1]^T$
$J = 2.$

## Change only sensor 1

$z' = [15, 11]^T$
Which common estimate fits best?

$\hat{x}' = 13, \quad r' = [2, -2]^T, \quad J' = 8.$




A calibrated alarm signals model inconsistency; it does not identify an attacker.

Original scalar illustration, equal unit variances. Statistical alarm assumptions are introduced on C06.

# An isolated error increases disagreement between sensors

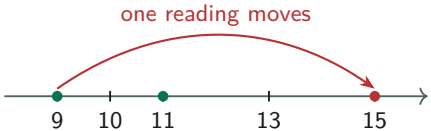
## Before corruption

$z = [9, 11]^T$
$\hat{x} = 10$
$r = [-1, 1]^T$
$J = 2.$

## Change only sensor 1

$z' = [15, 11]^T$
Which common estimate fits best?

$$\hat{x}' = 13, \quad r' = [2, -2]^T, \quad J' = 8.$$




A calibrated alarm signals model inconsistency; it does not identify an attacker.

Original scalar illustration, equal unit variances. Statistical alarm assumptions are introduced on C06.

# Coordinated errors can preserve the same disagreement

| | Measurements | Fitted state | Residual |
|---|---|---|---|
| Original | $[9, 11]^T$ | 10 | $[-1, 1]^T$ |
| Both shifted by $+3$ | $[12, 14]^T$ | 13 | $[-1, 1]^T$ |

Same residual statistic: $J' = J = 2$.

Different inferred state: $\hat{x}' = \hat{x} + 3$.

**Consistency is not authenticity**

Both readings fit a shifted state. Consistency alone cannot recover the missing truth.

**Predict:** Does LLM voting add an independent measurement?

**Explanation:** No. New physical evidence must come from outside the corrupted readings.

Original scalar adaptation of the false-data-injection principle in Liu, Ning and Reiter, CCS 2009.

# Coordinated errors can preserve the same disagreement

| | Measurements | Fitted state | Residual |
|---|---|---|---|
| Original | $[9, 11]^T$ | 10 | $[-1, 1]^T$ |
| Both shifted by $+3$ | $[12, 14]^T$ | 13 | $[-1, 1]^T$ |

Same residual statistic: $J' = J = 2$.

Different inferred state: $\hat{x}' = \hat{x} + 3$.

**Consistency is not authenticity**

Both readings fit a shifted state. Consistency alone cannot recover the missing truth.

**Predict:** Does LLM voting add an independent measurement?

**Explanation:** No. New physical evidence must come from outside the corrupted readings.

Original scalar adaptation of the false-data-injection principle in Liu, Ning and Reiter, CCS 2009.

# Weighted least squares fits the measurement model

$$z = Hx + e, \qquad \hat{x} = \arg\min_x (z - Hx)^T R^{-1} (z - Hx), \qquad R = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_m^2).$$

| | |
|---|---|
| $z \in \mathbb{R}^m$ | sensor measurements |
| $x \in \mathbb{R}^n$ | unknown state |
| $H \in \mathbb{R}^{m \times n}$ | linear measurement map |
| $e$ | measurement noise |
| $\sigma_j$ | channel $j$ noise scale |
| $R$ | noise covariance |

**Our fresh network run**

$m = 8$, $n = 3$, $\mathrm{rank}(H) = 3$.
Flow $\sigma = 0.004$ p.u.; injection 0.006 p.u.; angle 0.0005 rad.

Higher precision means larger weight. Each discrepancy is normalized by its own noise scale.

Classroom_Labs L8; standard_formulations.weighted_least_squares. WLS: Schweppe and Wildes, 1970.

# Weighted least squares fits the measurement model

$$z = Hx + e, \qquad \hat{x} = \arg\min_x (z - Hx)^T R^{-1} (z - Hx), \qquad R = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_m^2).$$

| | |
|---|---|
| $z \in \mathbb{R}^m$ | sensor measurements |
| $x \in \mathbb{R}^n$ | unknown state |
| $H \in \mathbb{R}^{m \times n}$ | linear measurement map |
| $e$ | measurement noise |
| $\sigma_j$ | channel $j$ noise scale |
| $R$ | noise covariance |

**Our fresh network run**

$m = 8$, $n = 3$, $\mathrm{rank}(H) = 3$.
Flow $\sigma = 0.004$ p.u.; injection 0.006 p.u.; angle 0.0005 rad.

Higher precision means larger weight. Each discrepancy is normalized by its own noise scale.

Classroom_Labs L8; standard_formulations.weighted_least_squares. WLS: Schweppe and Wildes, 1970.

# A residual alarm is a statistical test under stated assumptions

$$r = z - H\hat{x}, \qquad J = r^T R^{-1} r, \qquad J > \chi^2_{m-n,\,1-\alpha} \implies \text{alarm.}$$

## Classroom calibration

$m - n = 8 - 3 = 5$ residual degrees of freedom.
$\alpha = 0.01$; threshold $\chi^2_{5,0.99} = 15.0863$.

## Interpretation requires assumptions

Correct linear model; full column rank; stated independent Gaussian noise and covariance. $\alpha$ is the nominal false-alarm probability.

**Predict:** Does "no alarm" mean "safe to control"?

**Explanation:** No. It means this test found no excessive model inconsistency. Provenance and operating limits remain separate checks.

Residual test implementation: standard_formulations.weighted_least_squares. State estimation foundations: Schweppe and Wildes, 1970.

# A residual alarm is a statistical test under stated assumptions

$$r = z - H\hat{x}, \qquad J = r^T R^{-1} r, \qquad J > \chi^2_{m-n,\,1-\alpha} \implies \text{alarm.}$$

**Classroom calibration**

$m - n = 8 - 3 = 5$ residual degrees of freedom. $\alpha = 0.01$; threshold $\chi^2_{5,0.99} = 15.0863$.

**Interpretation requires assumptions**

Correct linear model; full column rank; stated independent Gaussian noise and covariance. $\alpha$ is the nominal false-alarm probability.

**Predict:** Does "no alarm" mean "safe to control"?

**Explanation:** No. It means this test found no excessive model inconsistency. Provenance and operating limits remain separate checks.

Residual test implementation: standard_formulations.weighted_least_squares. State estimation foundations: Schweppe and Wildes, 1970.

# The cancellation is visible in three lines

Coordinated measurement change: $z' = z + Hc,$

New WLS estimate: $\hat{x}' = \hat{x} + c,$

New residual: $r' = z + Hc - H(\hat{x} + c) = z - H\hat{x} = r.$

$c \in \mathbb{R}^n$: chosen state shift; $a = Hc$: coordinated change to sensor readings. Therefore $J' = J$ with unchanged covariance.

**The assumptions are part of the result**

Full-rank linear WLS; correct knowledge of $H$; ability to alter the required unprotected channels. A protected channel can restrict feasible shifts.

**Predict:** Which term disappears when we subtract the predicted measurements?

The added $Hc$ cancels exactly.

False-data-injection construction: Liu, Ning and Reiter, CCS 2009. Defensive four-bus exercise in L8.

# The cancellation is visible in three lines

$$\text{Coordinated measurement change:} \quad z' = z + Hc,$$

$$\text{New WLS estimate:} \quad \hat{x}' = \hat{x} + c,$$

$$\text{New residual:} \quad r' = z + Hc - H(\hat{x} + c) = z - H\hat{x} = r.$$

$c \in \mathbb{R}^n$: chosen state shift; $a = Hc$: coordinated change to sensor readings. Therefore $J' = J$ with unchanged covariance.

**The assumptions are part of the result**

Full-rank linear WLS; correct knowledge of $H$; ability to alter the required unprotected channels. A protected channel can restrict feasible shifts.

**Predict:** Which term disappears when we subtract the predicted measurements?

The added $Hc$ cancels exactly.

False-data-injection construction: Liu, Ning and Reiter, CCS 2009. Defensive four-bus exercise in L8.

# The cancellation is visible in three lines

$$\text{Coordinated measurement change:} \quad z' = z + Hc,$$

$$\text{New WLS estimate:} \quad \hat{x}' = \hat{x} + c,$$

$$\text{New residual:} \quad r' = z + Hc - H(\hat{x} + c) = z - H\hat{x} = r.$$

$c \in \mathbb{R}^n$: chosen state shift; $a = Hc$: coordinated change to sensor readings. Therefore $J' = J$ with unchanged covariance.

**The assumptions are part of the result**

Full-rank linear WLS; correct knowledge of $H$; ability to alter the required unprotected channels. A protected channel can restrict feasible shifts.

**Predict:** Which term disappears when we subtract the predicted measurements?
The added $Hc$ cancels exactly.

False-data-injection construction: Liu, Ning and Reiter, CCS 2009. Defensive four-bus exercise in L8.

# Hands-on: sweep isolated and coordinated corruption

**Notebook L8** Seven minutes: run the full cell; compare residual and state-error plots.

```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
timings['L8 fresh WLS attack sweep'] = time.perf_counter() - started
display(cyber['table'])
```

## Predict first

For a growing isolated bias, what happens to $J$? For a growing $Hc$ shift, what happens to $J$ and the state error?

## Read two different quantities

The residual is observable from measurements and the model. State error uses simulated truth and is an offline evaluation metric.

The scale multiplies a synthetic perturbation. It is not an estimate of attack likelihood or real-world severity.

Exact excerpt: Classroom_Labs L8 lines 1–5. Fixed synthetic seeds and attack directions in src/classroom_labs.py and standard_formulations.py.

# Hands-on: sweep isolated and coordinated corruption

**Notebook L8** Seven minutes: run the full cell; compare residual and state-error plots.

```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
timings['L8 fresh WLS attack sweep'] = time.perf_counter() - started
display(cyber['table'])
```

## Predict first

For a growing isolated bias, what happens to $J$? For a growing $Hc$ shift, what happens to $J$ and the state error?

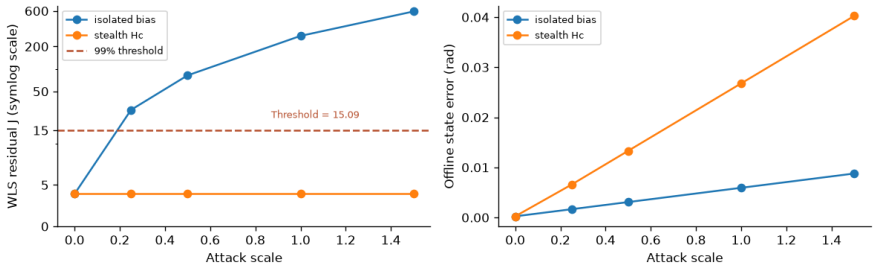
## Read two different quantities

The residual is observable from measurements and the model. State error uses simulated truth and is an offline evaluation metric.

The scale multiplies a synthetic perturbation. It is not an estimate of attack likelihood or real-world severity.

Exact excerpt: Classroom_Labs L8 lines 1–5. Fixed synthetic seeds and attack directions in src/classroom_labs.py and standard_formulations.py.

# Same residual does not imply the same state



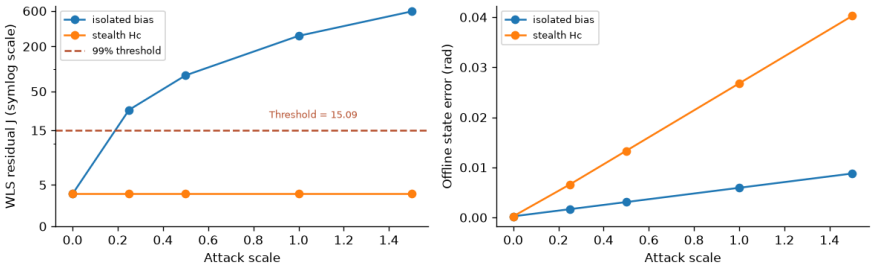

At scale 1: isolated-corruption $J \approx 277.50$; coordinated-shift $J \approx 3.92$. Threshold 15.09.

**Keep the metrics separate**

Left: consistency evidence available to the estimator. Right: state error against simulated truth, used only to evaluate the experiment.

Original computed figure: Classroom_Labs L8 / classroom_cyber.png. Coordinated $J \approx 3.92022$ remains constant across the sampled scales.

# Same residual does not imply the same state



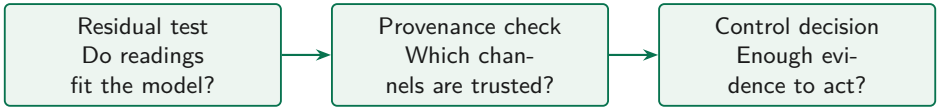

At scale 1: isolated-corruption $J \approx 277.50$; coordinated-shift $J \approx 3.92$. Threshold 15.09.

## Keep the metrics separate

Left: consistency evidence available to the estimator. Right: state error against simulated truth, used only to evaluate the experiment.

Original computed figure: Classroom_Labs L8 / classroom_cyber.png. Coordinated $J \approx 3.92022$ remains constant across the sampled scales.

# Awareness must distinguish consistency from trustworthy provenance




**A legitimate output is HOLD**

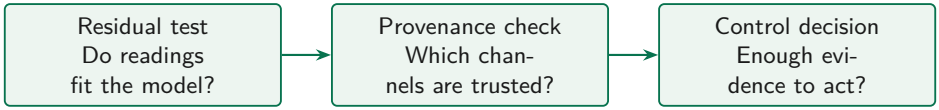
If measurements are suspect and no trustworthy replacement is available, request evidence instead of manufacturing a confident state estimate.

**Predict:** Can an LLM vote resolve missing observability or missing trusted channels?

**Explanation:** No. Voting over the same evidence does not add an independent physical constraint.

Information limitation illustrated by Liu et al., 2009; explicit protected-channel assumption in Classroom_Labs L9.

# Awareness must distinguish consistency from trustworthy provenance




**A legitimate output is HOLD**

If measurements are suspect and no trustworthy replacement is available, request evidence instead of manufacturing a confident state estimate.

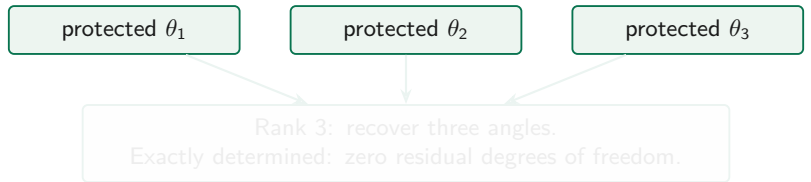
**Predict:** Can an LLM vote resolve missing observability or missing trusted channels?

**Explanation:** No. Voting over the same evidence does not add an independent physical constraint.

Information limitation illustrated by Liu et al., 2009; explicit protected-channel assumption in Classroom_Labs L9.

# Recovery assumes three independent protected angle readings

$$z_T = H_T x + e_T, \qquad H_T = I_3, \qquad \sigma_T = 0.00005 \text{ rad per channel.}$$




Subscript $T$: trusted set; $I_3$: identity matrix; $e_T$: assumed measurement noise.

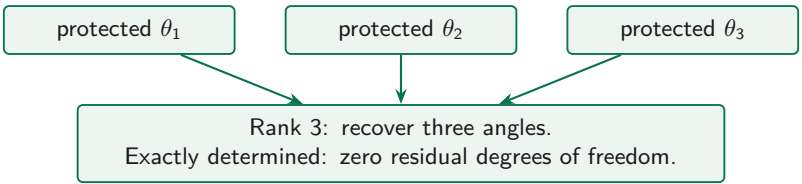
**An assumption, not an implemented security protocol**

Authentication and physical protection are external requirements. This three-channel fit provides no redundant chi-square detection test.

Exact classroom recovery model: standard_formulations.run_wls_demo; Classroom_Labs L9. Protected measurements are assumed trustworthy.

# Recovery assumes three independent protected angle readings

$$z_T = H_T x + e_T, \qquad H_T = I_3, \qquad \sigma_T = 0.00005 \text{ rad per channel.}$$



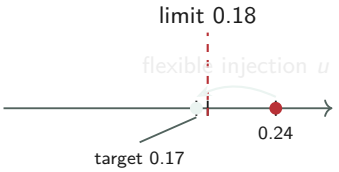

Subscript $T$: trusted set; $I_3$: identity matrix; $e_T$: assumed measurement noise.

**An assumption, not an implemented security protocol**

Authentication and physical protection are external requirements. This three-channel fit provides no redundant chi-square detection test.

Exact classroom recovery model: standard_formulations.run_wls_demo; Classroom_Labs L9. Protected measurements are assumed trustworthy.

# Corrective control: move a stressed flow back inside its limit




**Noiseless arithmetic illustration**

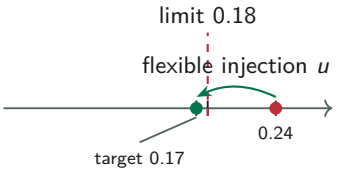
For branch 1–2, suppose estimated flow is 0.24 p.u. and sensitivity is $-4/7$.
A 0.01 p.u. margin gives target 0.17.

$u \geq (0.24 - 0.17)/(4/7) = 0.1225$ p.u.

$u$: injection at node 2, balanced by the reference source. The actual noisy protected-state run gives $u \approx 0.12169$ p.u.; the illustration above is not that exact solve.

Sensitivity $FB^{-1}[0, 1, 0]^T$ from standard_formulations._redispatch. Original noiseless arithmetic illustration; all quantities here are per unit.

# Corrective control: move a stressed flow back inside its limit




**Noiseless arithmetic illustration**

For branch 1–2, suppose estimated flow is 0.24 p.u. and sensitivity is $-4/7$.
A 0.01 p.u. margin gives target 0.17.

$u \geq (0.24 - 0.17)/(4/7) = 0.1225$ p.u.

$u$: injection at node 2, balanced by the reference source. The actual noisy protected-state run gives $u \approx 0.12169$ p.u.; the illustration above is not that exact solve.

Sensitivity $FB^{-1}[0, 1, 0]^T$ from standard_formulations._redispatch. Original noiseless arithmetic illustration; all quantities here are per unit.

# Planning uses a constrained quadratic control problem

$$\min_{0\le u\le 0.5} \ \tfrac{1}{2}u^2 \quad \text{s.t.} \quad |F\hat{x}_T + b_u u| \le \ell - \delta\mathbf{1}, \qquad b_u = FB^{-1}e_2.$$

$F$: angle-to-flow map; $B$: non-reference nodal injection matrix; $e_2 = [0,1,0]^T$; $\hat{x}_T$: trusted estimate; $b_u$: flow sensitivity.
$\ell = [0.35, 0.18, 0.35, 0.35]^T$ p.u.: branch limits; $\delta = 0.01$ p.u.: planning margin; $\mathbf{1}$: vector of ones. Absolute values and inequalities are componentwise.

**Notebook L9** Run both evidence conditions with the same numerical control method.

```
without_trust = lab.trusted_response(cyber, protected_available=False)
with_trust = lab.trusted_response(cyber, protected_available=True)
```

## Method and acceptance

SLSQP solves this scalar convex QP. Independent post-solve bounds and predicted-flow checks can still return HOLD.

Exact excerpt: Classroom_Labs L9 lines 1–2; constrained model and checks in standard_formulations._redispatch. DC illustration, not AC feeder resilience.

# Planning uses a constrained quadratic control problem

$$\min_{0\le u\le 0.5} \ \tfrac{1}{2}u^2 \quad \text{s.t.} \quad |F\hat{x}_T + b_u u| \le \ell - \delta\mathbf{1}, \qquad b_u = FB^{-1}e_2.$$

$F$: angle-to-flow map; $B$: non-reference nodal injection matrix; $e_2 = [0,1,0]^T$; $\hat{x}_T$: trusted estimate; $b_u$: flow sensitivity.
$\ell = [0.35, 0.18, 0.35, 0.35]^T$ p.u.: branch limits; $\delta = 0.01$ p.u.: planning margin; $\mathbf{1}$: vector of ones. Absolute values and inequalities are componentwise.

**Notebook L9** Run both evidence conditions with the same numerical control method.

```
without_trust = lab.trusted_response(cyber, protected_available=False)
with_trust = lab.trusted_response(cyber, protected_available=True)
```

## Method and acceptance

SLSQP solves this scalar convex QP. Independent post-solve bounds and predicted-flow checks can still return HOLD.

Exact excerpt: Classroom_Labs L9 lines 1–2; constrained model and checks in standard_formulations._redispatch. DC illustration, not AC feeder resilience.

# Hands-on: evidence availability changes the permitted response

**Notebook L9** Five minutes: run the full cell; identify the acceptance assumption.

| Condition | Response | Flexible injection |
|---|---|---|
| No trusted replacement channels | HOLD | none proposed |
| Three protected angles assumed | PROPOSE | 0.12169 p.u. |

**Two checks, with different information**

Predicted branch 1–2 flow: 0.17000 p.u. (operational model check).
Actual simulated flow: $0.17046 < 0.18$ p.u. (offline truth audit).

**Predict:** Why is the offline value not an input to the operator's verification rule?

**Explanation:** The real system does not reveal its exact state just because our simulator knows it.

Fresh L9 output with fixed synthetic seeds; standard_formulations.run_wls_demo. The zero-violation offline audit applies to this modeled case only.

# Hands-on: evidence availability changes the permitted response

**Notebook L9** Five minutes: run the full cell; identify the acceptance assumption.

| Condition | Response | Flexible injection |
|---|---|---|
| No trusted replacement channels | HOLD | none proposed |
| Three protected angles assumed | PROPOSE | 0.12169 p.u. |

**Two checks, with different information**

Predicted branch 1–2 flow: 0.17000 p.u. (operational model check).
Actual simulated flow: $0.17046 < 0.18$ p.u. (offline truth audit).

**Predict:** Why is the offline value not an input to the operator's verification rule?

**Explanation:** The real system does not reveal its exact state just because our simulator knows it.

Fresh L9 output with fixed synthetic seeds; standard_formulations.run_wls_demo. The zero-violation offline audit applies to this modeled case only.

# Every stage has a specific numerical or evidentiary job

| Stage | Specific method | Inspectable output |
|---|---|---|
| **Awareness** | WLS, residual and trust checks | Alarm, rank, provenance status |
| **Planning** | Constrained DC redispatch QP | Injection and predicted flow margins |
| **Simulation** | DC flow calculation | Response to the candidate injection |
| **Verification** | Provenance and limit gates | PROPOSE or HOLD, with failed condition |
| **LLM Optimizer** | Allowed refinement selection | New tool request and returned rationale |

**Use the method suited to the decision**

WLS estimates the state; the QP selects the correction. L10 replays the archived five-stage language trace.

Classroom_Labs L8–L10; archived cyber_stealth / dc run in proposal_complete_workflow.json. Fresh numerical fixtures and archived LLM calls are separately labeled.

# Every stage has a specific numerical or evidentiary job

| Stage | Specific method | Inspectable output |
|---|---|---|
| **Awareness** | WLS, residual and trust checks | Alarm, rank, provenance status |
| **Planning** | Constrained DC redispatch QP | Injection and predicted flow margins |
| **Simulation** | DC flow calculation | Response to the candidate injection |
| **Verification** | Provenance and limit gates | PROPOSE or HOLD, with failed condition |
| **LLM Optimizer** | Allowed refinement selection | New tool request and returned rationale |

**Use the method suited to the decision**

WLS estimates the state; the QP selects the correction. L10 replays the archived five-stage language trace.

Classroom_Labs L8–L10; archived cyber_stealth / dc run in proposal_complete_workflow.json. Fresh numerical fixtures and archived LLM calls are separately labeled.

# Actual Llama response: request a new checked candidate

**Planning, 0.829 s**

*"Requesting new candidates based on the given tools and objective."*

**LLM Optimizer selects TRUSTED_QP, 1.195 s**

*"The current lowest-cost candidate E03 has a cost of 0.008671610999497877, but the permitted refinement TRUSTED_QP offers a new approach to minimize the objective. Selecting this operation will request a new simulation and verification."*

**Numerical result:** $u = 0.12169$ p.u., cost 0.00740467.
Acceptance still depends on trusted-input and flow-limit gates.

Exact returned rationale fields: archived cyber_stealth / dc trace, proposal_complete_workflow.json. Model-call wall times; rounded only outside quotes.

# Actual Llama response: request a new checked candidate

## Planning, 0.829 s

*"Requesting new candidates based on the given tools and objective."*

## LLM Optimizer selects TRUSTED_QP, 1.195 s

*"The current lowest-cost candidate E03 has a cost of 0.008671610999497877, but the permitted refinement TRUSTED_QP offers a new approach to minimize the objective. Selecting this operation will request a new simulation and verification."*

**Numerical result:** $u = 0.12169$ p.u., cost 0.00740467.
Acceptance still depends on trusted-input and flow-limit gates.

Exact returned rationale fields: archived cyber_stealth / dc trace, proposal_complete_workflow.json. Model-call wall times; rounded only outside quotes.

# Actual Llama response: request a new checked candidate

## Planning, 0.829 s

*"Requesting new candidates based on the given tools and objective."*

## LLM Optimizer selects TRUSTED_QP, 1.195 s

*"The current lowest-cost candidate E03 has a cost of 0.008671610999497877, but the permitted refinement TRUSTED_QP offers a new approach to minimize the objective. Selecting this operation will request a new simulation and verification."*

**Numerical result:** $u = 0.12169$ p.u., cost 0.00740467.
Acceptance still depends on trusted-input and flow-limit gates.

Exact returned rationale fields: archived cyber_stealth / dc trace, proposal_complete_workflow.json. Model-call wall times; rounded only outside quotes.

# A matched baseline avoids crediting the LLM for better information

| Archived DC cyber response | Fixed supervisor | Llama supervisor |
|---|---|---|
| Flexible injection (p.u.) | 0.12169 | 0.12169 |
| Objective $\frac{1}{2}u^2$ | 0.00740467 | 0.00740467 |
| Total elapsed time (s) | 0.00381 | 5.203 |

**What actually enabled recovery?**

The trusted measurement assumption plus constrained numerical control. Llama added tool-selection language and a trace; it did not improve this recorded optimum.

**Predict:** Would comparing untrusted baseline data with trusted agent data be fair?

**Explanation:** No. That would mix the value of additional information with the value of orchestration.

ARCHIVED ACTUAL: fixed versus agentic cyber_stealth / dc, proposal_complete_workflow.json. Same objective and evidence assumptions; one recorded run, four Llama calls.

# A matched baseline avoids crediting the LLM for better information

| Archived DC cyber response | Fixed supervisor | Llama supervisor |
|---|---|---|
| Flexible injection (p.u.) | 0.12169 | 0.12169 |
| Objective $\frac{1}{2}u^2$ | 0.00740467 | 0.00740467 |
| Total elapsed time (s) | 0.00381 | 5.203 |

**What actually enabled recovery?**

The trusted measurement assumption plus constrained numerical control. Llama added tool-selection language and a trace; it did not improve this recorded optimum.

**Predict:** Would comparing untrusted baseline data with trusted agent data be fair?

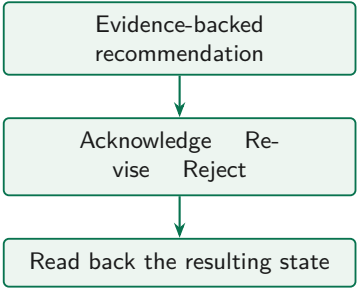
**Explanation:** No. That would mix the value of additional information with the value of orchestration.

ARCHIVED ACTUAL: fixed versus agentic cyber_stealth / dc, proposal_complete_workflow.json. Same objective and evidence assumptions; one recorded run, four Llama calls.

# The control-room handoff is a decision packet, not a green badge

## What the operator sees

Proposed action and objective.
Sensor trust and model scope.
Current revision and failed/passed checks.
Measured latency and available alternatives.




**Predict:** What did the three cases teach about choosing numerical methods?

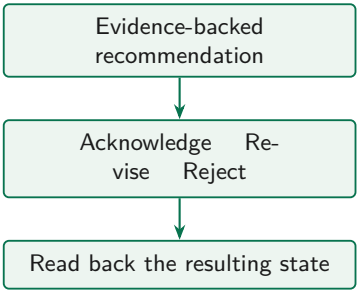
**Explanation:** Continuous voltage control, discrete restoration and evidence-limited cyber response need different tools within one auditable workflow.

Case synthesis: Classroom_Labs L1–L10; operator readback exercise L12. The tutorial dashboard is an educational interface, not a deployed control system.

# The control-room handoff is a decision packet, not a green badge

## What the operator sees

Proposed action and objective.
Sensor trust and model scope.
Current revision and failed/passed checks.
Measured latency and available alternatives.




**Predict:** What did the three cases teach about choosing numerical methods?

**Explanation:** Continuous voltage control, discrete restoration and evidence-limited cyber response need different tools within one auditable workflow.

Case synthesis: Classroom_Labs L1–L10; operator readback exercise L12. The tutorial dashboard is an educational interface, not a deployed control system.

# What did the framework actually improve?

**Numerical question**
Did the action satisfy the stated electrical limits?

**Workflow question**
Was the right study run and attached to this action?

**Operator question**
Can a reviewer identify the action and its remaining uncertainty?

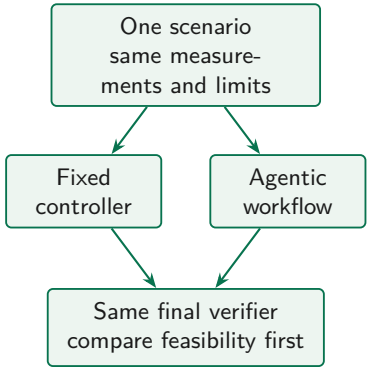
**Measure the workflow and the electrical outcome separately.**

- Study completed?
- Constraints passed?
- Claims supported?

**Predict:** Which claim is supported by a low cost and a fluent explanation?

PowerAgentBench-SS (2026), §§II–III; Executed tutorial: Classroom_Labs, L0–L12

# Compare methods on the same task




**Give every method the same physical constraints and verification budget.**

For case $i$, record whether every constraint passes.

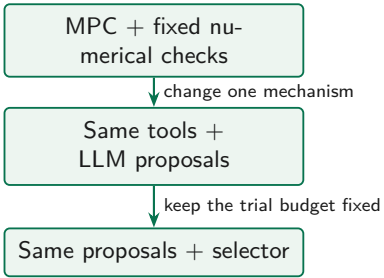
$$\widehat{p} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\text{all checks pass}]$$

$N$ is the number of cases. If 8 of 10 pass, $\widehat{p} = 0.8$. This arithmetic example is illustrative.

- Same case and admissible actions
- Same candidate trial budget
- Same final verifier

PowerAgentBench-SS (2026), §§II–III; Recorded voltage pilot: E1–E8

# Ablate one mechanism at a time

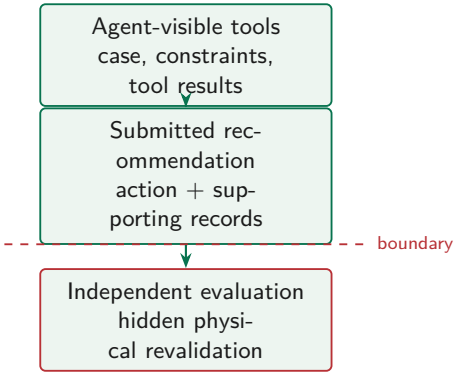



**A useful ablation keeps the numerical tools fixed.**

- Fixed numerical workflow
- Add language interpretation
- Add role-specific handoffs

PowerAgentBench-SS (2026), §§II–III; Executed LangGraph teaching lab: G0–G5; Recorded voltage pilot: E1–E8

# Frontier: evidence-backed evaluation

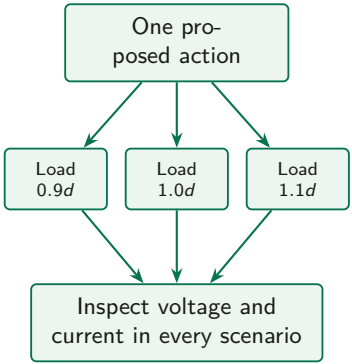



**The final report should survive independent recomputation.**

- ✓Public tools and fixed budgets
- ✓Hidden physical revalidation
- ✓Evidence-supported submissions

PowerAgentBench-SS (2026), §§II–III

# Frontier: request uncertainty studies



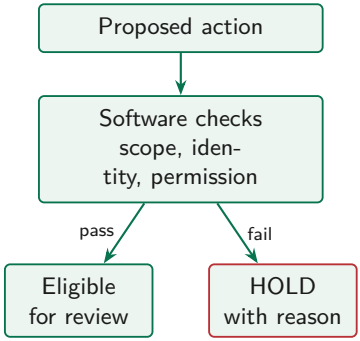

**The orchestrator can request a stronger study when uncertainty matters.**

Illustrative sensitivity study: use base demand $d$ and two nearby load levels. Keep the same action in all three trials.

- ✓Numerical engines remain explicit
- ✓Uncertainty-aware study selection
- ✓Corrective-dispatch tools

CONDUCTOR (2026), uncertainty-aware orchestration; Executed tutorial: Classroom_Labs, L0–L12

# Frontier: enforce authority in software




**Approval rules belong in the runtime, not only in the prompt.**

- ✓Whitelisted tools
- ✓Explicit step budgets
- ✓Typed numerical facts

# L11: one optional local Llama call

**Notebook L11** Exact notebook excerpt; run the complete cell in order.

```
client = ProposalLlama(journal, model='llama3.1:8b')
live_reply = client.ask('Optimization Agent', pool['context']['public_context'],
                        list(pool['context']['catalog']))
```

**Actual saved public response from Llama 3.1:8b**

"has the potential to reduce reactive effort magnitude by ten percent, which may improve the cost."

**Interpretation:** this proposes an allowed trial. The new numerical check decides whether the change is feasible and useful.

Enable `RUN_LOCAL_LLAMA=True` for a new call. The saved response remains available if the local service is slow.

Ollama chat API; Classroom L11 and saved actual validation

# L11: one optional local Llama call

**Notebook L11** Exact notebook excerpt; run the complete cell in order.

```
client = ProposalLlama(journal, model='llama3.1:8b')
live_reply = client.ask('Optimization Agent', pool['context']['public_context'],
                        list(pool['context']['catalog']))
```

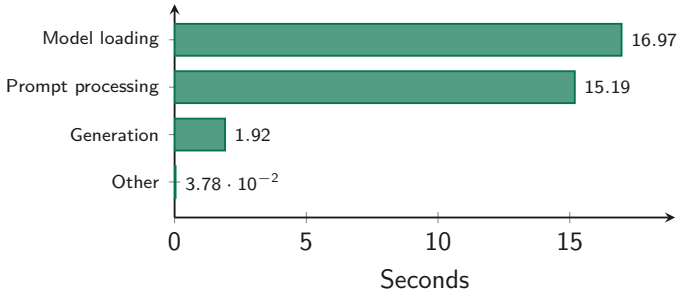
## Actual saved public response from Llama 3.1:8b

"has the potential to reduce reactive effort magnitude by ten percent, which may improve the cost."

**Interpretation:** this proposes an allowed trial. The new numerical check decides whether the change is feasible and useful.

Enable `RUN_LOCAL_LLAMA=True` for a new call. The saved response remains available if the local service is slow.

# Where did one Llama call spend its time?




**Actual local Llama call**
Client elapsed time: **34.113 s**.

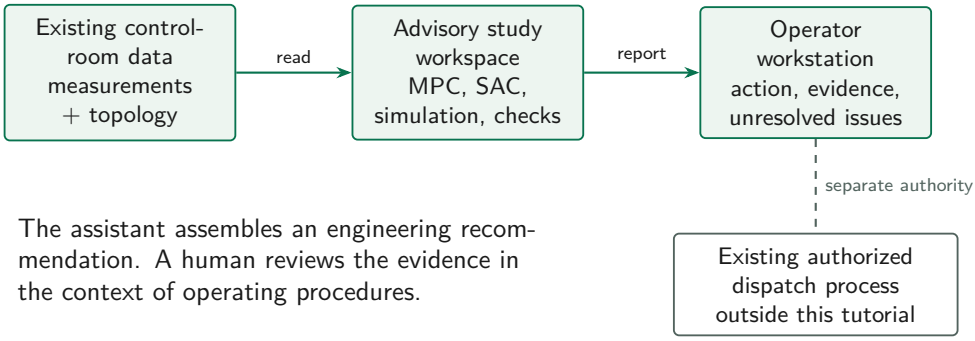
1,891 prompt tokens; 137 generated tokens.

Most time came before token generation. A warm repeat can behave differently.

**Do not call a replay “inference latency”.**

Ollama chat API: timing fields; Executed tutorial: Classroom_Labs, L0–L12

# How the workflow could fit beside control-room tools

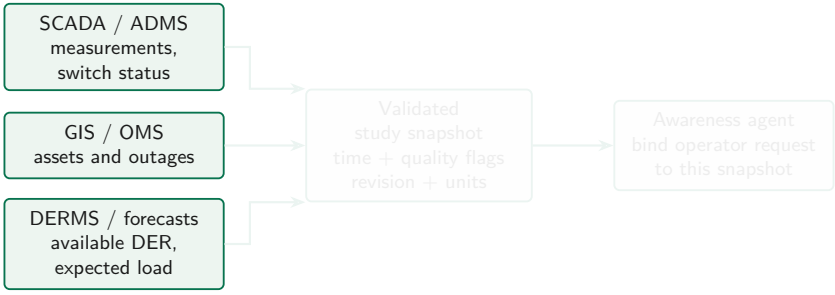



The assistant assembles an engineering recommendation. A human reviews the evidence in the context of operating procedures.

The tutorial is an advisory sandbox with an explicit human boundary.

Governance-aware TSO digital twin (2026); Executed tutorial: Classroom_Labs, L0–L12

# The study starts with a consistent operating snapshot



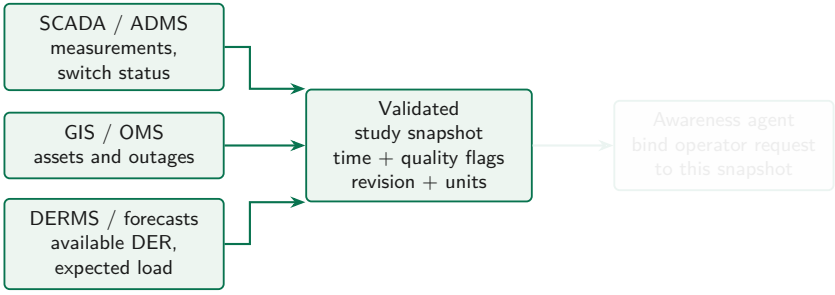

SCADA: supervisory control and data acquisition; ADMS: advanced distribution management system.
GIS: geographic information system; OMS: outage management system; DERMS: distributed energy resource management system.

**Predict:** What if the switch status and model describe different network revisions?

**Explanation:** Hold the study and reconcile the snapshot before calculating a recommendation.

Proposed integration, not implemented live. NLR ADMS Test Bed; tutorial L1 and L7 demonstrate input/evidence binding.

# The study starts with a consistent operating snapshot



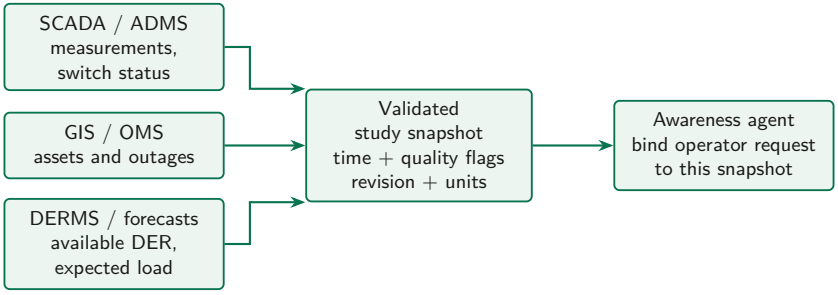

SCADA: supervisory control and data acquisition; ADMS: advanced distribution management system. GIS: geographic information system; OMS: outage management system; DERMS: distributed energy resource management system.

**Predict:** What if the switch status and model describe different network revisions?

**Explanation:** Hold the study and reconcile the snapshot before calculating a recommendation.

Proposed integration, not implemented live. NLR ADMS Test Bed; tutorial L1 and L7 demonstrate input/evidence binding.

# The study starts with a consistent operating snapshot



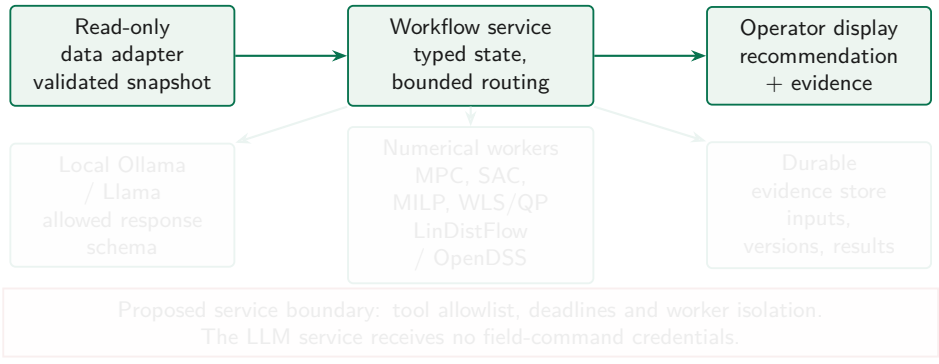

SCADA: supervisory control and data acquisition; ADMS: advanced distribution management system.
GIS: geographic information system; OMS: outage management system; DERMS: distributed energy resource management system.

**Predict:** What if the switch status and model describe different network revisions?

**Explanation:** Hold the study and reconcile the snapshot before calculating a recommendation.

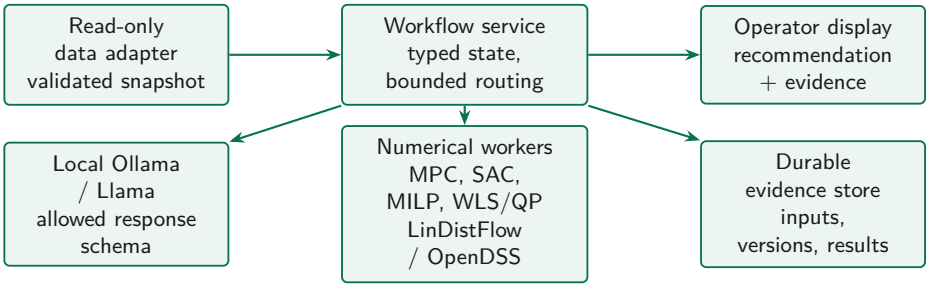
Proposed integration, not implemented live. NLR ADMS Test Bed; tutorial L1 and L7 demonstrate input/evidence binding.

# The assistant runs beside the existing control system



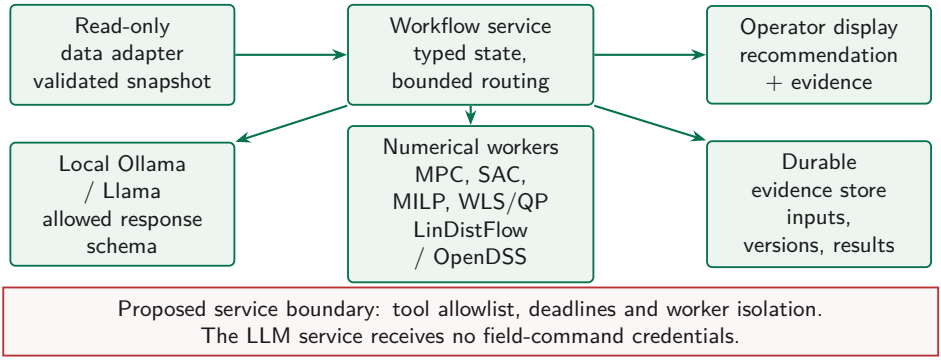

Proposed architecture. NIST SP 800-82r3, §5.2.3.1; LangGraph persistence; tutorial G0–G5 and L0–L12.

# The assistant runs beside the existing control system




Proposed architecture. NIST SP 800-82r3, §5.2.3.1; LangGraph persistence; tutorial G0–G5 and L0–L12.

# The assistant runs beside the existing control system




Proposed architecture. NIST SP 800-82r3, §5.2.3.1; LangGraph persistence; tutorial G0–G5 and L0–L12.

# Voltage desk: an alarm becomes a request for a checked action

**Illustrative operator request**
"High PV is raising voltage. Study a feasible inverter setting for the full feeder."

Awareness: snapshot + full scope

↓

Planning: MPC / saved SAC candidates

↓

Simulation: LinDistFlow / OpenDSS

↓

Verification: voltage / current / scope

↓

LLM Optimizer: refine and recheck

**Evidence from the tutorial**

Same-action audit across 12 test cases:
MV check: 12/12 pass.
Full MV/LV AC check: 0/12 pass.

MV evidence cannot certify the full-feeder request.

**Operator display**
Status: HOLD for broader evidence.
Next study: repair and validate at full scope.
Examples: L5/L12; batch: paired-model pilot.

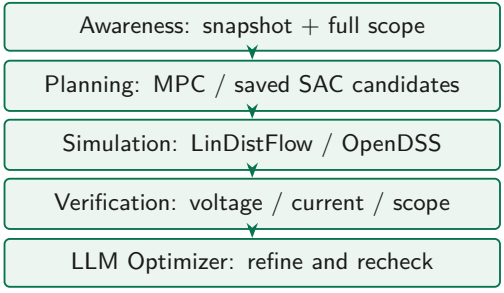
**Predict:** What would a useful assistant save the operator from doing manually?

Assistant: Assembling model scope and validating checks that are reusable record.

Illustrative scenario. 12-case audit: recorded paired-model pilot. One-case demo: L5; scope packet: L12. No field dispatch occurred.

# Voltage desk: an alarm becomes a request for a checked action

**Illustrative operator request**
"High PV is raising voltage. Study a feasible inverter setting for the full feeder."




**Evidence from the tutorial**
Same-action audit across 12 test cases:
MV check: 12/12 pass.
Full MV/LV AC check: 0/12 pass.
MV evidence cannot certify the full-feeder request.

**Operator display**
Status: HOLD for broader evidence.
Next study: repair and validate at full scope.
Examples: L5/L12; batch: paired-model pilot.

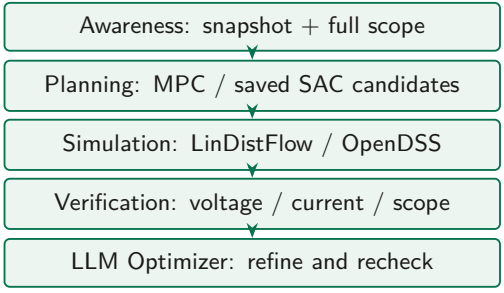
**Predict:** What would a useful assistant save the operator from doing manually?

**Explanation:** Assembling model scope and conflicting checks into one reviewable record.

Illustrative scenario. 12-case audit: recorded paired-model pilot. One-case demo: L5; scope packet: L12. No field dispatch occurred.

# Voltage desk: an alarm becomes a request for a checked action

**Illustrative operator request**
"High PV is raising voltage. Study a feasible inverter setting for the full feeder."




**Evidence from the tutorial**

Same-action audit across 12 test cases:
MV check: 12/12 pass.
Full MV/LV AC check: 0/12 pass.

MV evidence cannot certify the full-feeder request.

**Operator display**
Status: HOLD for broader evidence.
Next study: repair and validate at full scope.
Examples: L5/L12; batch: paired-model pilot.

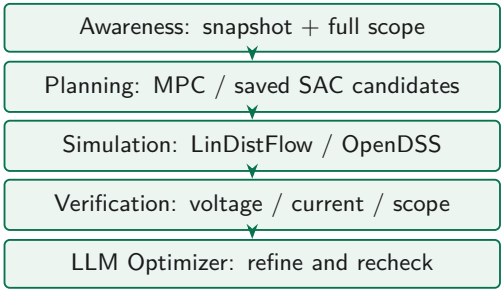
**Predict:** What would a useful assistant save the operator from doing manually?

**Explanation:** Assembling model scope and conflicting checks into one reviewable record.

Illustrative scenario. 12-case audit: recorded paired-model pilot. One-case demo: L5; scope packet: L12. No field dispatch occurred.

# Voltage desk: an alarm becomes a request for a checked action

**Illustrative operator request**
"High PV is raising voltage. Study a feasible inverter setting for the full feeder."




**Evidence from the tutorial**

Same-action audit across 12 test cases:
MV check: 12/12 pass.
Full MV/LV AC check: 0/12 pass.

MV evidence cannot certify the full-feeder request.

**Operator display**
Status: HOLD for broader evidence.
Next study: repair and validate at full scope.
Examples: L5/L12; batch: paired-model pilot.

**Predict:** What would a useful assistant save the operator from doing manually?

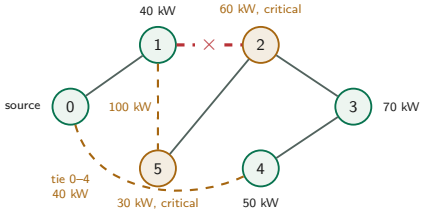
**Explanation:** Assembling model scope and conflicting checks into one reviewable record.

Illustrative scenario. 12-case audit: recorded paired-model pilot. One-case demo: L5; scope packet: L12. No field dispatch occurred.

# Restoration desk: the service priority is visible before review

**Illustrative operator request**

"The fault is isolated. Compare pickup plans that prioritize the two critical loads."




Six-node fixture; isolation is assumed in this story.

| Plan | Total kW | Critical kW |
|---|---|---|
| Unweighted | 140 | 30 |
| Weighted | 130 | 90 |

**Proposed operator display**

Requested policy: critical-load weights.
Candidate: weighted MILP solution.
Next: engineering and switching review.

Awareness: status. Planning: MILP. Simulation: candidate network. Verification: limits and evidence. LLM Optimizer: request a permitted refinement.

**Predict:** Does the higher critical service prove an LLM advantage?

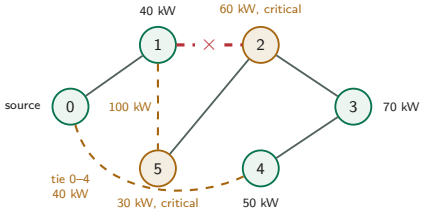
**Explanation:** No. It follows from the objective. A fixed workflow with the same MILP can achieve it.

Recorded Classroom_Labs L6–L7. Active-power transport model, not an AC switching certification. NLR ADMS Test Bed illustrates a route to lab integration.

# Restoration desk: the service priority is visible before review

**Illustrative operator request**

"The fault is isolated. Compare pickup plans that prioritize the two critical loads."




Six-node fixture; isolation is assumed in this story.

| Plan | Total kW | Critical kW |
|---|---|---|
| Unweighted | 140 | 30 |
| Weighted | 130 | 90 |

**Proposed operator display**

Requested policy: critical-load weights.
Candidate: weighted MILP solution.
Next: engineering and switching review.

Awareness: status. Planning: MILP. Simulation: candidate network. Verification: limits and evidence. LLM Optimizer: request a permitted refinement.

**Predict:** Does the higher critical service prove an LLM advantage?

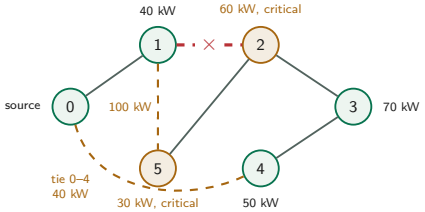
**Explanation:** No. It follows from the objective. A fixed workflow with the same MILP can achieve it.

Recorded Classroom_Labs L6–L7. Active-power transport model, not an AC switching certification. NLR ADMS Test Bed illustrates a route to lab integration.

# Restoration desk: the service priority is visible before review

**Illustrative operator request**

"The fault is isolated. Compare pickup plans that prioritize the two critical loads."




Six-node fixture; isolation is assumed in this story.

| Plan | Total kW | Critical kW |
|---|---|---|
| Unweighted | 140 | 30 |
| Weighted | 130 | 90 |

**Proposed operator display**

Requested policy: critical-load weights.
Candidate: weighted MILP solution.
Next: engineering and switching review.

Awareness: status. Planning: MILP. Simulation: candidate network. Verification: limits and evidence. LLM Optimizer: request a permitted refinement.

**Predict:** Does the higher critical service prove an LLM advantage?

**Explanation:** No. It follows from the objective. A fixed workflow with the same MILP can achieve it.

Recorded Classroom_Labs L6–L7. Active-power transport model, not an AC switching certification. NLR ADMS Test Bed illustrates a route to lab integration.

# A useful Llama role: understand the operator's restriction

**Saved operator request h06**

"Use the 0.25 km bypass to reconnect demand, with current no higher than **100 A**. Keep the 2 km bypass **open** and leave PV generation at **zero**."

The request restricts a permitted study; it cannot remove fault quarantine.

interpret

**Actual saved Llama fields**

```
{
  "status": "READY",
  "rating_fraction": 1.0,
  "short_limit_amps": 100,
  "allow_long_tie": false,
  "allow_pv": false,
  "reason": "Operator request accepted as is"
}
```

`READY` is an interpretation status; numerical feasibility still requires a separate check.

| Seven authored requests (post-hoc subset) | Documented parser | Parser + Llama |
|---|---|---|
| Clear requests completed correctly | 3/5 | 5/5 |
| Missing/prohibited requests clarified | 2/2 | 2/2 |
| Actual Llama calls across this subset | 0 | 4 |

**Saved h06 call: 3.160 s.** Historical feeder catalog + OpenDSS, separate from L6's six-node MILP.
The observed gain is request interpretation against this parser; better optimization and operator-time savings were not established.

Actual saved h06 request, schema, response and client duration; Recorded restoration comparison: five clear and two clarification cases.

# A useful Llama role: understand the operator's restriction

**Saved operator request h06**

"Use the 0.25 km bypass to reconnect demand, with current no higher than **100 A**. Keep the 2 km bypass **open** and leave PV generation at **zero**."

The request restricts a permitted study; it cannot remove fault quarantine.

interpret →

**Actual saved Llama fields**

```
{
  "status": "READY",
  "rating_fraction": 1.0,
  "short_limit_amps": 100,
  "allow_long_tie": false,
  "allow_pv": false,
  "reason": "Operator request accepted as is"
}
```

`READY` is an interpretation status; numerical feasibility still requires a separate check.

| Seven authored requests (post-hoc subset) | Documented parser | Parser + Llama |
|---|---|---|
| Clear requests completed correctly | 3/5 | 5/5 |
| Missing/prohibited requests clarified | 2/2 | 2/2 |
| Actual Llama calls across this subset | 0 | 4 |

**Saved h06 call: 3.160 s.** Historical feeder catalog + OpenDSS, separate from L6's six-node MILP.
The observed gain is request interpretation against this parser; better optimization and operator-time savings were not established.

Actual saved h06 request, schema, response and client duration; Recorded restoration comparison: five clear and two clarification cases.

# A useful Llama role: understand the operator's restriction

**Saved operator request h06**

"Use the 0.25 km bypass to reconnect demand, with current no higher than **100 A**. Keep the 2 km bypass **open** and leave PV generation at **zero**."

The request restricts a permitted study; it cannot remove fault quarantine.

interpret

**Actual saved Llama fields**

```
{
  "status": "READY",
  "rating_fraction": 1.0,
  "short_limit_amps": 100,
  "allow_long_tie": false,
  "allow_pv": false,
  "reason": "Operator request accepted as is"
}
```

`READY` is an interpretation status; numerical feasibility still requires a separate check.

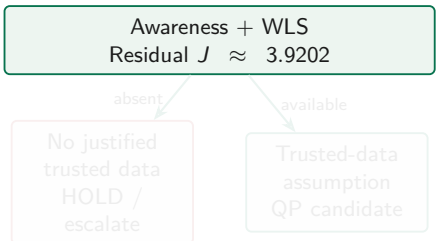
| Seven authored requests (post-hoc subset) | Documented parser | Parser + Llama |
|---|---|---|
| Clear requests completed correctly | 3/5 | 5/5 |
| Missing/prohibited requests clarified | 2/2 | 2/2 |
| Actual Llama calls across this subset | 0 | 4 |

**Saved h06 call: 3.160 s.** Historical feeder catalog + OpenDSS, separate from L6's six-node MILP.
The observed gain is request interpretation against this parser; better optimization and operator-time savings were not established.

Actual saved h06 request, schema, response and client duration; Recorded restoration comparison: five clear and two clarification cases.

# Cyber desk: a quiet residual still needs a trustworthy basis

**Illustrative operator request**
"A power-flow estimate looks plausible, but measurements may be corrupted. What evidence supports a correction?"



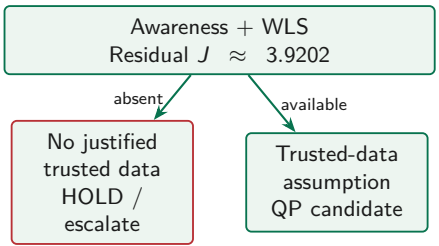

**Predict:** Can an LLM explanation establish that a sensor is trustworthy?

Recorded Classroom_Labs L8–L9 and archived trace. Liu, Ning & Reiter, CCS 2009; protected angles are assumed trusted in this teaching model.

# Cyber desk: a quiet residual still needs a trustworthy basis

**Illustrative operator request**

"A power-flow estimate looks plausible, but measurements may be corrupted. What evidence supports a correction?"




**Predict:** Can an LLM explanation establish that a sensor is trustworthy?

**Explanation:** No. Provenance and independent operational evidence must support that assumption.

**Recorded four-bus result**

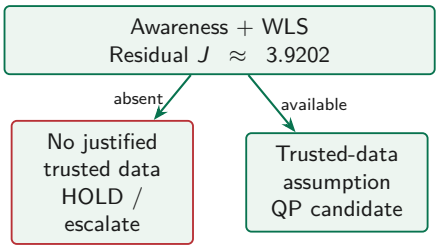
QP injection: $u \approx 0.12169$ p.u.
Simulated true branch flow:
$0.17046 \leq 0.18$ p.u.
Fixed and Llama supervisors return the same final $u$.

Planning requests the constrained correction. Simulation predicts its consequence. Verification checks flow and the declared data assumption. The LLM Optimizer can retain or refine only within those conditions.

Recorded Classroom_Labs L8–L9 and archived trace. Liu, Ning & Reiter, CCS 2009; protected angles are assumed trusted in this teaching model.

# Cyber desk: a quiet residual still needs a trustworthy basis

**Illustrative operator request**
"A power-flow estimate looks plausible, but measurements may be corrupted. What evidence supports a correction?"




**Recorded four-bus result**

QP injection: $u \approx 0.12169$ p.u.
Simulated true branch flow:
$0.17046 \leq 0.18$ p.u.
Fixed and Llama supervisors return the same final $u$.

Planning requests the constrained correction. Simulation predicts its consequence. Verification checks flow and the declared data assumption. The LLM Optimizer can retain or refine only within those conditions.

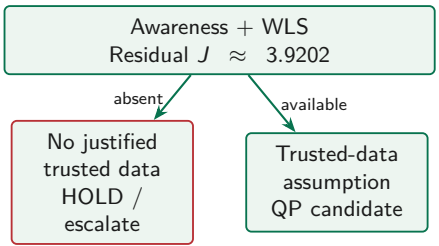
**Predict:** Can an LLM explanation establish that a sensor is trustworthy?

**Explanation:** No. Provenance and independent operational evidence must support that assumption.

Recorded Classroom_Labs L8–L9 and archived trace. Liu, Ning & Reiter, CCS 2009; protected angles are assumed trusted in this teaching model.

# Cyber desk: a quiet residual still needs a trustworthy basis

**Illustrative operator request**
"A power-flow estimate looks plausible, but measurements may be corrupted. What evidence supports a correction?"




**Recorded four-bus result**

QP injection: $u \approx 0.12169$ p.u.
Simulated true branch flow:
$0.17046 \leq 0.18$ p.u.
Fixed and Llama supervisors return the same final $u$.

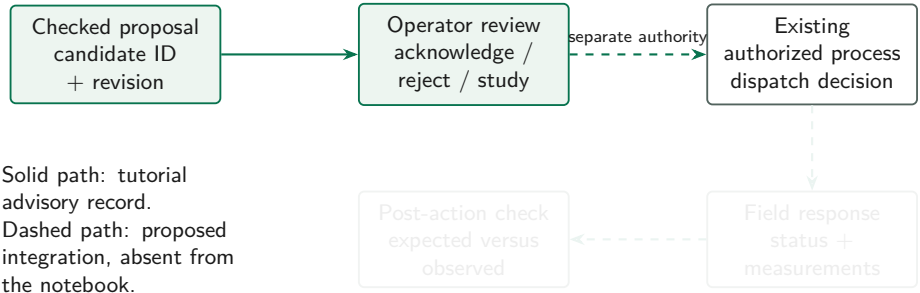
Planning requests the constrained correction. Simulation predicts its consequence. Verification checks flow and the declared data assumption. The LLM Optimizer can retain or refine only within those conditions.

**Predict:** Can an LLM explanation establish that a sensor is trustworthy?

**Explanation:** No. Provenance and independent operational evidence must support that assumption.

Recorded Classroom_Labs L8–L9 and archived trace. Liu, Ning & Reiter, CCS 2009; protected angles are assumed trusted in this teaching model.

# Operator acknowledgment and field execution are different events



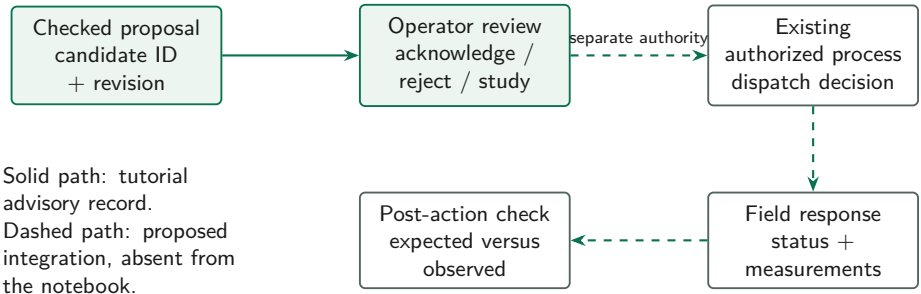

Solid path: tutorial advisory record.
Dashed path: proposed integration, absent from the notebook.

**Predict:** If the device does not reach the expected state, should the assistant mark the job complete?

**Explanation:** No. Preserve the discrepancy and hand it to the established operating procedure.

Proposed integration and tutorial L12 boundary. NIST AI RMF: GOVERN 3.2, MANAGE 2.4 and 4.1.

# Operator acknowledgment and field execution are different events

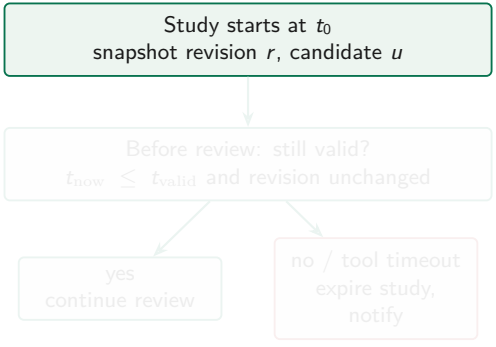



**Predict:** If the device does not reach the expected state, should the assistant mark the job complete?

**Explanation:** No. Preserve the discrepancy and hand it to the established operating procedure.

Proposed integration and tutorial L12 boundary. NIST AI RMF: GOVERN 3.2, MANAGE 2.4 and 4.1.

# Slow advice must not delay the existing control response




**Measured example**

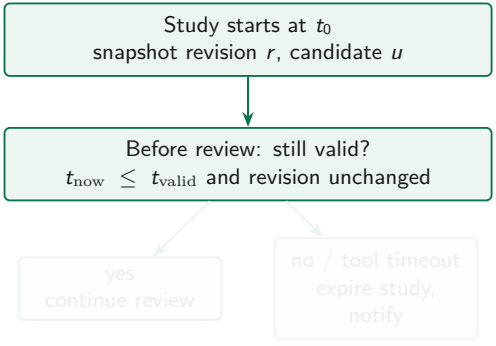
One local Llama call took 34.11 s, including 16.97 s loading. This is one measurement, not a service guarantee.

$t_{\text{valid}}$: site-defined evidence expiry.
$r$: network/data revision.
$u$: proposed control vector.

Design requirement: existing protection and approved controllers continue independently. A failed advisory service must not block them.

Recorded L11 call; Ollama timing fields. Proposed expiry/failure handling informed by NIST SP 800-82r3, OT performance and availability.

# Slow advice must not delay the existing control response




**Measured example**

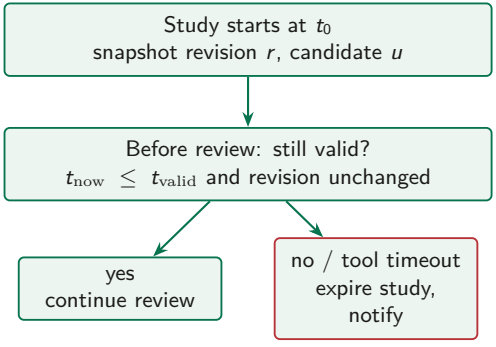
One local Llama call took 34.11 s, including 16.97 s loading. This is one measurement, not a service guarantee.

$t_{\text{valid}}$: site-defined evidence expiry.
$r$: network/data revision.
$u$: proposed control vector.

Design requirement: existing protection and approved controllers continue independently. A failed advisory service must not block them.

Recorded L11 call; Ollama timing fields. Proposed expiry/failure handling informed by NIST SP 800-82r3, OT performance and availability.

# Slow advice must not delay the existing control response




**Measured example**

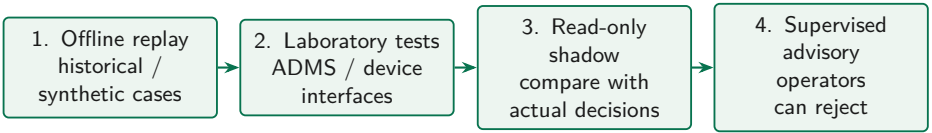
One local Llama call took 34.11 s, including 16.97 s loading. This is one measurement, not a service guarantee.

$t_{\text{valid}}$: site-defined evidence expiry.
$r$: network/data revision.
$u$: proposed control vector.

Design requirement: existing protection and approved controllers continue independently. A failed advisory service must not block them.

Recorded L11 call; Ollama timing fields. Proposed expiry/failure handling informed by NIST SP 800-82r3, OT performance and availability.

# A deployment pilot earns authority through measured evidence



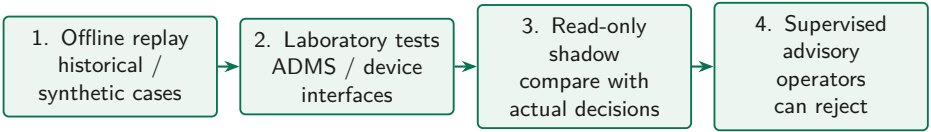

**Compare against the same approved fixed workflow**

Correct studies selected; constraint violations missed; stale evidence accepted; operator decision time; rejected recommendations; median and 95th-percentile total latency.

Define acceptance criteria before the pilot.
Include outages, bad data and unavailable tools.
Retain the fixed workflow when the assistant adds no measured benefit.

Proposed staged pilot, not a mandated sequence. NLR ADMS Test Bed; NIST AI RMF MEASURE 2.3; DOE Grid Control and Data Science.

# A deployment pilot earns authority through measured evidence

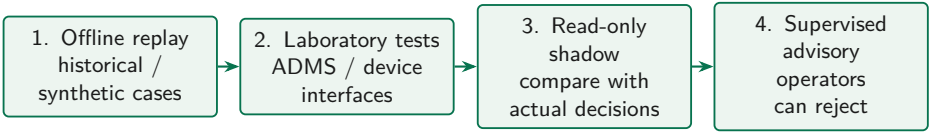



**Compare against the same approved fixed workflow**
Correct studies selected; constraint violations missed; stale evidence accepted; operator decision time; rejected recommendations; median and 95th-percentile total latency.

Define acceptance criteria before the pilot.
Include outages, bad data and unavailable tools.
Retain the fixed workflow when the assistant adds no measured benefit.

Proposed staged pilot, not a mandated sequence. NLR ADMS Test Bed; NIST AI RMF MEASURE 2.3; DOE Grid Control and Data Science.

# A deployment pilot earns authority through measured evidence




**Compare against the same approved fixed workflow**
Correct studies selected; constraint violations missed; stale evidence accepted; operator decision time; rejected recommendations; median and 95th-percentile total latency.

Define acceptance criteria before the pilot.
Include outages, bad data and unavailable tools.
Retain the fixed workflow when the assistant adds no measured benefit.

Proposed staged pilot, not a mandated sequence. NLR ADMS Test Bed; NIST AI RMF MEASURE 2.3; DOE Grid Control and Data Science.

# A four-minute rehearsal connects the desk to the notebook

| Control-room event | Notebook evidence | Operator's explanation |
|---|---|---|
| Voltage alert | L5 model audit; L12 scope mismatch | Why does an MV pass leave a full-feeder request on HOLD? |
| Restoration request | L6 service tradeoff; L7 changed rating | Which objective was requested, and does the old evidence still apply? |
| Suspect telemetry | L8 residual; L9 trusted-data QP | What supports the measurement assumption before any corrective recommendation? |

**Work in pairs**

One person acts as operator, the other as the assistant. Choose one row, inspect the saved output, and state: decision, missing evidence, next tool or responsible person.

**Instructor check:** A clear response cites the numerical result and its scope. It can legitimately end with a hold or a request for another study.

Tutorial Classroom_Labs L5–L9 and L12. Rehearsal is educational; the saved outputs do not certify a real operating decision.

# A four-minute rehearsal connects the desk to the notebook

| Control-room event | Notebook evidence | Operator's explanation |
|---|---|---|
| Voltage alert | L5 model audit; L12 scope mismatch | Why does an MV pass leave a full-feeder request on HOLD? |
| Restoration request | L6 service tradeoff; L7 changed rating | Which objective was requested, and does the old evidence still apply? |
| Suspect telemetry | L8 residual; L9 trusted-data QP | What supports the measurement assumption before any corrective recommendation? |

**Work in pairs**

One person acts as operator, the other as the assistant. Choose one row, inspect the saved output, and state: decision, missing evidence, next tool or responsible person.

**Instructor check:** A clear response cites the numerical result and its scope. It can legitimately end with a hold or a request for another study.

Tutorial Classroom_Labs L5–L9 and L12. Rehearsal is educational; the saved outputs do not certify a real operating decision.

# A four-minute rehearsal connects the desk to the notebook

| Control-room event | Notebook evidence | Operator's explanation |
|---|---|---|
| Voltage alert | L5 model audit; L12 scope mismatch | Why does an MV pass leave a full-feeder request on HOLD? |
| Restoration request | L6 service tradeoff; L7 changed rating | Which objective was requested, and does the old evidence still apply? |
| Suspect telemetry | L8 residual; L9 trusted-data QP | What supports the measurement assumption before any corrective recommendation? |

**Work in pairs**

One person acts as operator, the other as the assistant. Choose one row, inspect the saved output, and state: decision, missing evidence, next tool or responsible person.

**Instructor check:** A clear response cites the numerical result and its scope. It can legitimately end with a hold or a request for another study.

Tutorial Classroom_Labs L5–L9 and L12. Rehearsal is educational; the saved outputs do not certify a real operating decision.

# The operator sees scope before readback

**Saved recommendation ID:** `b67cff9c8eb444c2`

| Packet field | Full MV/LV request | Explicit MV teaching request |
|---|---|---|
| Verified scope | MV phases only | MV phases only |
| Scope matches | No | Yes |
| State | HOLD | READY_FOR_READBACK |
| Acknowledged | False | True |
| Equipment dispatch | False | False |

**Predict:** The numerical evidence is the same. Why is only the second packet eligible?

**Explanation:** The second request asks only for the scope that was checked. The first asks for more. Acknowledgment still does not dispatch equipment.

# The operator sees scope before readback

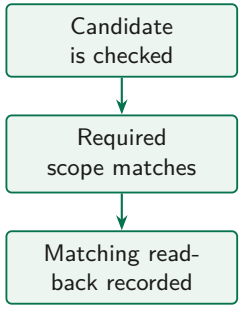
**Saved recommendation ID:** `b67cff9c8eb444c2`

| Packet field | Full MV/LV request | Explicit MV teaching request |
|---|---|---|
| Verified scope | MV phases only | MV phases only |
| Scope matches | No | Yes |
| State | HOLD | READY_FOR_READBACK |
| Acknowledged | False | True |
| Equipment dispatch | False | False |

**Predict:** The numerical evidence is the same. Why is only the second packet eligible?

**Explanation:** The second request asks only for the scope that was checked. The first asks for more. Acknowledgment still does not dispatch equipment.

# Acknowledgment is a guarded transition




**Only a matching, eligible packet can be acknowledged.**

- Proposed → checked
- Checked → awaiting review
- Matching readback → acknowledged

Executed tutorial: Classroom_Labs, L0–L12; Executed LangGraph teaching lab: G0–G5; LangGraph: interrupts and resume

# Code: bind review to the candidate

**Notebook L12** Exact notebook excerpt; run the complete cell in order.

```
wrong_id = lab.acknowledge(mv_scope, 'wrong-id', 'Student operator')
readback = lab.acknowledge(mv_scope, mv_scope['recommendation_id'], 'Student operator')
```

## What the code means

A wrong candidate ID cannot acknowledge the packet.

- First request uses a wrong ID
- Second reads back the packet ID
- The operator label is a classroom placeholder

Executed tutorial: Classroom_Labs, L0–L12

# Hands-on: read back, reject, inspect

**Four-minute notebook exercise**

Use L12 to distinguish scope, identity and acknowledgment.

1. Run L12 after its earlier cells
2. Find the full-scope HOLD
3. Inspect the wrong-ID rejection
4. Confirm no equipment dispatch

**Predict:** Which of the three outcomes is permission to dispatch equipment?

**Explanation:** None. Even the acknowledged packet is explicitly advisory.

Executed tutorial: Classroom_Labs, L0–L12

# Hands-on: read back, reject, inspect

**Four-minute notebook exercise**

Use L12 to distinguish scope, identity and acknowledgment.

1. Run L12 after its earlier cells
2. Find the full-scope HOLD
3. Inspect the wrong-ID rejection
4. Confirm no equipment dispatch

**Predict:** Which of the three outcomes is permission to dispatch equipment?

**Explanation:** None. Even the acknowledged packet is explicitly advisory.

Executed tutorial: Classroom_Labs, L0–L12

# The checks distinguish three outcomes

**Notebook L12** Exact notebook excerpt; run the complete cell in order.

```
assert full_scope['state'] == 'HOLD' and not full_scope['acknowledged']
assert wrong_id['acknowledgment_result'] == 'REJECTED'
assert readback['acknowledged'] and not readback['equipment_dispatch']
```

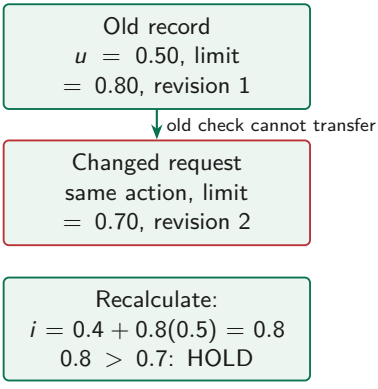
## What the code means

Scope mismatch holds; wrong identity rejects; valid readback remains advisory.

- Full MV/LV requirement: HOLD
- Wrong identifier: REJECTED
- Matching MV readback: acknowledged

Executed tutorial: Classroom_Labs, L0–L12

# A smaller current limit changes the decision




**A hash links content; it does not establish physical truth.**

$u$ is the proposed control; $i$ is the toy current. This uses the opening scalar model, not a feeder calculation.

- Change a limit or candidate
- Old binding no longer matches
- Recompute the affected study

**Predict:** Can the old acknowledgment survive a changed control?

Executed tutorial: Classroom_Labs, L0–L12

# Capstone: approve review, hold or request a study?

**Work with a partner**

Name the missing evidence and the next responsible step.

1. A: MV PASS; full-feeder AC FAIL
2. B: restoration PASS; isolation record absent
3. C: no alarm; protected measurements absent

**Predict:** For your card, identify one missing fact and the next tool or human step.

**Explanation:** A: broader-model repair. B: authorized isolation evidence. C: justified independent measurements or escalation.

Executed tutorial: Classroom_Labs, L0–L12; Liu, Ning & Reiter, false-data injection, CCS 2009

# Capstone: approve review, hold or request a study?

**Work with a partner**

Name the missing evidence and the next responsible step.

1. A: MV PASS; full-feeder AC FAIL
2. B: restoration PASS; isolation record absent
3. C: no alarm; protected measurements absent

**Predict:** For your card, identify one missing fact and the next tool or human step.

**Explanation:** A: broader-model repair. B: authorized isolation evidence. C: justified independent measurements or escalation.

Executed tutorial: Classroom_Labs, L0–L12; Liu, Ning & Reiter, false-data injection, CCS 2009

# Five stages, three cases, one evidence discipline

| Stage | Voltage | Restoration | Cyber response |
|---|---|---|---|
| Awareness | Operating point and limits | Fault, ties, load priorities | Measurements and trust assumptions |
| Planning | MPC; SAC candidate | Weighted pickup MILP | Constrained recovery QP |
| Simulation | Network response | Flow on selected paths | Estimated / evaluated flows |
| Verification | Voltage/current and scope | Capacity, connectivity, binding | Residual and protected-data basis |
| LLM Optimizer | Propose a permitted refinement | Explain or request a revised trial | Explain or request additional evidence |

The numerical method changes with the decision. The record passed between stages keeps the action, assumptions and results together.

**Predict:** Which stage would catch a stale network revision?

**Explanation:** The evidence-binding check in Verification; a fluent summary cannot repair the mismatch.

Executed tutorial: Classroom_Labs, L0–L12; Recorded voltage pilot: E1–E8

# Five stages, three cases, one evidence discipline

| Stage | Voltage | Restoration | Cyber response |
|---|---|---|---|
| Awareness | Operating point and limits | Fault, ties, load priorities | Measurements and trust assumptions |
| Planning | MPC; SAC candidate | Weighted pickup MILP | Constrained recovery QP |
| Simulation | Network response | Flow on selected paths | Estimated / evaluated flows |
| Verification | Voltage/current and scope | Capacity, connectivity, binding | Residual and protected-data basis |
| LLM Optimizer | Propose a permitted refinement | Explain or request a revised trial | Explain or request additional evidence |

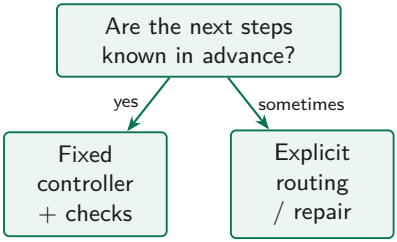
The numerical method changes with the decision. The record passed between stages keeps the action, assumptions and results together.

**Predict:** Which stage would catch a stale network revision?

**Explanation:** The evidence-binding check in Verification; a fluent summary cannot repair the mismatch.

Executed tutorial: Classroom_Labs, L0–L12; Recorded voltage pilot: E1–E8

# Choose the simplest sufficient workflow




Add language interpretation only where requests or evidence require it. Test whether it improves a measured outcome.

**Add an agent only when it addresses a measured task need.**

- Repeated calculation → fixed workflow
- Varied requests → typed LLM interface
- Independent studies → specialist coordination

Kim et al., scaling agent systems (2026); Executed LangGraph teaching lab: G0–G5

# A reproducible result carries its context

**Experiment record**

Scenario: load, topology, revision
Candidate: control values + ID
Model: equations + network scope
Checks: margins and failed limits
Time: solver, LLM, full workflow
Decision: accept for review / HOLD

**Archive the inputs, methods, outputs and evidence mode together.**

- Case, model scope and data revision
- Solver, policy and LLM versions
- Seeds, budgets and measured durations

Executed tutorial: Classroom_Labs, L0–L12; Recorded voltage pilot: E1–E8; Ollama chat API: timing fields

# Design your next experiment

**Work with a partner**

Change one factor and state what would falsify your claim.

1. Hypothesis: name the expected benefit
2. Control: retain a strong fixed baseline
3. Measure: outcome, failures and latency
4. Boundary: state the model and task scope

**Predict:** Write one testable benefit and one result that would refute it.

**Explanation:** Example: fewer unsupported reports at equal tool budget; equal or higher unsupported-report rate refutes the claimed benefit.

PowerAgentBench-SS (2026), §§II–III; Kim et al., scaling agent systems (2026)

# Design your next experiment

**Work with a partner**

Change one factor and state what would falsify your claim.

1. Hypothesis: name the expected benefit
2. Control: retain a strong fixed baseline
3. Measure: outcome, failures and latency
4. Boundary: state the model and task scope

**Predict:** Write one testable benefit and one result that would refute it.

**Explanation:** Example: fewer unsupported reports at equal tool budget; equal or higher unsupported-report rate refutes the claimed benefit.

PowerAgentBench-SS (2026), §§II–III; Kim et al., scaling agent systems (2026)

# Exit ticket: explain, run, interpret

**Work with a partner**

Connect one concept, one cell and one observed result.

1. Explain a DAG versus a bounded cycle
2. Name a model-based and model-free method
3. Interpret one failed candidate correctly
4. Identify the operator acknowledgment boundary

**Predict:** Which checkpoint would you show a colleague to explain the framework?

**Explanation:** Choose a checkpoint and support the explanation with its exact inputs, output and stated limitation.

Executed LangGraph teaching lab: G0–G5; Executed tutorial: Classroom_Labs, L0–L12

# Exit ticket: explain, run, interpret

**Work with a partner**

Connect one concept, one cell and one observed result.

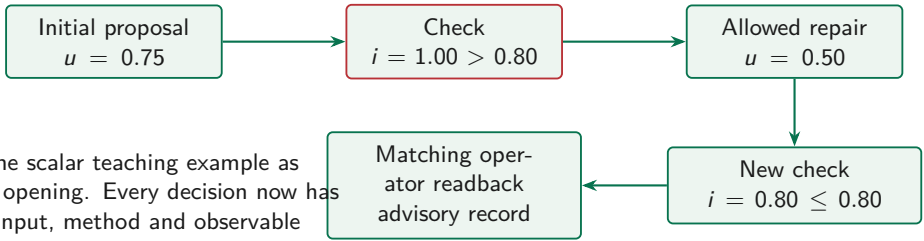
1. Explain a DAG versus a bounded cycle
2. Name a model-based and model-free method
3. Interpret one failed candidate correctly
4. Identify the operator acknowledgment boundary

**Predict:** Which checkpoint would you show a colleague to explain the framework?

**Explanation:** Choose a checkpoint and support the explanation with its exact inputs, output and stated limitation.

Executed LangGraph teaching lab: G0–G5; Executed tutorial: Classroom_Labs, L0–L12

# From request to a reviewable recommendation




Same scalar teaching example as the opening. Every decision now has an input, method and observable output.

Agentic assistance earns trust through useful tools, explicit evidence and accountable decisions.

Executed tutorial: Classroom_Labs, L0–L12; Executed LangGraph teaching lab: G0–G5; Recorded voltage pilot: E1–E8